

CORE COMPUTER SCIENCE · DATA STRUCTURES

Everything About Linked Lists

A Complete Study & Handnote Reference

All 88 Topics · 12 Sections · C++ / DSA Handnote

FOUNDATION · OPERATIONS · VARIANTS · ALGORITHMS · DESIGN · SAFETY

Contents

PART I — STRUCTURE AND OPERATIONS

SECTION 1 — FOUNDATION	9
1. Introduction to Linked List	9
2. Why Linked Lists Matter	10
3. Node Structure	11
4. Array vs Linked List	13
5. Types of Linked Lists	13
6. Linked List Terminology	15
SECTION 2 — MEMORY & INTERNAL WORKING	17
7. Heap Allocation per Node	17
8. Non-Contiguous Memory Layout	18
9. Pointer Overhead	19
10. Cache Behavior	19
11. Allocation and Deallocation Cost	21
PRACTICE SET — Topics 1-11	23
ANSWER KEY	24
HANDNOTE SUMMARY CARD	26
SECTION 3 — SINGLY LINKED LIST OPERATIONS	27
12. Creating a Node	27
13. Building a List	28
14. Traversal	29
15. Recursive Traversal	29
16. Insert at Head	30
17. Insert at Tail	31
18. Insert at Position	32
19. Insert in Sorted List	33
20. Delete Head	34
21. Delete Tail	34
22. Delete at Position	36
23. Delete by Value	37
24. Delete Entire List	38
25. Search an Element	39
26. Length of List	40
27. Update a Node Value	41
28. Get Nth Node	41
PRACTICE SET — Topics 12-28	43
ANSWER KEY	45
HANDNOTE SUMMARY CARD	47
CONSOLIDATED CHEATSHEET — PART I	48

PART II — VARIANTS

SECTION 4 — REVERSAL	51
29. Reverse Iteratively	51

30. Reverse Recursively	52
31. Reverse in Groups of K	53
32. Reverse Sublist (m to n)	54
33. Reverse Alternate K Nodes	54
PRACTICE SET — Topics 29-33	56
ANSWER KEY	58
HANDNOTE SUMMARY CARD	60
SECTION 5 — DOUBLY LINKED LIST	61
34. DLL Node Structure	61
35. DLL Insertion	61
36. DLL Deletion	63
37. Forward and Backward Traversal	64
38. Reverse a DLL	64
39. DLL Advantages and Costs	66
SECTION 6 — CIRCULAR LINKED LIST	67
40. Circular Singly Linked List	67
41. Circular Doubly Linked List	68
42. Circular Insertion and Deletion	69
43. Traversal Termination	70
44. Josephus Problem	71
45. Split a Circular List	72
PRACTICE SET — Topics 34-45	74
ANSWER KEY	76
HANDNOTE SUMMARY CARD	78
CONSOLIDATED CHEATSHEET — PART II	79
PART III — ALGORITHMS	
SECTION 7 — TWO-POINTER PATTERNS	83
46. Slow and Fast Pointers	83
47. Find Middle Node	84
48. Nth Node from End	84
49. Remove Nth from End	85
50. Cycle Detection (Floyd)	86
51. Find Cycle Start Node	87
52. Find Cycle Length	89
53. Remove Cycle	89
54. Palindrome Linked List	90
55. Intersection of Two Lists	92
PRACTICE SET — Topics 46-55	94
ANSWER KEY	96
HANDNOTE SUMMARY CARD	98
SECTION 8 — SORTING & MERGING	99
56. Merge Two Sorted Lists	99
57. Merge Sort on Linked List	100
58. Insertion Sort on Linked List	101

59. Quick Sort on Linked List	102
60. Merge K Sorted Lists	103
61. Sort 0s, 1s and 2s	104
62. Check if List is Sorted	106
PRACTICE SET — Topics 56-62	107
ANSWER KEY	109
HANDNOTE SUMMARY CARD	111
CONSOLIDATED CHEATSHEET — PART III	112

PART IV — REARRANGEMENT AND DESIGN

SECTION 9 — REARRANGEMENT PROBLEMS	115
63. Rotate List by K	115
64. Odd-Even Node Split	116
65. Segregate Even and Odd Values	117
66. Reorder List	117
67. Swap Nodes in Pairs	118
68. Swap Kth from Start and End	119
69. Partition List Around Value	120
70. Remove Duplicates (Sorted)	121
71. Remove Duplicates (Unsorted)	123
72. Add Two Numbers as Lists	123
73. Flatten a Multilevel List	125
PRACTICE SET — Topics 63-73	127
ANSWER KEY	129
HANDNOTE SUMMARY CARD	131
SECTION 10 — ADVANCED VARIANTS	133
74. Dummy / Sentinel Node	133
75. Tail Pointer Optimization	134
76. Clone List with Random Pointer	134
77. LRU Cache (DLL + Hash Map)	136
78. XOR Linked List	138
79. Unrolled Linked List	139
80. Skip List	140
81. Static Linked List Using Arrays	141
PRACTICE SET — Topics 74-81	143
ANSWER KEY	145
HANDNOTE SUMMARY CARD	147
CONSOLIDATED CHEATSHEET — PART IV	148

PART V — TOOLS, SAFETY AND PRACTICE

SECTION 11 — C++ TOOLS & SAFETY	151
82. std::list	151
83. std::forward_list	152
84. Smart Pointers in Linked Lists	153
85. Common Bugs	154
86. Memory Leaks and Double Free	155

SECTION 12 — REAL-WORLD & INTERVIEW	159
87. Real-World Use Cases	159
88. Final Cheatsheet	160
PRACTICE SET — Topics 82-88	162
ANSWER KEY	164
HANDNOTE SUMMARY CARD	166
MASTER CHEATSHEET — ALL 88 TOPICS	167
AFTERWORD — WHERE TO GO NEXT	168

How to use this book: read a topic, close the book, and re-derive the pointer diagram on paper before looking at the code again. Every section ends with a practice set, a fully worked answer key, and a one-page summary card built for revision the night before an interview. The wide outer margin is deliberate — it is there for your own notes. Fill the Status column of your tracker as you go: blank / L (learning) / D (done and solved).

PART I

Structure and Operations

Foundation · Memory · Singly Linked List Operations — Topics 1-28

SECTION 1 — FOUNDATION

1. Introduction to Linked List

Definition

A **linked list** is a linear collection of elements in which each element is stored in its own separately allocated block of memory, called a **node**, and each node stores an explicit reference to the next node.

Two words carry all the weight:

- **Node** → one payload plus one or more links. It is allocated on its own, at its own address.
- **Link** → the address of the next node, stored *inside* the current node.

Why linked lists exist

An array must know its size before it is filled, and every element must sit immediately after the previous one. That single constraint — contiguity — is what makes insertion in the middle expensive: to make room at index 0 of an n -element array, all n elements must physically move.

```
// array: insert 5 at the front of {10,20,30,40}
// 10 20 30 40 _      shift right
// _ 10 20 30 40      four moves
// 5 10 20 30 40      then write

// linked list: insert 5 at the front
//      10 -> 20 -> 30 -> 40 -> null
// 5 -> 10 -> 20 -> 30 -> 40 -> null      one write, nothing moved
```

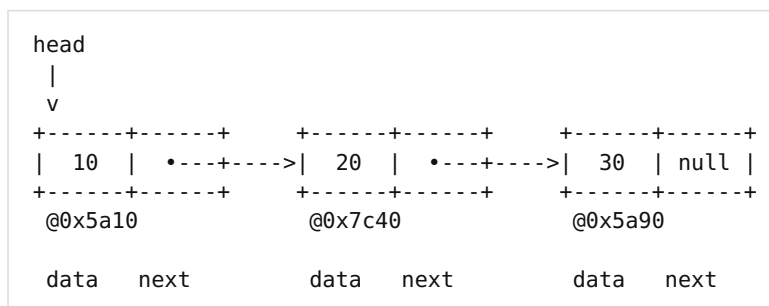
A linked list drops the contiguity requirement. Nodes may live anywhere in the heap, in any order, with arbitrary gaps between them. The *order of the list* is no longer the order of the addresses — it is whatever the **next** pointers say it is.

THE KEY INSIGHT (MEMORIZE THIS)

An array computes where the next element is. A linked list is *told* where the next element is.

Arrays trade flexibility for arithmetic: $\text{addr} = \text{base} + i \times \text{sizeof}(T)$, one multiply, one add, $O(1)$ to any element. Linked lists trade arithmetic for flexibility: there is no formula, so reaching element i costs i dereferences — but rearranging costs nothing but pointer writes.

How it actually looks in memory



Three things to read out of that picture, because every later topic depends on them:

1. The addresses are **unordered and unrelated**. `0x7c40` comes after `0x5a10` in list order but the third node sits back at `0x5a90`, below it.
2. The list is the `head` pointer. There is no separate list object. If you lose `head`, every node is unreachable and leaked.
3. `null` in the final `next` is the only end marker. There is no stored length, and no way to know you are at the last node except by looking at `next`.

Mental model

Think of a **treasure hunt**. Each clue is on a card in some random room of a large building. Each card gives you the answer plus the room number of the next card. You cannot jump to the seventh clue — nobody knows where it is except the sixth card. But you can insert a new clue anywhere in the hunt by rewriting exactly one room number.

An array, by contrast, is a row of numbered lockers bolted to one wall. Locker #7 is instantly reachable, but adding a locker between #3 and #4 means unbolting and re-bolting everything after it.

2. Why Linked Lists Matter

The four real advantages

1. Size is decided at runtime, one node at a time

A linked list grows by exactly one node per insertion. There is no capacity, no `reserve()`, no doubling, and no reallocation. Memory is requested only when an element actually arrives.

2. Insertion and deletion are $O(1)$ — *given a pointer to the position*

This is the most misquoted property in all of data structures. Read the qualifier carefully.

```
// You already hold Node* p. Insert after it:
Node* q = new Node{value, p->next};
p->next = q;                               //  $O(1)$  — two writes, nothing shifted

// You only know "position 5000". Insert there:
Node* p = head;
for (int i = 0; i < 4999; i++) p = p->next;    //  $O(n)$  — the walk dominates
```

INTERVIEW TRAP

"Linked list insertion is $O(1)$, array insertion is $O(n)$ " is only true when you are handed the node. Insertion *by index* is $O(n)$ in a linked list too — and in practice slower than the array's $O(n)$ shift, because `memmove` on contiguous bytes is far faster than chasing n scattered pointers. Say the qualifier out loud in interviews; it is exactly what the interviewer is listening for.

3. No reallocation, so pointers and references stay valid

When a `std::vector` grows past its capacity it allocates a new block, copies or moves every element, and frees the old block. Every pointer, reference and iterator into that vector becomes dangling. A linked list never moves an existing node, so a pointer to a node stays valid for the entire lifetime of that node, no matter how many other elements are inserted. This is the property that makes linked lists the backbone of an LRU cache, an intrusive kernel list, or a free list.

4. Splicing is free

Cutting a run of nodes out of one list and stitching it into another is a constant number of pointer writes regardless of how many nodes move. `std::list::splice` is $O(1)$ for a single element for exactly this reason; no equivalent exists for a vector.

The costs you accept in exchange

- **No random access.** `list[i]` does not exist and cannot exist. Anything requiring indexing, binary search, or sorting by index is off the table.
- **Memory overhead per element.** On 64-bit, an 8-byte pointer per node — often more than the payload itself (Topic 9).
- **Cache hostility.** Usually the dominant cost in practice (Topic 10).
- **Allocation cost.** One `new` per element (Topic 11).

WHEN TO ACTUALLY REACH FOR ONE

Use a linked list when you already hold pointers to the elements you will modify, when elements must never move, or when the structure is being spliced constantly. For "I need a resizable sequence," `std::vector` wins on nearly every real benchmark — including insertions in the middle, up to surprisingly large n .

3. Node Structure

The minimal definition

```
struct Node {
    int    data;
    Node*  next;
};
```

`Node* next` inside `struct Node` is legal even though `Node` is not yet a complete type: a *pointer* to an incomplete type has a known size (8 bytes), so the compiler can lay out the struct. A member of type `Node next`; would be an error — infinitely recursive size.

Adding a constructor (do this — it removes a whole class of bug)

```
struct Node {
    int    data;
    Node*  next;

    Node(int d) : data(d), next(nullptr) {}
    Node(int d, Node* n) : data(d), next(n) {}
};

Node* a = new Node(10);           // next is guaranteed null
Node* b = new Node(20, a);        // b -> a
```

THE #1 BEGINNER BUG

`Node* p = new Node;` on a struct without a constructor leaves `next` **uninitialized** — a garbage address, not null. Traversal then walks off into random memory and segfaults somewhere unrelated to the real mistake. Either write a constructor, or use `new Node{10, nullptr}`, or at minimum `new Node();` with parentheses (which value-initializes and zeroes the members).

The generic version

```
template <typename T>
struct Node {
    T      data;
    Node<T>* next;
    Node(const T& d, Node<T>* n = nullptr) : data(d), next(n) {}
};
```

What a node actually costs

```
struct Node { int data; Node* next; };    // 64-bit build

offset 0  1  2  3  4  5  6  7  8 ..... 15
      [ int data ][ 4 bytes PADDING ][ Node* next   ]

sizeof(Node)  = 16      alignof(Node) = 8
useful bytes  = 4       → 25% of the struct is your data
```

The four padding bytes are not optional. `Node*` must be 8-byte aligned, so the compiler inserts padding to push it to offset 8, then pads the struct up to a multiple of its alignment. Reordering the members does not help here — but with two ints and a pointer it does:

```
struct A { int x; Node* p; int y; };    // 24 bytes (pad, pointer, pad)
struct B { int x; int y; Node* p; };    // 16 bytes – same data, 33% smaller
```

RULE OF THUMB

Declare members in descending order of alignment: pointers and 8-byte types first, then 4-byte, then 2, then 1. Costs nothing, and on a million-node list it saves megabytes.

4. Array vs Linked List

Aspect	Array / vector	Singly linked list
Memory layout	One contiguous block	n scattered blocks
Access by index	$O(1)$	$O(n)$
Search (unsorted)	$O(n)$	$O(n)$
Search (sorted)	$O(\log n)$ binary search	$O(n)$ – no binary search possible
Insert at front	$O(n)$ shift	$O(1)$
Insert at back	$O(1)$ amortised	$O(1)$ with tail ptr, else $O(n)$
Insert at known position	$O(n)$ shift	$O(1)$
Insert at <i>index</i> k	$O(n)$ – but fast memmove	$O(n)$ – slow pointer walk
Delete at known position	$O(n)$ shift	$O(1)$ if you hold the predecessor
Memory per element	Payload only	Payload + 8 B link + allocator header
Cache behaviour	Excellent — prefetchable	Poor — dependent loads
Resize cost	Occasional $O(n)$ realloc + copy	None — one node at a time
Pointer stability	Invalidated on growth	Stable for the node's lifetime
Splice / concatenate	$O(n)$	$O(1)$
Memory freed on delete	Capacity retained	Returned immediately

THE HONEST SUMMARY

The table above is the Big-O story, and Big-O is not the whole story. On modern hardware a **vector** beats a **list** for middle insertion up to n in the tens of thousands, because a cache-friendly **memmove** of 100 000 bytes is cheaper than 50 000 cache-missing pointer dereferences to find the insertion point. Choose a linked list for **pointer stability** and **splicing**, not for asymptotic insertion speed.

5. Types of Linked Lists

Singly linked list (SLL)

```
head -> [10|•] -> [20|•] -> [30|null]
```

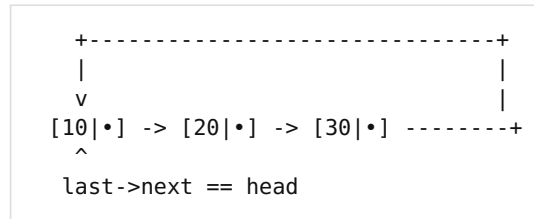
One link per node. Forward traversal only. Cheapest in memory. To delete a node you must hold its *predecessor*, which is why so many singly-linked algorithms carry a **prev** variable or use a dummy node.

Doubly linked list (DLL)

```
null <- [•|10|•] <-> [•|20|•] <-> [•|30|•] -> null
head ---^                ^--- tail
```

Two links per node (**prev**, **next**). Traversal in both directions, and deletion of a node in $O(1)$ *given only that node*. Costs 8 more bytes per node and one more write per structural change. This is what `std::list` is.

Circular singly linked list (CSLL)



The tail's **next** points back to the head instead of null. There is no null anywhere, so traversal must terminate on a *revisit* condition, not on null (Topic 43). Natural fit for round-robin schedulers and the Josephus problem.

Circular doubly linked list (CDLL)

Head's **prev** points to the tail and the tail's **next** points to the head. Every node has exactly two valid neighbours, which eliminates every null check in insertion and deletion — this is why the Linux kernel's **list_head** and most LRU caches use it.

Structural variants (orthogonal to the four above)

Variant	What changes
With tail pointer	Keep a second pointer to the last node → append becomes O(1)
With dummy / sentinel head	A permanent nodeless-payload node before the real head → head insertion and deletion stop being special cases
With size counter	Cached length → size() becomes O(1) instead of O(n)
Intrusive list	Links live inside the user's own struct, not in a wrapper node → zero extra allocation
XOR linked list	One field storing prev XOR next → doubly linked in the space of singly (Topic 78)
Unrolled list	Each node holds an array of <i>k</i> values → recovers cache locality (Topic 79)
Skip list	Layered express lanes over a sorted list → O(log n) search (Topic 80)

6. Linked List Terminology

Term	Meaning
node	One allocated unit: payload plus link(s).
data / payload	The value the node carries.
next	Pointer to the following node; <code>nullptr</code> in the last node.
prev	Pointer to the preceding node (doubly linked only).
head	Pointer to the first node. <i>The list is this pointer.</i> Empty list $\Leftrightarrow$ <code>head == nullptr</code> .
tail	The last node — the one whose <code>next</code> is null. Often cached in a separate pointer.
nullptr	The end-of-list marker. Use <code>nullptr</code> , never <code>NULL</code> or <code>0</code> , in C++11 and later.
traversal	Walking the list by repeatedly following <code>next</code> .
cursor / current	The working pointer during a traversal, conventionally <code>curr</code> or <code>temp</code> .
predecessor	The node whose <code>next</code> points at the node of interest. Required to unlink in an SLL.
dummy / sentinel	A dataless node placed before the head so no operation needs a special case for position 0.
dangling pointer	A pointer to memory that has already been <code>deleted</code> .
memory leak	An allocated node that no reachable pointer refers to.
self-referential struct	A struct containing a pointer to its own type — the definition of a node.

THE SENTENCE THAT PREVENTS MOST BUGS

"The list is the head pointer." Every function that can change which node is first must therefore either return the new head, or take the head *by reference* (`Node*& head`), or operate through a dummy node. Passing `Node* head` by value and assigning to it inside the function changes nothing at the call site — this is the single most common linked-list mistake in C++ and it compiles silently.

SECTION 2 — MEMORY & INTERNAL WORKING

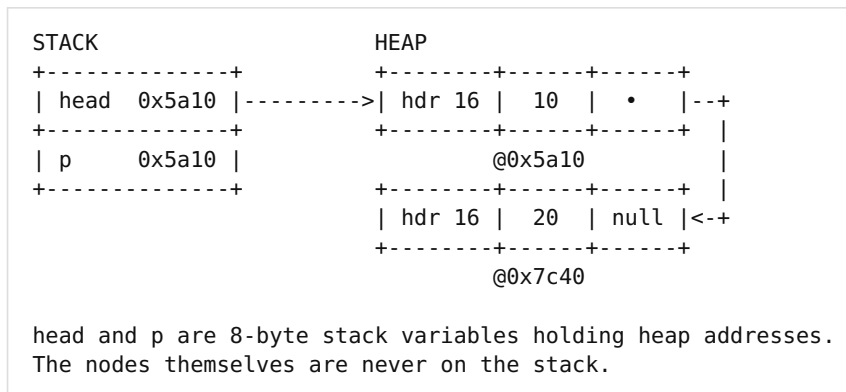
7. Heap Allocation per Node

What `new Node(10)` actually does

```
Node* p = new Node(10);
```

1. Calls `operator new(16)` — a request to the C++ allocator for 16 bytes.
2. The allocator (glibc `malloc`, typically) searches its free lists, or a size-class bin, for a suitable free chunk; if none exists it extends the heap via `brk` or `mmap`.
3. It writes a **chunk header** immediately before the returned address recording the chunk's size and flags.
4. The `Node` constructor runs at the returned address, initialising `data` and `next`.
5. The address comes back and is stored in `p` — which itself lives on the stack, and is 8 bytes.

Where the pieces live



THE MODEL TO CARRY

A linked list is n independent heap allocations tied together by addresses. Nothing owns the nodes except your own pointers. Nothing frees them except your own `delete`. There is no container object in the middle keeping score.

Consequences that follow directly

- **Every node needs a matching `delete`.** Losing `head` leaks the entire list, and the leak is unrecoverable — there is no other way to reach those addresses.
- **Node lifetime is independent of scope.** A node created inside a function survives after the function returns. That is the point — but it means the compiler will not clean up for you.
- **Allocation can fail.** `new` throws `std::bad_alloc`; `new (std::nothrow)` returns null instead.
- **Addresses are unpredictable.** Never assume node ordering, never compare node addresses for list order, and never assume a "next" allocation is adjacent.

8. Non-Contiguous Memory Layout

Prove it to yourself

```
#include <iostream>
int main() {
    Node* head = nullptr;
    for (int i = 5; i >= 1; --i) head = new Node(i * 10, head);

    for (Node* c = head; c; c = c->next)
        std::cout << c->data << " @ " << static_cast<void*>(c) << '\n';

    int arr[5] = {10,20,30,40,50};
    for (int i = 0; i < 5; ++i)
        std::cout << arr[i] << " @ " << static_cast<void*>(&arr[i]) << '\n';
}
```

A representative run:

LINKED LIST	ARRAY
10 @ 0x5581f2c0e2e0	10 @ 0x7ffd4b2c1a30
20 @ 0x5581f2c0e2c0	20 @ 0x7ffd4b2c1a34
30 @ 0x5581f2c0e2a0	30 @ 0x7ffd4b2c1a38
40 @ 0x5581f2c0e280	40 @ 0x7ffd4b2c1a3c
50 @ 0x5581f2c0e260	50 @ 0x7ffd4b2c1a40
gaps of 32 bytes, DESCENDING (and only because allocation was sequential and unfragmented)	exactly +4 each, ascending guaranteed by the standard

Read this carefully

- The list addresses *decrease* here, because nodes were created in reverse order and glibc reused the free region downward. List order and address order are unrelated.
- The gap is 32, not 16 — the allocator rounded a 16-byte request up to a 32-byte chunk including its header and alignment (see Topic 9).
- The array addresses are `0x...30, 34, 38, 3c, 40`: the contiguity guarantee, visible.
- Run the same program after a workload that fragments the heap and the list addresses will scatter across megabytes.

DO NOT DO THIS

Comparing node addresses (`if (a < b)`) to infer list position is meaningless and undefined for pointers into distinct allocations. Position is defined *only* by following `next`.

9. Pointer Overhead

The arithmetic, honestly

Storing 1 000 000 ints	Bytes per element	Total	Payload efficiency
<code>int[1000000]</code>	4	4 MB	100%
<code>std::vector<int></code>	4 (+24 header once)	~4 MB	~100%
SLL node, <i>struct size</i>	16	16 MB	25%
SLL node, <i>real heap cost</i>	32	32 MB	12.5%
DLL node, <i>real heap cost</i>	32	32 MB	12.5%

Where the extra 16 bytes come from

You asked for: `sizeof(Node) = 16 bytes`

What glibc malloc actually consumes:

```
[ 8 B chunk header ][ 4 B data ][ 4 B pad ][ 8 B next ][ 8 B slack ]
\_____ 32 bytes total _____/
```

- minimum chunk size on 64-bit glibc = 32 bytes
- chunk sizes are rounded to a multiple of 16 (alignment)
- so a 16-byte request and a 24-byte request cost the SAME 32 bytes

THE PRACTICAL COROLLARY

Because a 16-byte and a 24-byte request cost the same 32 bytes on a typical 64-bit allocator, upgrading a singly linked list to a **doubly** linked list is often *free in real memory*. Say this in an interview and you will be well ahead of the memorised answer that "a DLL costs 8 more bytes per node."

How to actually reduce it

- **Fatten the payload.** The overhead is per-node, not per-byte. A node carrying a 64-byte struct is 80/96 bytes — 67% efficient rather than 12.5%. Linked lists of tiny elements are the pathological case.
- **Unrolled list** (Topic 79): one node holds an array of 16 values, so one link is amortised over 16 elements.
- **Pool / arena allocation** (Topic 11): allocate 4096 nodes in one block, hand them out from a free list — kills both the chunk header and the allocator call.
- **Intrusive links:** put `next` inside the object you already allocate, so the list costs 8 bytes and zero extra allocations.
- **XOR list** (Topic 78): one pointer field for bidirectional traversal — clever, but breaks debuggers, garbage collectors and pointer-safe C++. Know it; do not ship it.

10. Cache Behavior

Why this is the topic that decides real performance

A modern CPU reads memory in **64-byte cache lines**. It never fetches 4 bytes; it fetches the whole line containing them.

```

ARRAY – one cache miss buys 16 ints
|<----- 64-byte line ----->|
[ a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 a10 ... a15 ]
miss hit hit hit hit hit hit hit hit ... hit
→ 1 miss per 16 elements = 0.0625 misses/element

LINKED LIST – one cache miss buys 1 node
line A: [ hdr | 10 | pad | ptr | ..... 32 bytes wasted ..... ]
line D: [ hdr | 20 | pad | ptr | ..... 32 bytes wasted ..... ]
→ 1 miss per element = 1.0 misses/element (16x worse)

```

The second, worse problem: dependent loads

```

for (Node* c = head; c; c = c->next)    sum += c->data;
//                                ^^^^^^^^^
// The address of the NEXT load is the RESULT of the current load.

```

The CPU cannot begin fetching node $k+1$ until node k has arrived, because it does not know the address yet. The hardware prefetcher — which detects strided access and fetches ahead — is useless here; there is no stride to detect. Every miss therefore costs its full latency, serialised:

Level	Latency	Array walk	List walk
L1 hit	~4 cycles	almost always	rarely
L2 hit	~12 cycles	—	sometimes
L3 hit	~40 cycles	—	sometimes
DRAM	~200–300 cycles	1 per 16 elements, prefetched	up to 1 per element, <i>not</i> prefetchable

THE NUMBER TO REMEMBER

Summing 10 million ints: a **vector** runs roughly **5-20× faster** than a **list** holding the same values, and the gap widens as the heap fragments. Both are $O(n)$. Big-O measures operation count; the cache measures what each operation actually costs.

Benchmark it yourself (do this once — it will stay with you)

```
#include <chrono>
#include <vector>
#include <list>
#include <numeric>
#include <iostream>

int main() {
    const int N = 10'000'000;
    std::vector<int> v(N, 1);
    std::list<int> l(N, 1);

    auto t0 = std::chrono::steady_clock::now();
    long long sv = std::accumulate(v.begin(), v.end(), 0LL);
    auto t1 = std::chrono::steady_clock::now();
    long long sl = std::accumulate(l.begin(), l.end(), 0LL);
    auto t2 = std::chrono::steady_clock::now();

    using ms = std::chrono::duration<double, std::milli>;
    std::cout << "vector " << ms(t1-t0).count() << " ms (" << sv << ")\n"
              << "list   " << ms(t2-t1).count() << " ms (" << sl << ")\n";
}
// build with -O2. Shuffle the list nodes first for the honest worst case.
```

MEASUREMENT NOTE

A freshly built list is the *best* case: its nodes were allocated sequentially and are still roughly adjacent. A list that has lived through a million random inserts and deletes is scattered, and the gap grows accordingly. Benchmark the aged case, not the fresh one.

11. Allocation and Deallocation Cost

The per-node tax

Operation	Typical cost
Write one element into a vector (capacity available)	~1 ns
new Node — glibc fast-bin hit	~15–30 ns
new Node — general path / heap growth	~50–200 ns
delete p	~15–50 ns
new under thread contention on a shared arena	hundreds of ns

Building a 1 000 000-node list therefore spends roughly **20-100 ms in the allocator alone**, before a single useful operation. Filling a **vector** of the same values costs a few milliseconds, in three or four reallocations total.

Deallocation is not free either

```
void destroy(Node*& head) {
    while (head) {
        Node* nxt = head->next;    // save BEFORE deleting
        delete head;               // one allocator call per node
        head = nxt;
    }
}
```

Destroying the list is n allocator calls, each of which may coalesce the freed chunk with its neighbours and re-link it into a bin. Destroying a vector is one **free**.

ORDER MATTERS — READ THE LINE ABOVE

`delete head; head = head->next;` is use-after-free: you read `next` out of a block you already returned to the allocator. It usually appears to work, which is what makes it dangerous. Always save `next` first.

The fix when you genuinely need list semantics at scale: a pool allocator

```
class NodePool {
    static const int BLOCK = 4096;
    std::vector<Node*> blocks;    // raw slabs
    Node* freeList = nullptr;

public:
    Node* acquire(int v) {
        if (!freeList) {
            Node* slab = static_cast<Node*>(
                ::operator new(sizeof(Node) * BLOCK));
            blocks.push_back(slab);
            // thread them into the free list
            for (int i = 0; i < BLOCK; ++i) {
                slab[i].next = freeList;
                freeList = &slab[i];
            }
        }
        Node* n = freeList;
        freeList = freeList->next;
        n->data = v; n->next = nullptr;
        return n;
    }
    // O(1), no free()
    void release(Node* n) { n->next = freeList; freeList = n; }
    ~NodePool() { for (Node* b : blocks) ::operator delete(b); }
};
```

WHAT THE POOL BUYS YOU

One allocator call per 4096 nodes instead of one per node — typically a **5-10× speedup** on build and teardown. And because the slab is contiguous, nodes allocated together sit together, which partially restores cache locality. This is exactly how kernel slab allocators, memory arenas in game engines, and the free lists inside allocators themselves are built. Note that the free list in the code above *is itself a singly linked list* — the structure allocating memory is the structure you are studying.

PRACTICE SET — Topics 1-11

A. Conceptual (answer in your own words)

1. Complete the sentence precisely: "Insertion into a linked list is $O(1)$ provided that _____.
Then state what the complexity becomes without that provision.
2. Why is `Node* next;` legal inside `struct Node`, while `Node next;` is a compile error?
3. A colleague says a doubly linked list "obviously uses 50% more memory than a singly linked list for an `int` payload." Give the number that shows this is usually false in practice, and explain why.
4. Both an array scan and a linked list scan are $O(n)$. Give the two independent hardware reasons the array is 5–20× faster, and explain why the second one cannot be fixed by a bigger cache.
5. Why does a `vector` invalidate pointers to its elements while a linked list never does? Name one data structure that depends on this property.
6. You are told "just use a linked list, it grows dynamically." Give two situations where that is the right call and two where `std::vector` is clearly better.

B. Memory arithmetic (pen and paper, 64-bit, glibc)

1. `struct Node { int data; Node* next; };` — give `sizeof`, `alignof`, the offset of each member, and the number of padding bytes.
2. `struct N2 { char c; Node* p; int x; };` — give `sizeof` and the byte offsets. Then reorder the members for minimum size and give the new `sizeof`.
3. A singly linked list of 2 000 000 `ints`. Compute (a) struct bytes, (b) real heap bytes at 32 bytes per chunk, (c) the equivalent `vector<int>`, (d) the ratio (b)/(c).
4. Node payload is now a 48-byte struct. Recompute payload efficiency for the singly linked list and compare it with the 4-byte case. What general rule does this demonstrate?
5. A 64-byte cache line, nodes of real size 32 bytes scattered across a fragmented heap. How many useful payload bytes does one cache line deliver, in the worst case, for a 4-byte payload? Express it as a percentage of the line.
6. An unrolled linked list stores 16 `ints` per node plus one `next` pointer and one `count`. Give bytes per element and compare with the plain SLL's 32.

C. Code tasks

1. Build a 5-node list and print each node's value with its address, alongside a 5-element array printed the same way. Confirm the array gaps equal `sizeof(int)` and record what the list gaps actually are.
2. Write `struct Node` with no constructor, allocate with `new Node;`, and print `next` without setting it. Then allocate with `new Node();` and print again. Explain the difference in one sentence.
3. Time building and summing a 5 000 000-element `std::vector<int>` against a `std::list<int>`. Then shuffle the list (build it by inserting at random positions) and re-time the sum. Report all three numbers.
4. Implement `NodePool` from Topic 11. Time building a 1 000 000-node list with plain `new` versus with the pool. Report the speedup.
5. Write a destroy function containing the bug `delete head; head = head->next;`. Run it under `g++ -fsanitize=address` and paste the first line of the report.

ANSWER KEY — Topics 1-11

A. Conceptual

1. "...provided that you already hold a pointer to the node after which you are inserting." Without it you must walk from the head, so insertion *by index* is $O(n)$ — and slower than the array's $O(n)$ shift, because **memmove** on contiguous memory is far cheaper per element than n cache-missing dereferences.
2. A pointer has a fixed, known size (8 bytes) regardless of what it points to, so the compiler can compute the struct's layout even though **Node** is still incomplete at that point. A *member of type Node* would require **sizeof(Node)** to be known while defining **Node** — infinitely recursive.
3. Both node types cost **32 bytes** of real heap. glibc's minimum chunk is 32 bytes and chunks are rounded to a multiple of 16, so a 16-byte request (SLL: int + pad + next) and a 24-byte request (DLL: int + pad + next + prev) land in the same 32-byte chunk. The extra **prev** is free in the common case — it costs a write per structural change, not memory.
4. (a) **Cache line utilisation**: one 64-byte fetch delivers ~16 array ints, but only one scattered node. (b) **Dependent loads**: the address of node $k+1$ is the value loaded from node k , so the prefetcher has no stride to predict and the misses serialise at full DRAM latency. A bigger cache cannot fix (b) because the problem is not capacity — it is that the address is unknown until the previous load retires.
5. A vector guarantees contiguity, so exceeding capacity forces a new block plus a copy or move of every element; the old addresses are freed. A list never relocates a node, so its address is stable for the node's lifetime. **LRU cache** depends on this: the hash map stores node pointers and the list is reordered constantly, yet the stored pointers stay valid.
6. Right call: an LRU cache (pointer stability plus $O(1)$ unlink); constant splicing of large runs between lists; an intrusive free list inside an allocator. **vector** better: any indexed or binary-searched access; any hot iteration loop; and — counter-intuitively — middle insertion at small and medium n , where the memmove beats the pointer walk to the insertion point.

B. Memory arithmetic

1. **sizeof = 16, alignof = 8**. **data** at offset 0 (4 bytes), 4 bytes padding at offsets 4-7, **next** at offset 8 (8 bytes). **4 padding bytes**.
2. As written: **c** @0, pad 1-7, **p** @8, **x** @16, pad 20-23 → **sizeof = 24**. Reordered { **Node* p;** **int x;** **char c;** }: **p** @0, **x** @8, **c** @12, pad 13-15 → **sizeof = 16**. Descending alignment order, one-third smaller.
3. (a) $2\,000\,000 \times 16 = \mathbf{32\ MB}$. (b) $2\,000\,000 \times 32 = \mathbf{64\ MB}$. (c) $2\,000\,000 \times 4 = \mathbf{8\ MB}$. (d) ratio = **8×**.
4. 48-byte payload → struct is $48 + 8 = 56$, rounded to a 64-byte chunk. Efficiency $48/64 = \mathbf{75\%}$, against $4/32 = \mathbf{12.5\%}$ for the int. Rule: *linked-list overhead is charged per node, not per byte*, so it is ruinous for tiny payloads and negligible for fat ones.
5. Worst case the line holds one node and nothing else useful: 4 useful bytes out of 64 = **6.25%**. The array delivers $64/64 = 100\%$.
6. 16 ints (64 B) + **next** (8 B) + **count** (4 B) + 4 B pad = 80 bytes per node, rounded to a 80-byte chunk (multiple of 16). $80/16 = \mathbf{5\ bytes\ per\ element}$ versus **32** — a 6.4× improvement, and the 16 ints inside a node are contiguous, so scanning is nearly array-speed.

C. Code tasks — what to verify

- **C1.** Array gaps are exactly 4. List gaps are typically 32 on an unfragmented heap and may be *negative* (descending addresses) — this is normal and is the whole point of Topic 8.
- **C2.** `new Node;` default-initialises: for a trivial struct that means *no initialisation*, so `next` holds garbage. `new Node();` value-initialises and zeroes the members, so `next` prints 0. Traversing the first one walks into arbitrary memory.
- **C3.** Expect roughly: vector sum ~5 ms, freshly built list ~40–80 ms, shuffled list ~150–400 ms at 5 M elements on a typical desktop. The third number is the honest one.
- **C4.** Expect a **5–10×** speedup on build. If you see less, you are probably timing a debug build — compile with `-O2`.
- **C5.** ASan reports `heap-use-after-free ... READ of size 8` at the `head->next` line, with a "freed by thread T0 here" stack pointing at the `delete` one line above.

HANDNOTE SUMMARY CARD

LINKED LIST = NODES + EXPLICIT LINKS (array = base + arithmetic)

THE LIST IS THE HEAD POINTER

empty list head == nullptr
lose head entire list leaked, unrecoverable
change the head return new head, or take Node*& head, or use a dummy

NODE LAYOUT (64-bit)

```
struct Node { int data; Node* next; };
```

off 0 [int data][4B PAD][Node* next] = sizeof 16, align 8
real heap chunk = 32 bytes (glibc min 32, rounds to multiple of 16)
=> SLL and DLL of an int usually cost the SAME 32 bytes

MEMORY MATH (1e6 ints)

vector<int>	4 MB	100%	payload efficiency
SLL struct	16 MB	25%	
SLL real heap	32 MB	12.5%	<- the number that matters

rule: overhead is PER NODE, not per byte. Fat payload => fine.

CACHE

line = 64 B array: 1 miss / 16 ints list: 1 miss / 1 node
dependent load: addr of next load = result of current load
 => prefetcher blind, misses serialise at ~200-300 cyc
real gap on a 10M scan: vector 5-20x faster than list

COST PER NODE

write into vector	~1 ns	new Node	~15-200 ns
delete	~15-50 ns	pool alloc	5-10x faster build

COMPLEXITY – SLL

access by index	0(n)	insert at head	0(1)
search	0(n)	insert at tail (no tail*)	0(n)
insert AT KNOWN ptr	0(1)	insert at tail (tail*)	0(1)
delete AT KNOWN ptr	0(1)	*needs predecessor in an SLL	

CHOOSE A LIST FOR pointer stability · 0(1) splice · intrusive/free lists

CHOOSE A VECTOR FOR indexing · iteration speed · almost everything else

FATAL ORDER BUG

```
delete head; head = head->next;      <- use-after-free  
Node* nxt = head->next; delete head; head = nxt;    <- correct
```

SECTION 3 — SINGLY LINKED LIST OPERATIONS

Every function in this section assumes the following declaration. Read it once and keep it in mind for the next seventeen topics.

```
struct Node {
    int data;
    Node* next;
    Node(int d, Node* n = nullptr) : data(d), next(n) {}
};
```

THE THREE QUESTIONS TO ASK OF EVERY OPERATION BELOW

1. Does it change which node is first? → then the head must be returned or passed by reference.
2. Does it need the *predecessor*? → then the walk must stop one node early.
3. Does it remove a node? → then **next** must be saved before the **delete**.

Nearly every linked-list bug is one of these three, and every one of them compiles cleanly.

12. Creating a Node

```
Node* makeNode(int value) {
    return new Node(value);           // next = nullptr by default
}

Node* n = makeNode(10);
std::cout << n->data;                // 10
std::cout << (n->next == nullptr);  // 1
```

Arrow versus dot

```
Node* p = new Node(10);
Node s(20);

p->data    // pointer → arrow.   Equivalent to (*p).data
s.data    // object  → dot.
```

FOUR WAYS TO GET THIS WRONG

- `Node* p; p->data = 5;` — `p` is uninitialised; writing through it is undefined behaviour. Always `Node* p = nullptr;` at declaration.
- `Node n; n.next` is garbage — a stack node with no constructor is not zeroed.
- Creating a node and never linking it → immediate leak.
- Returning the address of a *local* node (`Node n(5); return &n;`) → dangling pointer the moment the function returns. Nodes must be heap-allocated.

create node — Time $O(1)$ · Space $O(1)$ · one allocator call (~15–200 ns, Topic 11)

13. Building a List

Method 1 — manual wiring (understand this once, then never write it again)

```
Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);
// head -> 10 -> 20 -> 30 -> null
```

Method 2 — build with a tail pointer (the standard idiom, preserves order, $O(n)$)

```
Node* buildFromArray(const int a[], int n) {
    if (n == 0) return nullptr;

    Node* head = new Node(a[0]);
    // tail always points at the last node
    Node* tail = head;

    for (int i = 1; i < n; ++i) {
        tail->next = new Node(a[i]);    // link the new node on
        tail = tail->next;              // and advance the tail
    }
    return head;
}
```

i=1 head[10] -> [20] tail moves right, one step per node
^tail -----^

i=2 head[10] -> [20] -> [30]
^tail ---^

Without the tail pointer you would walk from head every time: $O(n^2)$.

Method 3 — front insertion ($O(1)$ per node, but reverses the input)

```
Node* buildReversed(const int a[], int n) {
    Node* head = nullptr;
    for (int i = 0; i < n; ++i) head = new Node(a[i], head);
    return head;    // a = {1,2,3} → 3 -> 2 -> 1
}

// Feed the array backwards to get the original order back:
for (int i = n - 1; i >= 0; --i) head = new Node(a[i], head);
```

Method 4 — dummy node (cleanest for building; see Topic 74)

```
Node* buildWithDummy(const int a[], int n) {
    Node dummy(0);    // stack-allocated, never leaks
    Node* tail = &dummy;
    for (int i = 0; i < n; ++i) {
        tail->next = new Node(a[i]);
        tail = tail->next;
    }
    // real head, no empty-list special case
    return dummy.next;
}
```

14. Traversal

The canonical loop — memorise the shape

```
void printList(Node* head) {
    for (Node* curr = head; curr != nullptr; curr = curr->next)
        std::cout << curr->data << " -> ";
    std::cout << "null\n";
}
```

Three rules encoded in that one line:

- **Never move head itself.** Use a separate cursor. Advancing **head** destroys your only handle on the list.
- **Test `curr != nullptr`, not `curr->next != nullptr`** — unless you deliberately want to stop on the *last* node.
- **Advance with `curr = curr->next`**, the last statement of the body.

The two stop conditions — this distinction drives half of Section 3

```
// visits EVERY node; ends with curr == nullptr
while (curr != nullptr)
// stops ON the last node; curr == tail afterwards
while (curr->next != nullptr)
```

```
list: [10] -> [20] -> [30] -> null
```

while (curr)	curr: 10, 20, 30, null	(4 checks, 3 visits)
while (curr->next)	curr: 10, 20, 30	(stops holding the TAIL)

Need the tail, or the predecessor of the tail?	Use the second form.
Need to visit everything?	Use the first.

EMPTY-LIST TRAP

while (curr->next) dereferences **curr** before testing it. If the list is empty (**head == nullptr**) that is an immediate null dereference. Every function using the second form needs an **if (!head) return;** guard on the first line.

15. Recursive Traversal

Forward — recursion after the visit

```
void printForward(Node* head) {
    if (head == nullptr) return;           // base case
    std::cout << head->data << " ";       // process, THEN recurse
    printForward(head->next);
}
```

Backward — recursion before the visit (the whole trick)

```
void printBackward(Node* head) {
    if (head == nullptr) return;
    printBackward(head->next);           // recurse FIRST
    std::cout << head->data << " ";     // process on the way back up
}
```

list 10 -> 20 -> 30, printBackward:

call(10)	going down
call(20)	(nothing printed)
call(30)	
call(null) -> return	
print 30	coming back up
print 20	(printing happens here)
print 10	

output: 30 20 10 — a singly linked list printed in reverse,
using the call stack as the storage.

WHY THIS MATTERS BEYOND PRINTING

Swapping one line — process before recursing, or after — turns forward into backward. That single idea is the engine behind recursive reversal (Topic 30), palindrome checking (Topic 54), and adding two numbers stored in forward order (Topic 72). Whenever you need "the last node first" in a singly linked list, the call stack is your reverse iterator.

THE REAL COST

Recursion is $O(n)$ *stack* space, one frame per node. On a typical 8 MB stack with ~48-byte frames, a list of about 100 000+ nodes will overflow the stack and crash. Note that `printForward` is tail-recursive and GCC will usually convert it to a loop at -O2 — but `printBackward` cannot be, because work remains after the call. Never ship recursion over user-controlled list lengths.

recursive traversal – Time $O(n)$ · Space $O(n)$ stack · iterative preferred in production

16. Insert at Head

The cheapest operation a linked list has, and the one that justifies its existence.

```
void insertAtHead(Node*& head, int value) {    // note the reference
    // new node points at old head
    head = new Node(value, head);
}
```

Written out, so the order is unmistakable:

```
Node* n = new Node(value);
n->next = head;    // 1. new node points to the current first node
head     = n;      // 2. head now points to the new node
```

```

before   head -> [10] -> [20] -> null

step 1   [5] -> [10] -> [20] -> null      n->next = head
         head ---^

step 2   head -> [5] -> [10] -> [20] -> null  head = n

```

REVERSE THE TWO LINES AND YOU LOSE THE LIST

`head = n; n->next = head;` makes the node point at **itself** — a one-node cycle — and every original node is leaked. Rule: *always attach the new node to the list before detaching the old handle.*

Why `Node*& head` and not `Node* head`

```

// caller unaffected!
void wrong(Node* head, int v) { head = new Node(v, head); }
// caller's head updated
void right(Node*& head, int v) { head = new Node(v, head); }

Node* alt(Node* head, int v) { return new Node(v, head); } // or return it
head = alt(head, 5);

```

The empty list needs no special case: `new Node(v, nullptr)` is exactly correct.

insert at head – Time $O(1)$ · Space $O(1)$ · no traversal, no special cases

17. Insert at Tail

Without a tail pointer — $O(n)$

```

void insertAtTail(Node*& head, int value) {
    Node* n = new Node(value);

    if (head == nullptr) { head = n; return; } // special case: empty list

    Node* curr = head;
    // stop ON the last node
    while (curr->next != nullptr) curr = curr->next;
    curr->next = n;
}

```

```

curr walks: [10] -> [20] -> [30] -> null
              ^ stops here, because curr->next == null
then:       [10] -> [20] -> [30] -> [40] -> null

```

With a tail pointer — $O(1)$

```
struct List {
    Node* head = nullptr;
    Node* tail = nullptr;

    void pushBack(int value) {
        Node* n = new Node(value);
        if (!head) { head = tail = n; return; } // first node is both
        tail->next = n;
        tail = n;
    }
};
```

THE TAIL POINTER'S HIDDEN CONTRACT

Once you cache a tail pointer, **every** operation that can remove or add the last node must update it: delete-tail, delete-by-value, delete-at-position, reverse, and clear. A stale tail pointing at a deleted node is a dangling pointer, and the crash will appear later, in an unrelated `pushBack`. If you keep a tail, audit every mutating function.

insert at tail – Time $O(n)$ without tail pointer · $O(1)$ with · Space $O(1)$

18. Insert at Position

Convention used throughout this book: positions are **0-based**, and inserting at position k means the new node ends up at index k .

```
void insertAtPosition(Node*& head, int value, int pos) {
    if (pos < 0) return;
    // delegates to Topic 16
    if (pos == 0) { head = new Node(value, head); return; }

    Node* prev = head;
    for (int i = 0; i < pos - 1 && prev != nullptr; ++i)
        prev = prev->next; // land on the node BEFORE pos

    if (prev == nullptr) return; // pos beyond the end – reject

    prev->next = new Node(value, prev->next); // splice in
}
```

```
insert 25 at pos 2:  [10] -> [20] -> [30] -> [40]
                     0      1      2      3

walk pos-1 = 1 step: prev lands on [20]

prev->next = new Node(25, prev->next)

result:                [10] -> [20] -> [25] -> [30] -> [40]
                        ^ index 2
```


THE ONE-LINE SPLICE

`prev->next = new Node(value, prev->next);` does both writes in the correct order by construction: the constructor reads the old `prev->next` before the assignment overwrites it. Writing it as two statements is where the order bug appears — `prev->next = n; n->next = prev->next;` makes the node point at itself and orphans the entire tail.

The four cases to test, every time

Case	Expected
Empty list, <code>pos == 0</code>	New node becomes the head
Empty list, <code>pos > 0</code>	Rejected, list unchanged
<code>pos == 0</code> on a non-empty list	Head insertion, head updated
<code>pos == length</code>	Appends at the tail (valid)
<code>pos > length</code>	Rejected, no crash

insert at position – Time $O(pos)$, worst case $O(n)$ · Space $O(1)$

19. Insert in Sorted List

Keep an ascending list ascending. This is the core step of insertion sort on a linked list (Topic 58) and the reason a linked list can be a usable priority structure.

```
void insertSorted(Node*& head, int value) {
    Node* n = new Node(value);

    // Case 1: empty list, or the new value belongs at the front
    if (head == nullptr || value <= head->data) {
        n->next = head;
        head = n;
        return;
    }

    // Case 2: walk to the last node whose value is still <= value
    Node* curr = head;
    while (curr->next != nullptr && curr->next->data < value)
        curr = curr->next;

    // may be nullptr – that is the append case, free
    n->next = curr->next;
    curr->next = n;
}
```

insert 25 into [10] -> [20] -> [30] -> [40] -> null

curr = 10: curr->next->data = 20 < 25 → advance

curr = 20: curr->next->data = 30 !< 25 → stop. curr = [20]

n->next = [30] [25] -> [30] -> [40]

curr->next = n [10] -> [20] -> [25] -> [30] -> [40]

WHY THE LOOP TESTS `curr->next->data`, NOT `curr->data`

To splice a node in, you must be standing on its **predecessor**. Testing `curr->data < value` would overshoot by exactly one node and you would then have no way back — a singly linked list has no reverse gear. This "look one ahead, stop on the predecessor" pattern recurs in every delete operation in this section.

Note the tail case needs no special handling: if `value` is larger than everything, the loop runs to the last node, `curr->next` is `nullptr`, and `n->next = nullptr` is exactly right. Note also that `<=` in case 1 and `<` in the loop together make the insertion **stable**: equal values are inserted after existing equals.

insert sorted – Time $O(n)$ · Space $O(1)$ · building a sorted list this way is $O(n^2)$

20. Delete Head

```
void deleteHead(Node*& head) {
    // empty list – nothing to do, no crash
    if (head == nullptr) return;

    Node* doomed = head;           // 1. remember the node
    // 2. unlink it (works even if next is null)
    head = head->next;
    delete doomed;                // 3. free it
}
```

before	head -> [10] -> [20] -> [30]	
	doomed --^	
step 2	head -----> [20] -> [30]	[10] now unreachable but alive
	doomed -> [10]	
step 3	head -----> [20] -> [30]	[10] freed

THE TWO WAYS TO GET THIS WRONG

- `delete head; head = head->next;` — **use-after-free**. You read a field out of memory you already returned to the allocator. It often appears to work, which is why it survives code review.
- `head = head->next;` alone — **memory leak**. The node is unlinked but never freed, and no pointer to it remains anywhere.

The sequence is always: **save** → **unlink** → **delete**.

Single-node list: `head->next` is `nullptr`, so `head` becomes `nullptr` and the list is correctly left empty. No special case needed.

delete head – Time $O(1)$ · Space $O(1)$ · the only $O(1)$ deletion in a plain SLL

21. Delete Tail

The asymmetry that defines a singly linked list: removing the *first* node is $O(1)$, removing the *last* is $O(n)$ — because you need the second-to-last node, and there is no way back.

```

void deleteTail(Node*& head) {
    if (head == nullptr) return;           // empty

    if (head->next == nullptr) {           // exactly one node
        delete head;
        head = nullptr;
        return;
    }

    Node* prev = head;
    // stop on the SECOND-TO-LAST
    while (prev->next->next != nullptr)
        prev = prev->next;

    delete prev->next;
    // new tail must be terminated
    prev->next = nullptr;
}

```

```

[10] -> [20] -> [30] -> [40] -> null

prev = 10: prev->next->next = [30]  != null → advance
prev = 20: prev->next->next = [40]  != null → advance
prev = 30: prev->next->next = null    → stop.  prev = [30]

delete prev->next (frees [40])
prev->next = nullptr

[10] -> [20] -> [30] -> null

```

DOUBLE-DEREFERENCE GUARD

`prev->next->next` reads two pointers. It is safe only because the two guards above it have already proved the list has at least two nodes. Remove either guard and the single-node list segfaults. This is the reason the one-node case is written out explicitly rather than folded into the loop.

A cached tail pointer does **not** help here: to delete the tail you need the node *before* it, and an SLL cannot give you that in less than $O(n)$. Constant-time tail deletion is one of the main reasons to upgrade to a doubly linked list.

delete tail – Time $O(n)$ always, even with a tail pointer · Space $O(1)$

22. Delete at Position

```
void deleteAtPosition(Node*& head, int pos) {
    if (head == nullptr || pos < 0) return;

    if (pos == 0) {                                // delegates to Topic 20
        Node* doomed = head;
        head = head->next;
        delete doomed;
        return;
    }

    Node* prev = head;
    for (int i = 0; i < pos - 1 && prev->next != nullptr; ++i)
        prev = prev->next;                        // land on pos-1

    if (prev->next == nullptr) return;             // pos is out of range

    Node* doomed = prev->next;                     // save
    // unlink (relink around it)
    prev->next = doomed->next;
    delete doomed;                               // free
}
```

```
delete pos 2 from  [10] -> [20] -> [30] -> [40]
                   0      1      2      3

prev walks 1 step → prev = [20], doomed = [30]

prev->next = doomed->next:

[10] -> [20] -----> [40]
                   [30] (detached, then deleted)
```

THE UNIVERSAL UNLINK

`prev->next = doomed->next; delete doomed;` is *the* deletion primitive of a singly linked list. Every deletion in this section — head, tail, by position, by value, duplicates — is that same pair of lines wrapped in a different way of finding `prev`. Learn the primitive; the rest is search.

The loop condition `prev->next != nullptr` stops the walk from running off the end when `pos` exceeds the length, and the check immediately after it converts that into a clean rejection rather than a crash. Note that `doomed->next` may itself be `nullptr` — that is the delete-the-last-node case, and it is handled for free.

delete at position — Time $O(\text{pos})$, worst case $O(n)$ · Space $O(1)$

23. Delete by Value

Delete the first occurrence

```
void deleteFirstOccurrence(Node*& head, int value) {
    if (head == nullptr) return;

    if (head->data == value) {                // it is the head
        Node* doomed = head;
        head = head->next;
        delete doomed;
        return;
    }

    Node* prev = head;
    while (prev->next != nullptr && prev->next->data != value)
        prev = prev->next;                    // look one ahead

    if (prev->next == nullptr) return;        // not found

    Node* doomed = prev->next;
    prev->next = doomed->next;
    delete doomed;
}
```

Delete *all* occurrences — and the trick that removes the special case

```
void deleteAllOccurrences(Node*& head, int value) {
    // sentinel in front of the real head
    Node dummy(0, head);
    Node* prev = &dummy;

    while (prev->next != nullptr) {
        if (prev->next->data == value) {
            Node* doomed = prev->next;
            prev->next = doomed->next;
            delete doomed;                    // do NOT advance prev
        } else {
            prev = prev->next;
        }
    }
    head = dummy.next;                       // head may have changed
}
```

TWO IDEAS WORTH MORE THAN THE CODE

1. The dummy node. `dummy` sits in front of the real head, so the head is no longer a special case — it now has a predecessor like every other node. The dummy is a *stack* object, so it costs nothing and cannot leak. This is Topic 74, and it simplifies roughly a third of all linked-list problems.

2. Do not advance after a deletion. After unlinking, `prev->next` already points at the next unexamined node. Advancing anyway skips it — which is exactly why the naive version fails on consecutive duplicates like `[3,3,3]`.

```
delete all 3s from dummy -> [3] -> [3] -> [5]
```

```
prev=dummy: next is 3 → unlink    dummy -> [3] -> [5]    (prev stays!)
prev=dummy: next is 3 → unlink    dummy -> [5]
prev=dummy: next is 5 → advance   prev = [5]
prev=[5]:   next is null → stop
```

If prev had advanced after the first deletion it would have landed on the second 3 and then tested [5] – leaving one 3 in the list.

delete by value – Time $O(n)$ · Space $O(1)$ · all-occurrences version is single-pass

24. Delete Entire List

```
void deleteList(Node*& head) {
    Node* curr = head;
    while (curr != nullptr) {
        Node* nxt = curr->next;    // 1. SAVE the link out
        delete curr;              // 2. free the node
        curr = nxt;               // 3. move on
    }
    head = nullptr;              // 4. leave the handle in a valid state
}
```

All four lines matter. Line 1 is the difference between correct code and use-after-free. Line 4 is the difference between an empty list and a dangling **head** that will crash the next time anyone touches it.

DO NOT DELETE A LIST RECURSIVELY

```
void destroy(Node* h) { if (!h) return; destroy(h->next); delete h; }
```

It is elegant, it is correct, and it will blow the stack on a list of ~100 000 nodes. Real bug: this is also why a naïve `~Node() { delete next; }` destructor is a trap — destroying a long list becomes a recursive chain of destructor calls and overflows. Delete iteratively.

Ownership, properly

```
class LinkedList {
    Node* head = nullptr;
public:
    ~LinkedList() { clear(); }                // RAII: no leak, ever
    void clear() {
        while (head) { Node* n = head->next; delete head; head = n; }
    }
    LinkedList(const LinkedList&)             = delete; // rule of three:
    // a shallow copy would
    LinkedList& operator=(const LinkedList&) = delete;
    // double-free every node
};
```

delete entire list – Time $O(n)$ · Space $O(1)$ iterative, $O(n)$ stack if recursive

25. Search an Element

```
bool contains(Node* head, int value) {
    for (Node* curr = head; curr != nullptr; curr = curr->next)
        if (curr->data == value) return true;
    return false;
}

// returns the node, or nullptr
Node* find(Node* head, int value) {
    for (Node* curr = head; curr; curr = curr->next)
        if (curr->data == value) return curr;
    return nullptr;
}

// returns 0-based index, or -1
int indexOf(Node* head, int value) {
    int i = 0;
    for (Node* curr = head; curr; curr = curr->next, ++i)
        if (curr->data == value) return i;
    return -1;
}
```

Recursive form

```
bool containsRec(Node* head, int value) {
    if (!head) return false;
    if (head->data == value) return true;
    return containsRec(head->next, value);
}
```

WHY BINARY SEARCH IS IMPOSSIBLE HERE

Binary search needs the middle element in $O(1)$. Finding the middle of a linked list is itself $O(n)$ (Topic 47), so the "divide" step costs as much as the linear scan it was meant to replace: the recurrence becomes $T(n) = T(n/2) + O(n) = O(n)$. **A sorted linked list is still $O(n)$ to search.** If you need sub-linear search over linked nodes, that is precisely what a *skip list* (Topic 80) was invented for — express lanes give it $O(\log n)$ expected.

Return the `Node*` rather than a `bool` whenever the caller might act on the node — because holding the node is what makes subsequent operations $O(1)$. Returning `true` throws away the only expensive thing the function computed.

search — Time $O(n)$ sorted or unsorted · Space $O(1)$ iterative, $O(n)$ recursive

26. Length of List

```
int length(Node* head) {
    int count = 0;
    for (Node* curr = head; curr != nullptr; curr = curr->next) ++count;
    return count;
}

int lengthRec(Node* head) {
    return head == nullptr ? 0 : 1 + lengthRec(head->next);
}
```

The O(1) alternative — cache it

```
struct List {
    Node* head = nullptr;
    int sz = 0; // maintained by every mutator
    void pushFront(int v) { head = new Node(v, head); ++sz; }
    void popFront() { if (!head) return;
                    Node* d = head;
                    head = head->next;
                    delete d; --sz; }
    int size() const { return sz; } // O(1)
};
```

A REAL PIECE OF C++ HISTORY

Before C++11, `std::list::size()` was allowed to be O(n) — and libstdc++ implemented it that way, so that `splice()` could stay O(1) (splicing a range does not tell you how many nodes moved). C++11 mandated O(1) `size()`, which forced `splice()` of a sub-range to become O(n). You cannot have both. `std::forward_list` resolved it the other way: it has **no `size()` at all**. If an interviewer asks why, this is the answer.

THE CACHED-COUNTER CONTRACT

A cached size is only as correct as your least careful mutator. Every insert, every delete, and `clear()` must update it. A single missed decrement produces a phantom element that no traversal will ever find, and the bug surfaces far from its cause.

length — Time O(n) computed, O(1) cached · Space O(1) · never assume a list knows its size

27. Update a Node Value

```
bool updateAtPosition(Node* head, int pos, int newValue) {
    if (pos < 0) return false;
    Node* curr = head;
    for (int i = 0; i < pos && curr != nullptr; ++i) curr = curr->next;
    if (curr == nullptr) return false;    // out of range
    curr->data = newValue;
    return true;
}

int updateAllByValue(Node* head, int oldValue, int newValue) {
    int changed = 0;
    for (Node* curr = head; curr; curr = curr->next)
        if (curr->data == oldValue) { curr->data = newValue; ++changed; }
    // report how many, not just true
    return changed;
}
```

THE OPERATION THAT CHANGES NO POINTERS

Updating is the only structural non-event in this section: no allocation, no free, no relink, and `head` cannot change. That is also why it needs `Node* head` and not `Node*& head` — a useful check on your understanding of Topic 6.

Be aware of the mirror-image trap. Many problems (swap nodes in pairs, Topic 67; reverse, Topic 29) explicitly require you to **relink nodes rather than swap their data**. Swapping values is easier and passes the toy test, but it breaks every external pointer that was aimed at a particular node — which is exactly the property an LRU cache relies on. Read the problem statement carefully: "swap the nodes" and "swap the values" are different problems.

Return a `bool` or a count rather than `void`. "Position 7 does not exist" is information the caller needs, and silently doing nothing is how a bug hides for a month.

update — Time $O(\text{pos})$ by position, $O(n)$ by value · Space $O(1)$ · no allocation

28. Get Nth Node

```
// 0-based, nullptr if out of range
Node* getNth(Node* head, int n) {
    if (n < 0) return nullptr;
    Node* curr = head;
    for (int i = 0; i < n && curr != nullptr; ++i) curr = curr->next;
    // nullptr if we ran off the end
    return curr;
}

int getNthValue(Node* head, int n, int fallback = -1) {
    Node* p = getNth(head, n);
    return p ? p->data : fallback;
}
```

```

getNth(head, 3):   [10] -> [20] -> [30] -> [40] -> null
                   0       1       2       3

i=0  curr=[10] -> advance      after n advances, curr is at index n
i=1  curr=[20] -> advance
i=2  curr=[30] -> advance
      curr=[40]  → returned

```

THE LOOP GUARD IS NOT OPTIONAL

Without `&& curr != nullptr`, asking for index 10 of a 4-node list dereferences `nullptr` and crashes. With it, the function returns `nullptr` and the caller decides. Out-of-range access is the single most common way linked-list code fails in production, and it costs one condition to prevent.

Do not build an `operator[]` on top of this

```

for (int i = 0; i < length(head); ++i)      // O(n) length, EVERY iteration
    std::cout << getNth(head, i)->data;    // O(i) walk,   EVERY iteration
// total: O(n^2). On 100 000 nodes this runs for minutes.

for (Node* c = head; c; c = c->next)        // O(n) total
    std::cout << c->data;

```

THE RULE

Index-style access on a linked list is a *debugging* convenience, never an iteration strategy. If your code contains a loop whose body calls `getNth`, you have written an accidental $O(n^2)$ algorithm. Traverse with a cursor instead — and if you genuinely need indexing, you needed an array.

Note the useful special case: the "n-th node from the end" is **not** solved by computing the length and calling `getNth`. A single-pass gap-pointer method does it in one traversal (Topic 48), and that pattern is the gateway to the entire two-pointer section.

get n-th node – Time $O(n)$ · Space $O(1)$ · never call it inside a loop

PRACTICE SET — Topics 12-28

A. Conceptual (answer in your own words)

1. Why must `insertAtHead` take `Node*& head` while `updateAtPosition` can take `Node* head`? State the rule that decides it.
2. Delete-head is $O(1)$ but delete-tail is $O(n)$, even if you cache a tail pointer. Explain precisely why the tail pointer does not help.
3. In `insertSorted`, the loop tests `curr->next->data < value` rather than `curr->data < value`. What goes wrong with the second form, and why can't you recover from it?
4. A sorted array supports binary search in $O(\log n)$. A sorted linked list does not. Give the recurrence that proves it, and name the structure invented to fix it.
5. Give the three-step sequence that every deletion must follow, and name the specific bug produced by each possible wrong ordering.
6. Why does `std::forward_list` have no `size()` member? Tie your answer to the pre-C++11 behaviour of `std::list::size()`.
7. Explain why writing `~Node() { delete next; }` is a trap, given that it is technically correct.
8. "Swap the nodes" and "swap the values" produce the same printed output. Name a real data structure that breaks if you choose the wrong one, and say why.

B. Pointer tracing (pen and paper — draw the boxes)

Start from `head -> [10] -> [20] -> [30] -> [40] -> null` for each question, unless stated otherwise.

1. Trace `insertAtPosition(head, 25, 2)`. Give the value of `prev` at every iteration and the final list.
2. Trace `deleteTail(head)`. Give `prev` at every iteration, and state exactly which guard prevents a crash if the list had only one node.
3. Someone writes head insertion as `head = n; n->next = head;`. Draw the resulting structure and state (a) how many nodes are leaked and (b) what an ensuing traversal does.
4. Trace `deleteAllOccurrences(head, 3)` on `[3] -> [3] -> [5] -> [3]`. Then re-trace it with `prev = prev->next;` wrongly executed after each deletion, and give the incorrect final list.
5. For the list above, how many nodes does `while (curr)` visit, and how many does `while (curr->next)` visit? Which one leaves the cursor on the tail?
6. Give the exact output of `printBackward` on `[1] -> [2] -> [3] -> [4]`, then draw the call stack at its deepest point.
7. `buildFromArray` is rewritten without the tail pointer, walking from the head to append each element. Give the exact number of pointer dereferences for $n = 5$, and the complexity class.
8. Count the total operations for a loop over 1000 elements that calls `getNth(head, i)` each iteration. Give the exact count and the complexity class.

C. Code tasks

1. Implement all seventeen operations of this section as a single `LinkedList` class with a cached `tail` and a cached `size`. Then write a test that deletes the tail and verifies *both* cached fields are still correct — this is the audit described in Topic 17.
2. Write `insertAtPosition` using a **dummy node** so that `pos == 0` requires no special case at all. Compare the line count with the version in Topic 18.

3. Implement `deleteAllOccurrences` both ways — with a dummy and without — and test both against `[3,3,3]`, `[3]`, `[]`, and `[1,2,3]`.
4. Write a `validate(Node* head)` function that returns false if the list contains a cycle, using only $O(1)$ extra space. (You are re-deriving Topic 50 from scratch; do it before you read it.)
5. Take a correct `deleteList` and introduce the use-after-free bug. Build both under `g++ -O2 -fsanitize=address` and note that the buggy one may still print correct output without the sanitiser.
6. Write `getNthFromEnd(head, n)` in a single traversal, without computing the length. (Topic 48 — attempt it first, then compare.)

ANSWER KEY — Topics 12-28

A. Conceptual

1. The rule: **if the operation can change which node is first, the caller's head must be updated.** Head insertion always changes it, so it needs `Node*&` (or must return the new head). Updating a value never changes the structure, so a by-value pointer is enough. Passing `Node*` by value and assigning to it modifies only the local copy — and compiles silently.
2. To unlink the tail you must write `nullptr` into the **second-to-last** node's `next`. A tail pointer gives you the last node, and a singly linked list has no `prev`, so the only way to reach its predecessor is to walk from the head: $O(n)$. This is the single strongest argument for a doubly linked list.
3. Testing `curr->data` stops on the first node greater than or equal to the value — one node too far. You need its predecessor to splice, and an SLL has no reverse gear, so you cannot back up; you would have to restart from the head. Hence the "look one ahead, stop on the predecessor" pattern.
4. Binary search needs the midpoint in $O(1)$; in a linked list finding it is $O(n)$, so $T(n) = T(n/2) + O(n)$, which solves to **$O(n)$** — no better than a linear scan. The structure invented to fix this is the **skip list**, which adds probabilistic express lanes for $O(\log n)$ expected search.
5. **Save → unlink → delete.** Delete before saving `next` → *use-after-free*. Unlink without deleting → *memory leak*. Delete without unlinking → *dangling pointer* still reachable from the list, and the next traversal reads freed memory.
6. Because `size()` and $O(1)$ `splice()` of a sub-range are mutually exclusive: splicing a range does not reveal how many nodes moved, so maintaining a counter forces an $O(n)$ count. Pre-C++11 `libstdc++` chose $O(n)$ `size()` to keep splice fast; C++11 mandated $O(1)$ `size()` and splice-a-range became $O(n)$. `forward_list`, designed as the zero-overhead option, refused the trade-off by omitting `size()` entirely.
7. It is correct but recursive by construction: `delete head` destroys node 1, whose destructor deletes node 2, whose destructor deletes node 3 — one live stack frame per node. A list of ~100 000 nodes overflows the stack, and the crash appears at destruction time, far from the code that built the list.
8. An **LRU cache**. Its hash map stores `Node*` values; the list is reordered on every access. Swapping node *contents* would leave the map's pointers aimed at nodes now holding different keys, silently corrupting every lookup. Relinking preserves the identity that the external pointers depend on.

B. Pointer tracing

1. `pos - 1 = 1`, so one iteration: `prev` starts at `[10]`, becomes `[20]`. Then `prev->next = new Node(25, [30])`. Result: `[10] -> [20] -> [25] -> [30] -> [40]`.
2. `prev = [10] → [20] → [30]`, stopping when `prev->next->next == nullptr`. Deletes `[40]`, sets `[30]->next = nullptr`. The guard `if (head->next == nullptr)` is what saves the one-node case: without it, `prev->next->next` would dereference `nullptr`.
3. `head` is overwritten first, so `n->next = head` assigns `n` to itself: a **self-loop**. (a) All four original nodes are leaked — nothing points to `[10]` any more. (b) Traversal never reaches `nullptr`: an infinite loop printing the new value forever.
4. Correct: dummy→[3] unlink; dummy→[3] unlink; dummy→[5] advance; [5]→[3] unlink; stop. Final: `[5]`. With the wrong advance: unlink first 3, advance to the second 3, test [5] — advance, test [3] — unlink, stop. Final: `[3] -> [5]` — one survivor.

5. `while (curr)` visits 4 nodes and ends with `curr == nullptr`. `while (curr->next)` visits 3 and ends with `curr` on [40] — **the second form leaves the cursor on the tail**, which is why every tail-related operation uses it.
6. Output: 4 3 2 1. Deepest stack (top last): `printBackward(1) → (2) → (3) → (4) → (nullptr)` — five frames; the null frame returns and printing unwinds upward.
7. Appends require walks of length 0, 1, 2, 3, 4 → $0+1+2+3+4 = 10$ dereferences, i.e. $n(n-1)/2$, so $O(n^2)$. With a tail pointer it is 4, i.e. $O(n)$.
8. Each iteration does an $O(n)$ `length()` plus an $O(i)$ `getNth`. The `getNth` walks alone total $0+1+...+999 = 499\,500$ steps; adding the repeated length calls gives another $1000 \times 1000 = 1\,000\,000$. Roughly **1.5 million operations to print 1000 elements**, versus 1000 for a cursor loop. $O(n^2)$.

C. Code tasks — what to verify

- **C1.** The test should fail on a first attempt: almost everyone forgets to update `tail` inside `delete-tail` and `delete-by-value`. That failure is the lesson.
- **C2.** The dummy version drops the entire `if (pos == 0)` branch — typically five lines shorter, and it is the version to write in an interview.
- **C3.** The no-dummy version needs an explicit leading loop to strip matching heads before the main loop. [3,3,3] is the case that exposes a missing one; [] exposes a missing null guard.
- **C4.** Floyd's tortoise and hare: advance `slow` by one and `fast` by two; a cycle exists iff they ever meet. $O(n)$ time, $O(1)$ space.
- **C5.** Expect the unsanitised buggy build to print correct output — freed memory is usually not yet overwritten. ASan reports `heap-use-after-free` immediately. This is why "it worked when I ran it" is not evidence.
- **C6.** Advance `fast` by n nodes first, then advance `slow` and `fast` together until `fast` hits null; `slow` is the answer. One pass, $O(1)$ space — and it generalises to Topics 49 and 54.

HANDNOTE SUMMARY CARD

THE THREE QUESTIONS BEFORE WRITING ANY OPERATION

1. Can the FIRST node change? -> Node*& head, or return the new head
2. Do I need the PREDECESSOR? -> stop one early: while (curr->next ...)
3. Am I removing a node? -> SAVE next, THEN unlink, THEN delete

THE TWO PRIMITIVES – EVERYTHING ELSE IS A WRAPPER

```
SPLICE IN    prev->next = new Node(v, prev->next);
UNLINK OUT   Node* d = prev->next;
              prev->next = d->next;
              delete d;
```

THE TWO LOOP SHAPES

```
while (curr)      visits every node, ends curr == nullptr
while (curr->next) stops ON the tail, needs an empty-list guard first
```

COMPLEXITY TABLE – SINGLY LINKED LIST

create node	0(1)	search value	0(n)
build n nodes	0(n)	index of value	0(n)
traverse	0(n)	length (computed)	0(n)
insert head	0(1)	length (cached)	0(1)
insert tail no tail*	0(n)	get n-th node	0(n)
insert tail tail*	0(1)	update by position	0(pos)
insert position	0(pos)	update by value (all)	0(n)
insert sorted	0(n)	delete head	0(1)
delete tail	0(n)	<- even WITH a tail pointer	
delete position	0(pos)	delete by value	0(n)
delete whole list	0(n)	binary search	IMPOSSIBLE

THE FIVE BUGS THAT COMPILE CLEANLY

- 1 Node* head by value caller's head never changes
- 2 head = n; n->next = head self-loop + whole list leaked
- 3 delete p; p = p->next use-after-free
- 4 advance prev after unlink skips a node (fails on [3,3,3])
- 5 no null guard in getNth crash on out-of-range index

ALWAYS-TEST CASES empty · one node · head · tail · out of range
all-duplicates [3,3,3] · not found

DUMMY NODE (stack-allocated, cannot leak)

```
Node dummy(0, head); Node* prev = &dummy;
... operate uniformly, head is no longer special ...
head = dummy.next;
```

RECURSION

```
process THEN recurse -> forward      recurse THEN process -> backward
0(n) stack frames, ~100k nodes = stack overflow. Iterate in production.
```

NEVER for (i..n) getNth(head, i) accidental 0(n^2)
ALWAYS for (Node* c = head; c; c = c->next)

CONSOLIDATED CHEATSHEET — PART I

DEFINITION

```
struct Node { int data; Node* next; Node(int d, Node* n=nullptr)
              : data(d), next(n) {} };
```

The list IS the head pointer. empty $\Leftrightarrow$ head == nullptr

MEMORY — 64-BIT, GLIBC

sizeof(Node) 16 align 8 4 padding bytes real heap chunk 32 B
SLL and DLL of an int both cost 32 B -> prev is effectively free
1e6 ints: vector 4 MB (100%) · SLL 32 MB real (12.5%)
overhead is PER NODE, not per byte -> fat payloads amortise it

CACHE

64-byte line: array 1 miss / 16 ints · list 1 miss / node
dependent loads -> prefetcher blind -> misses serialise ~200-300 cyc
measured: vector scan 5-20x faster than list scan. Both $O(n)$.

PICK THE STRUCTURE

LIST pointer stability · $O(1)$ splice · intrusive / free lists · LRU
VECTOR indexing · binary search · iteration speed · nearly everything

THE UNLINK PRIMITIVE

```
Node* d = prev->next;
prev->next = d->next;
delete d;
```

THE SPLICE PRIMITIVE

```
prev->next = new Node(v, prev->next);
```

HEAD-CHANGING? -> Node*& head | return new head | dummy node
NEED PREV? -> while (curr->next != nullptr) (guard empty list first)
DELETING? -> save next -> unlink -> delete

$O(1)$ insert head · delete head · insert/delete at a KNOWN pointer
 insert tail WITH a tail pointer · cached size · splice
 $O(n)$ everything else, including delete tail and sorted search
 $O(n^2)$ any loop whose body calls getNth() or length()

TEST MATRIX [] [x] head middle tail out-of-range [3,3,3]

WHAT COMES NEXT (Volume II)

Reversal (29-33) · Doubly (34-39) · Circular (40-45)
Two-pointer patterns (46-55) — the gap method from Topic 28 opens it
Sorting & merging (56-62) · Rearrangement (63-73)
Advanced variants (74-81) · C++ tools & safety (82-86) · Interview (87-88)

End of Volume I — Topics 1-28 of 88. Mark each topic in your tracker as blank / L (learning) / D (done and solved), then solve 10-15 problems on singly linked list operations before starting Volume II. The two-pointer section will assume every operation in Section 3 is reflex-level.

PART II

Variants

Reversal · Doubly Linked · Circular Linked — Topics 29–45

SECTION 4 — REVERSAL

29. Reverse Iteratively

The single most asked linked-list question, and the template for the whole section. The task: turn `1 → 2 → 3 → null` into `3 → 2 → 1 → null` by **relinking nodes**, not by moving data, in $O(1)$ extra space.

The three-pointer method

```
Node* reverse(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;

    while (curr != nullptr) {
        Node* nxt = curr->next; // 1. SAVE the rest of the list
        curr->next = prev;      // 2. FLIP this node's link backwards
        prev = curr;           // 3. advance prev
        curr = nxt;            // 4. advance curr
    }
    return prev;               // curr is null; prev is the new head
}
```

```
start   prev=null   curr=[1] -> [2] -> [3] -> null

iter 1  nxt=[2]     null <- [1]   [2] -> [3] -> null
                        ^prev    ^curr

iter 2  nxt=[3]     null <- [1] <- [2]   [3] -> null
                        ^prev    ^curr

iter 3  nxt=null    null <- [1] <- [2] <- [3]   null
                        ^prev    ^curr

loop ends (curr == null). return prev = [3].
```

WHY THE SAVE MUST COME FIRST

Line 2 overwrites `curr->next` — the only reference to the unreversed remainder. Once overwritten, the rest of the list is unreachable and leaked. Saving into `nxt` first is the same "save before you destroy" discipline as the delete primitive in Topic 22. Every reversal variant in this section is this loop, fenced by extra bookkeeping.

What breaks around it

THE OLD HEAD IS NOW THE TAIL

If your list caches a `tail` pointer, reversal invalidates it: the new tail is the old head. Swap them (`std::swap(head, tail)`) or you will append to the middle of the list on the next `push-Back`. Same for any external pointer that meant "the last node."

Edge cases need no special handling. Empty list: the loop body never runs, `prev` stays `nullptr`, correct. One node: one iteration sets `[1]->next = nullptr` (it already was) and returns `[1]`, correct.

reverse iteratively – Time $O(n)$ · Space $O(1)$ · n pointer writes, one pass

30. Reverse Recursively

```
Node* reverseRec(Node* head) {  
    // 0 or 1 node  
    if (head == nullptr || head->next == nullptr) return head;  
  
    // 1. reverse everything AFTER head  
    Node* newHead = reverseRec(head->next);  
  
    // 2. make my successor point BACK at me  
    head->next->next = head;  
    head->next      = nullptr;           // 3. I am now the tail  
  
    // 4. propagate the new head unchanged  
    return newHead;  
}
```

The line everybody stares at

At the frame holding `head = [2]`, with `[3]` already reversed:

```
[1] -> [2] -> [3] -> null      head=[2], head->next=[3]  
                        (already the reversed tail-end)
```

```
head->next->next = head    means    [3]->next = [2]
```

```
[1] -> [2] <- [3]  
      |_____ still points forward at [3] -> a 2-cycle!
```

```
head->next = nullptr      cuts it
```

```
[1] -> [2] <- [3]        and [2]->next = null  
      (correct: [3] -> [2] -> null)
```

READ IT AS A CONTRACT

`reverseRec(head->next)` promises to return the head of the fully reversed remainder, and guarantees that `head->next` — my old successor — is now the *last* node of that reversed piece. So all I must do is attach myself behind it and terminate. Trusting the recursive contract instead of tracing every frame is the skill this problem is really testing.

Iterative versus recursive, honestly

	Iterative	Recursive
Time	$O(n)$	$O(n)$
Space	$O(1)$	$O(n)$ stack
Safe at $n = 10^6$	Yes	No — stack overflow
Tail-call optimisable	n/a	No — work remains after the call
Use it when	Always, in production	Explaining the idea; when the problem is naturally recursive (Topic 72)

reverse recursively — Time $O(n)$ · Space $O(n)$ stack · ~100k nodes will crash

31. Reverse in Groups of K

Reverse the first k nodes, then the next k , and so on: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ with $k=2$ becomes $2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 5$.

```
Node* reverseKGroup(Node* head, int k) {
    // 1. Are there k nodes left? If not, leave the remainder untouched.
    Node* check = head;
    for (int i = 0; i < k; ++i) {
        if (check == nullptr) return head;
        check = check->next;
    }

    // 2. Reverse exactly k nodes (Topic 29's loop, bounded by a counter).
    Node* prev = nullptr;
    Node* curr = head;
    for (int i = 0; i < k; ++i) {
        Node* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }

    // 3. head is now the LAST node of this reversed block. Attach the rest.
    head->next = reverseKGroup(curr, k);

    return prev;          // prev is the FIRST node of this block
}
```

1 -> 2 -> 3 -> 4 -> 5, k = 2

block 1: reverse [1,2] 2 -> 1 head=[1] is now the block tail
[1]->next = reverseKGroup([3], 2)

block 2: reverse [3,4] 4 -> 3
[3]->next = reverseKGroup([5], 2)

block 3: only 1 node left, k=2 -> step 1 returns head unchanged: [5]

stitch: 2 -> 1 -> 4 -> 3 -> 5

THE TWO ROLES OF ONE POINTER

After reversing a block, **head** (the node you started from) is the block's *tail*, and **prev** is its *head*. Every group-reversal problem is bookkeeping around that single swap of roles: return **prev** upward, attach the next group to **head->next**.

READ THE PROBLEM STATEMENT FOR THE REMAINDER

Two conventions exist and they give different answers. **LeetCode 25** leaves a trailing group of fewer than k nodes *unreversed* — that is the **return head** in step 1. The other convention reverses the short remainder too; to get it, delete the length check and simply reverse **min(k, remaining)** nodes. Ask which one is wanted before writing a line.

Space is $O(n/k)$ stack frames for the recursive form. A fully iterative version keeps a **groupPrev** pointer and stitches blocks as it goes — $O(1)$ space, and worth writing once.

reverse in groups of k — Time $O(n)$, each node touched once · Space $O(n/k)$ recursive, $O(1)$ iterative

32. Reverse Sublist (m to n)

Reverse only positions $m..n$, 1-based, in **one pass**: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ with $m=2$, $n=4$ becomes $1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 5$.

```
Node* reverseBetween(Node* head, int m, int n) {
    if (head == nullptr || m >= n) return head;

    // sentinel: handles m == 1 with no special case
    Node dummy(0, head);
    Node* prev = &dummy;

    for (int i = 1; i < m; ++i)        // walk to the node BEFORE position m
        prev = prev->next;

    // curr stays pinned on the original m-th node
    Node* curr = prev->next;

    for (int i = 0; i < n - m; ++i) {    // "pull to front" n-m times
        Node* mov = curr->next;          // the node to move
        curr->next = mov->next;          // unlink it
        // put it at the front of the block
        mov->next = prev->next;
        prev->next = mov;
    }
    return dummy.next;
}
```

```
1 -> 2 -> 3 -> 4 -> 5,    m=2, n=4
prev=[1]   curr=[2]  (curr NEVER moves – it sinks as others jump over it)

pull 1: mov=[3]    1 -> 3 -> 2 -> 4 -> 5
pull 2: mov=[4]    1 -> 4 -> 3 -> 2 -> 5

n-m = 2 pulls. Done, single pass, O(1) space.
```

THE HEAD-INSERTION TECHNIQUE

Rather than reversing the block and re-attaching both ends, repeatedly take the node *after* **curr** and move it to the *front* of the block. **curr** stays fixed and drifts to the back on its own, so the two boundary links are written once and never juggled. This same "pull-to-front" idea reappears in rotation (Topic 63) and reorder (Topic 66).

The dummy node earns its keep here: when $m = 1$ the reversed block includes the head, and without a sentinel that requires a whole separate branch. With one, **prev** is simply **&dummy** and **dummy.next** carries the new head out.

reverse sublist – Time $O(n)$ one pass · Space $O(1)$ · $n-m$ pointer triples rewritten

33. Reverse Alternate K Nodes

Reverse k nodes, skip k , reverse k , skip k ... $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$ with $k=2$ becomes $2 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 5 \rightarrow 7 \rightarrow 8$.

```

Node* reverseAltK(Node* head, int k) {
    if (head == nullptr) return nullptr;

    Node* curr = head;
    Node* prev = nullptr;
    int count = 0;

    while (curr != nullptr && count < k) {      // 1. REVERSE k nodes
        Node* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
        ++count;
    }

    // 2. old head (now block tail) -> skipped run
    head->next = curr;

    count = 0;
    // 3. SKIP k-1 more, landing on the k-th
    while (curr != nullptr && count < k - 1) {
        curr = curr->next;
        ++count;
    }

    // 4. recurse from the node AFTER the skip
    if (curr != nullptr)
        curr->next = reverseAltK(curr->next, k);

    return prev;
}

```

1 2 3 4 5 6 7 8, k = 2

reverse [1,2] 2 -> 1, prev=[2], curr=[3], head=[1] is block tail
 head->next = curr 2 -> 1 -> 3 ...

skip k-1 = 1 step curr moves [3] -> [4] (now on the LAST skipped node)
 curr->next = reverseAltK([5], 2)

recursion: reverse [5,6] -> 6 -> 5, skip [7,8]

final: 2 -> 1 -> 3 -> 4 -> 6 -> 5 -> 7 -> 8

WHY THE SKIP LOOP RUNS $k-1$ TIMES, NOT k

After the reversal loop, **curr** is *already standing on* the first node to skip. You need to land on the *last* node of the skipped run, so that its **next** can receive the recursive result — that is $k-1$ further steps, not k . Off-by-one here is the single most common failure on this problem, and it silently drops or duplicates a node rather than crashing.

The null guards in both loops handle every short-list case: fewer than k nodes remaining, or a skipped run that ends the list. Trace $k=3$ on a 4-node list by hand before trusting it.

reverse alternate k – Time $O(n)$ · Space $O(n/2k)$ stack · each node visited once

PRACTICE SET — Topics 29-33

A. Conceptual (answer in your own words)

1. In the three-pointer loop, what exactly is lost if you write `curr->next = prev;` before saving `nxt`? Describe the state of the list immediately afterwards.
2. Why does iterative reversal return `prev` rather than `curr`? What is the value of `curr` when the loop exits?
3. Explain `head->next->next = head;` in one sentence, then explain why the very next line must be `head->next = nullptr;`.
4. Recursive reversal cannot be tail-call optimised. Why not — and how does that differ from `printForward` in Topic 15?
5. A list caches a `tail` pointer. Name every field that becomes wrong after a full reversal, and state what an unfixed `pushBack` would then do.
6. In Topic 32, `curr` is never reassigned inside the loop, yet it ends up at the back of the reversed block. Explain how, without tracing.
7. Give the two conventions for the trailing remainder in reverse-in-k-groups, and say which line of code selects between them.
8. Why is reversing by swapping `data` fields between the front and back halves an unacceptable solution, even though it prints the right answer?

B. Pointer tracing (pen and paper)

1. Trace `reverse()` on `[7] → [8] → [9]`. Give `prev`, `curr` and `nxt` at the top of every iteration, and the returned pointer.
2. Trace `reverseRec()` on `[1] → [2] → [3]`. Draw the call stack at maximum depth, then give the list state after each frame's step 2 and step 3.
3. `reverseKGroup([1,2,3,4,5,6,7], 3)`. Give the result, and state how many nodes the trailing group has and whether they are reversed.
4. `reverseBetween([1,2,3,4,5,6], 3, 5)`. Give `prev`, `curr`, and the full list after each of the pulls. How many pulls occur?
5. `reverseAltK([1,2,3,4,5,6,7], 3)`. Give the result and the value of `curr` at the moment step 4 executes in the outermost call.
6. Someone changes Topic 33's skip loop from `count < k - 1` to `count < k`. Give the output for `[1..8]`, $k=2$, and say which node is corrupted.
7. Reverse a 5-node list with the iterative method. Count the exact number of pointer *writes* performed (assignments to a `next` field only), then generalise to n .
8. Run `reverseBetween` with $m = n = 3$ on a 5-node list. What does the guard `m >= n` prevent, and what would happen without it?

C. Code tasks

1. Implement `reverseKGroup` **iteratively**, in $O(1)$ space, using a `groupPrev` pointer and a dummy node. Test against the recursive version on $n = 0 \dots 10$ for every $k = 1 \dots 5$.
2. Write a `reverse()` for a class that caches both `head` and `tail`, and add an assertion that `tail->next == nullptr` after the call. Deliberately omit the swap first and watch the assertion fire.
3. Implement `reverseBetween` using the "reverse the block then re-attach" method instead of pull-to-front. Compare line counts and the number of boundary pointers you had to name.

4. Build a 1 000 000-node list and call the recursive reversal. Record where it crashes, then run the iterative one on the same list and confirm it succeeds.
5. Write `isPalindrome(Node*)` by reversing the second half in place, comparing, and **restoring the list before returning**. Verify with an assertion that the input is unchanged. (This is Topic 54 — attempt it now, from reversal alone.)

ANSWER KEY — Topics 29-33

A. Conceptual

1. Everything from `curr` onward is lost. `curr->next` is the *only* pointer to the unreversed remainder; overwriting it with `prev` leaves those nodes unreachable — leaked — and the loop then terminates one step later having reversed a two-node fragment.
2. The loop exits when `curr == nullptr` (it has walked past the end), so `curr` is null and useless. `prev` trails one node behind and therefore holds the last node processed — the original tail, which is the new head.
3. "Make my successor point back at me." It is required because `head->next` still points *forward* at that successor, so the two nodes now point at each other — a 2-cycle. Setting `head->next = nullptr` cuts the forward link and makes `head` the tail.
4. Because real work — the two relink lines — happens *after* the recursive call returns, so the frame must survive the call and cannot be replaced. `printForward` recurses as its very last action with nothing pending, so GCC at -O2 rewrites it into a loop.
5. `tail` becomes wrong (it now points at the new *head*), and `head` must be reassigned to the returned pointer. An unfixed `pushBack` would append to the node that is now *first*, splicing the new element into position 2 and silently orphaning the rest of the list.
6. Because every pull removes the node immediately after `curr` and reinserts it at the front of the block. `curr` stays physically put while $n-m$ nodes are moved in front of it, so its relative position slides from first to last without it ever being reassigned.
7. Leave a short trailing group unreversed (LeetCode 25), or reverse it anyway. The selecting line is the length check — `if (check == nullptr) return head;`. Deleting that pre-check gives the second convention.
8. It breaks **node identity**. Any external pointer aimed at a specific node — a hash map in an LRU cache, an iterator, an intrusive kernel list member — now refers to a node holding a different value. It is also $O(n)$ space or $O(n^2)$ time for a singly linked list, since you cannot walk backwards. Reversal problems are about relinking; that is the point of the exercise.

B. Pointer tracing

1. i1: `prev=null, curr=[7], nxt=[8]`. i2: `prev=[7], curr=[8], nxt=[9]`. i3: `prev=[8], curr=[9], nxt=null`. Exit: `prev=[9], curr=null`. Returns **[9]**; list is `9 → 8 → 7 → null`.
2. Frames (deepest last): `rev([1]) → rev([2]) → rev([3])`. `rev([3])` returns [3] immediately (base case). In `rev([2])`: step 2 gives `3→2` plus `2→3` (a 2-cycle); step 3 cuts it to `3→2→null`. In `rev([1])`: step 2 gives `2→1`; step 3 cuts `1→2`. Final `3→2→1→null`, and **[3] is returned unchanged through every frame**.
3. `3 → 2 → 1 → 6 → 5 → 4 → 7`. The trailing group has **1 node** and is **not reversed**, because the length pre-check returns early.
4. `prev=[2], curr=[3]`. Pull 1 (`mov=[4]`): `1,2,4,3,5,6`. Pull 2 (`mov=[5]`): `1,2,5,4,3,6`. **2 pulls** ($n-m = 5-3$).
5. Result `3 → 2 → 1 → 4 → 5 → 6 → 7...` wait, trace it: reverse `[1,2,3] → 3,2,1`; skip `[4,5,6]`; reverse `[7] → [7]`. Final `3 → 2 → 1 → 4 → 5 → 6 → 7`. At step 4 in the outermost call, `curr = [6]` — the last node of the skipped run, having advanced $k-1 = 2$ steps from [4].
6. The skip loop advances one node too far, landing on the first node of the *next* block. The result is `2 → 1 → 3 → 4 → 5 → 6 → 8 → 7` — wrong: **[5,6] is not reversed** because the recursion started one node late, and the alternation drifts out of phase from there on.

7. Exactly **5** writes — one `curr->next = prev` per node, one per iteration. Generally **n** writes, plus $2n$ pointer-variable assignments that are not memory writes to nodes. This is why reversal is cheap: it touches each node's link exactly once.
8. It prevents a no-op from doing work: with $m == n$ the block is a single node and there is nothing to reverse. Without the guard, `n - m = 0` so the pull loop runs zero times anyway and the function is still correct — but with $m > n$ (a caller error) the walk to m could run off the end and dereference null. The guard is defensive, not load-bearing for the valid case.

C. Code tasks — what to verify

- **C1.** The iterative version needs `groupPrev` (node before the block) and `groupNext` (node after it); after reversing, the old block head becomes the new `groupPrev`. $k = 1$ must be an identity operation — that is the test people forget.
- **C2.** Without the swap, the assertion fires immediately: the cached tail points at the new head, whose `next` is non-null.
- **C3.** The re-attach method needs four named boundary pointers (`preM`, `nodeM`, `nodeN`, `postN`) and is typically 6–8 lines longer. Both are $O(n)/O(1)$; pull-to-front is simply harder to get wrong.
- **C4.** Expect a segfault from stack exhaustion somewhere between 100 000 and 250 000 frames depending on your stack limit (`ulimit -s`, usually 8 MB). The iterative version handles 10^6 without difficulty.
- **C5.** Find the middle with slow/fast, reverse the second half, compare the two halves, then reverse the second half again and re-link before returning. Skipping the restore is the usual bug: the function returns the right answer but leaves the caller's list mangled.

HANDNOTE SUMMARY CARD

THE THREE-POINTER LOOP – LEARN THIS BY MUSCLE MEMORY

```
Node* prev = nullptr;
Node* curr = head;
while (curr) {
    Node* nxt = curr->next;    // SAVE (or the rest of the list is lost)
    curr->next = prev;         // FLIP
    prev = curr;              // ADVANCE
    curr = nxt;
}
return prev;                  // curr == nullptr, prev == new head
```

RECURSIVE FORM

```
if (!head || !head->next) return head;
Node* newHead = reverseRec(head->next);
head->next->next = head;    // successor points BACK at me
head->next = nullptr;      // ...and cut, or you get a 2-cycle
return newHead;            // passed up UNCHANGED through every frame
```

AFTER ANY REVERSAL – THE ROLE SWAP

```
old head -> new TAIL      old tail -> new HEAD
cached tail pointer is now WRONG. swap(head, tail).
```

THE FOUR VARIANTS

29 whole list	three-pointer loop	$O(n) / O(1)$
31 groups of k	check k exist -> reverse k ->	$O(n) / O(n/k)$
32 sublist m..n	dummy + PULL-TO-FRONT, n-m pulls	$O(n) / O(1)$
33 alternate k	curr never moves; others jump over it	
	reverse k, head->next = curr,	$O(n) / O(n/2k)$
	skip k-1 (NOT k), recurse from curr->next	

PULL-TO-FRONT (topic 32 inner loop)

```
Node* mov = curr->next;
curr->next = mov->next;
mov->next = prev->next;
prev->next = mov;
```

ITERATIVE vs RECURSIVE

both $O(n)$ time. iterative $O(1)$ space, recursive $O(n)$ stack.
~100k nodes overflows the stack. NOT tail-call optimisable
(work remains after the call). Ship the iterative one.

TRAPS

save nxt BEFORE flipping	forgetting = whole remainder leaked
skip loop is k-1, not k	off-by-one, silent wrong answer
swapping DATA instead of links	breaks node identity (LRU, iterators)
remainder convention in k-groups	ASK: reverse it, or leave it?

PRE-TEST MATRIX [] · [x] · k=1 · k=n · k>n · m==n · m==1

SECTION 5 — DOUBLY LINKED LIST

34. DLL Node Structure

```
struct DNode {
    int    data;
    DNode* prev;
    DNode* next;

    DNode(int d, DNode* p = nullptr, DNode* n = nullptr)
        : data(d), prev(p), next(n) {}
};
```

```
null <- [ • | 10 | • ] <-> [ • | 20 | • ] <-> [ • | 30 | • ] -> null
           prev      next
head ----^                               ^---- tail
```

INVARIANT, and you should check it in every debug build:

- for every node *x* with *x*->next != null : *x*->next->prev == *x*
- for every node *x* with *x*->prev != null : *x*->prev->next == *x*
- head->prev == nullptr tail->next == nullptr

What it costs — the answer that surprises people

```
struct DNode { int data; DNode* prev; DNode* next; }; // 64-bit

off 0    [ int data ][ 4B PAD ][ DNode* prev  ][ DNode* next  ]
      0-3      4-7      8-15      16-23

sizeof(DNode) = 24      sizeof(Node) = 16      (SLL, Topic 3)
real heap chunk = 32    real heap chunk = 32    <- IDENTICAL
```

THE SECOND POINTER IS USUALLY FREE

glibc's minimum chunk on 64-bit is 32 bytes and chunk sizes round up to a multiple of 16, so a 16-byte request and a 24-byte request land in the same 32-byte chunk. For an `int` payload a doubly linked list costs **the same memory** as a singly linked list. What it genuinely costs is *writes*: every structural change now maintains two links instead of one, and every invariant has two halves to keep consistent. Memory is not the trade-off; correctness effort is.

The saving evaporates once the payload pushes the struct past a chunk boundary. With a 16-byte payload the SLL node is 24 bytes (chunk 32) while the DLL node is 32 bytes (chunk 32) — still equal. With a 24-byte payload: SLL 32 → chunk 32, DLL 40 → chunk 48. Compute it for your actual payload rather than assuming either way.

DNode — sizeof 24 · align 8 · 4 padding bytes · real heap cost 32 B, same as an SLL node

35. DLL Insertion

Every DLL insertion writes **four** pointers: two on the new node, one on each neighbour. Miss one and the list is traversable in one direction only — a bug that hides until someone walks backwards.

Insert at head

```
void pushFront(DNode*& head, DNode*& tail, int v) {
    DNode* n = new DNode(v, nullptr, head);    // prev=null, next=old head
    if (head != nullptr) head->prev = n;        // old head points back at n
    // list was empty: n is also the tail
    else
        tail = n;
    head = n;
}
```

Insert at tail — genuinely $O(1)$, unlike an SLL

```
void pushBack(DNode*& head, DNode*& tail, int v) {
    DNode* n = new DNode(v, tail, nullptr);    // prev=old tail, next=null
    if (tail != nullptr) tail->next = n;
    else
        head = n;                            // list was empty
    tail = n;
}
```

Insert after a known node — the general case

```
void insertAfter(DNode*& tail, DNode* p, int v) {
    DNode* n = new DNode(v);

    n->next = p->next;                        // 1. new node looks forward
    n->prev = p;                              // 2. new node looks back
    // 3. successor looks back at new node
    if (p->next != nullptr) p->next->prev = n;
    else
        tail = n;                            // (p was the tail)
    // 4. p looks forward at new node — LAST
    p->next = n;
}
```

ORDER IS NOT NEGOTIABLE

Step 4 must come **last**. It overwrites `p->next`, which steps 1 and 3 still need to read. Do it first and `n->next` points at `n` itself and the entire tail of the list is orphaned. The safe habit: *write the new node's links first, then the neighbours', then the link you are replacing.*

Insert before a known node

```
void insertBefore(DNode*& head, DNode* p, int v) {
    DNode* n = new DNode(v, p->prev, p);
    if (p->prev != nullptr) p->prev->next = n;
    else
        head = n;                            // p was the head
    p->prev = n;
}
```

WHAT A DLL BUYS YOU HERE

"Insert *before* a given node in $O(1)$ " is impossible in a singly linked list — you would need the predecessor, and there is no way back. In a DLL it is symmetric with insert-after. That symmetry is half the reason `std::list` exists.

DLL insert — $O(1)$ at head, at tail, and before/after any known node · 4 pointer writes each

36. DLL Deletion

The headline feature: **delete a node in O(1) given only that node**. No predecessor search, no traversal.

```
void unlink(DNode*& head, DNode*& tail, DNode* d) {
    if (d == nullptr) return;

    if (d->prev != nullptr) d->prev->next = d->next;    // bridge forward
    else                    head = d->next;             // d was the head

    if (d->next != nullptr) d->next->prev = d->prev;    // bridge backward
    else                    tail = d->prev;            // d was the tail

    delete d;
}
```

```
      [10] <-> [20] <-> [30]
              ^d

d->prev->next = d->next    [10] -----> [30]
d->next->prev = d->prev    [10] <----- [30]
delete d                [10] <-----> [30]
```

Four boundary cases, all handled by the two if/else pairs:

middle node	both branches take the "bridge" path
head	first else fires: head = d->next
tail	second else fires: tail = d->prev
only node	BOTH elses fire: head = nullptr, tail = nullptr

THIS IS THE WHOLE REASON LRU CACHES USE A DLL

An LRU cache stores **key** → **DNode*** in a hash map. On a cache hit it must move that node to the front — which means unlinking it from wherever it currently is. With a DLL that is O(1) from the stored pointer alone. With an SLL you would have to find the predecessor: O(n), and the whole design collapses. Topic 77 builds it; this function is its engine.

Delete by value and delete at position

```
void deleteByValue(DNode*& head, DNode*& tail, int v) {
    for (DNode* c = head; c != nullptr; c = c->next)
        // O(n) to FIND, O(1) to remove
        if (c->data == v) { unlink(head, tail, c); return; }
}
```

The search is still O(n) — a DLL does not make finding things faster. What it removes is the *second* O(n): the walk back to the predecessor.

THE DANGLING-POINTER RULE

After **unlink**, the caller's copy of **d** is a dangling pointer. If a hash map or another container still holds it, erase that entry **before** the **delete**. This is the most common crash in hand-written LRU caches.

DLL delete – O(1) given the node · O(n) to search by value · head/tail/only-node all handled

37. Forward and Backward Traversal

```
void printForward(DNode* head) {
    for (DNode* c = head; c != nullptr; c = c->next)
        std::cout << c->data << " ";
}

void printBackward(DNode* tail) {           // requires a tail pointer
    for (DNode* c = tail; c != nullptr; c = c->prev)
        std::cout << c->data << " ";
}
```

COMPARE WITH TOPIC 15

A singly linked list can only print backwards by recursing to the end and printing on the way back up — $O(n)$ stack, and a crash past ~100 000 nodes. A DLL walks backwards in a plain loop with $O(1)$ space. This is the same capability the SLL was *simulating* with the call stack; the `prev` pointer makes it real.

Finding the tail if you did not cache it

```
DNode* findTail(DNode* head) {
    if (head == nullptr) return nullptr;
    DNode* c = head;
    while (c->next != nullptr) c = c->next;    // O(n) – cache it instead
    return c;
}
```

Both directions from an arbitrary node

```
// print k nodes either side of p
void neighbourhood(DNode* p, int k) {
    DNode* left = p;
    for (int i = 0; i < k && left->prev; ++i) left = left->prev;
    for (DNode* c = left; c != nullptr; c = c->next) { /* ... */ }
}
```

This is what a text editor's cursor, a browser's back/forward history, and a music player's previous-track button all reduce to: a pointer into the middle of a DLL that must move both ways from wherever it currently is.

DLL traversal – $O(n)$ either direction · $O(1)$ space both ways · backward needs a tail pointer

38. Reverse a DLL

Reversing a doubly linked list means swapping `prev` and `next` in every node, then swapping the head and tail handles.


```

void reverseDLL(DNode*& head, DNode*& tail) {
    DNode* curr = head;

    while (curr != nullptr) {
        std::swap(curr->prev, curr->next); // flip this node's two links
        // prev is the OLD next – that is forward now
        curr = curr->prev;
    }
    std::swap(head, tail);
}

```

```

start    null <- [1] <-> [2] <-> [3] -> null      head=[1] tail=[3]

at [1]   swap: prev=[2] next=null
          advance via curr->prev -> [2]      (the old next!)

at [2]   swap: prev=[3] next=[1]
          advance -> [3]

at [3]   swap: prev=null next=[2]
          advance -> null, loop ends

swap(head, tail)    null <- [3] <-> [2] <-> [1] -> null

```

THE LINE THAT LOOKS WRONG BUT ISN'T

`curr = curr->prev;` to move *forward*. After the swap, this node's `prev` field holds what used to be its `next`. Advance via `next` instead and you walk straight back to where you came from — an infinite loop between two nodes. Do the swap, then read the field that now means "onward."

THE O(1) REVERSAL NOBODY MENTIONS

If reversal is a *view* rather than a permanent change, do not touch the nodes at all. Keep a `bool reversed` flag in the list object, and have your iterator follow `next` or `prev` depending on it. Reversal becomes one boolean flip — **O(1)** instead of $O(n)$ — and this is exactly how `std::reverse_iterator` and `rbegin()/rend()` work. Worth saying in an interview: "if the caller only needs to iterate in reverse, I would not mutate the list at all."

reverse a DLL – Time $O(n)$ in place · Space $O(1)$ · $O(1)$ if a direction flag will do

39. DLL Advantages and Costs

Operation	Singly linked	Doubly linked
Traverse forward	$O(n)$	$O(n)$
Traverse backward	$O(n)$ with $O(n)$ stack	$O(n)$, $O(1)$ space
Insert at head	$O(1)$	$O(1)$
Insert at tail (cached tail)	$O(1)$	$O(1)$
Insert <i>before</i> a known node	$O(n)$	$O(1)$
Delete a known node	$O(n)$ – needs predecessor	$O(1)$
Delete the tail	$O(n)$	$O(1)$
Reverse	$O(n)$	$O(n)$, or $O(1)$ with a flag
Pointer writes per insertion	1–2	4
Pointer writes per deletion	1	2
<code>sizeof</code> node (int payload)	16	24
Real heap cost (glibc)	32	32 – identical
Invariants to maintain	1	2 (both directions must agree)

THE DECISION RULE

Use a **doubly** linked list when you will hold pointers to interior nodes and delete or reposition them — LRU caches, editor undo stacks, browser history, scheduler run queues, any intrusive kernel list. Use a **singly** linked list when you only ever push and pop at the front — stacks, adjacency lists, hash-table chains, allocator free lists — where `prev` would be maintained for nothing.

The naive answer, "a DLL uses more memory," is usually false (Topic 34). The true cost is that every structural change has twice the invariant surface, so twice as many ways to leave the list half-correct.

Where you meet it in the standard library

Container	Structure	Notable
<code>std::list</code>	Circular DLL with a sentinel	$O(1)$ <code>size()</code> since C++11; $O(1)$ single-element <code>splice</code>
<code>std::forward_list</code>	Singly linked	No <code>size()</code> , no <code>back()</code> ; <code>insert_after</code> only

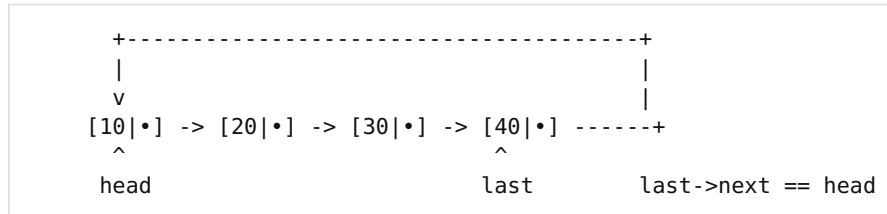
`std::list` being *circular with a sentinel* is not an implementation detail worth skipping: the sentinel is why `end()` is a valid dereferenceable-adjacent iterator, why `--end()` gives the last element, and why insertion needs no null checks at all. Section 6 builds exactly that structure.

choose a DLL for interior deletion and bidirectional movement · choose an SLL for front-only work

SECTION 6 — CIRCULAR LINKED LIST

40. Circular Singly Linked List

The last node's `next` points back at the first node instead of at `nullptr`. There is no null anywhere in the structure — which changes every loop you have written so far.



Keep a `last` pointer, not a `head` pointer

THE ONE DESIGN DECISION THAT MATTERS HERE

From `last` you reach the head in $O(1)$ — it is `last->next`. From `head` you cannot reach the last node in less than $O(n)$. So a single `last` pointer gives you **$O(1)$ insertion at both ends**, while a single `head` pointer gives you $O(1)$ at the front only. Store `last`; derive `head`.

```
void insertFront(Node*& last, int v) {
    Node* n = new Node(v);
    if (last == nullptr) {           // empty list
        last = n;
        // a one-node circle points at itself
        n->next = n;
        return;
    }
    n->next = last->next;           // n -> old head
    // last -> n (n is the new head)
    last->next = n;
}

void insertEnd(Node*& last, int v) {
    insertFront(last, v);           // same splice...
    // ...then just move the "last" label!
    last = last->next;
}
```

WHY `insertEnd` IS TWO LINES

In a circle, the head and the tail are adjacent — the position "just after `last`" is both "the front of the list" and "the back of the list." Inserting there and then relabelling which node is `last` converts a front-insertion into a back-insertion. Nothing moves; only the label does. This is the kind of observation circular-list questions are built to test.

Where circular singly linked lists are actually used

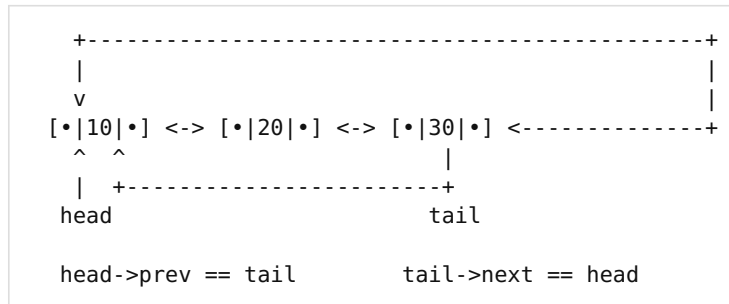
- **Round-robin schedulers** — each process gets a quantum, then `curr = curr->next`; the wrap-around is free.
- **Ring buffers** for streaming audio, logs, or network frames.
- **Multiplayer turn order** — after the last player, play returns to the first.

- **The Josephus problem** (Topic 44), which is really a repeated-deletion drill on a circle.

CSLL with a **last** pointer – insert front $O(1)$ · insert end $O(1)$ · no nulls in the structure

41. Circular Doubly Linked List

Every node has two valid neighbours. The head's **prev** is the tail; the tail's **next** is the head. **There is not a single null pointer in the entire structure.**



The sentinel version — how **std::list** is really built

```

struct DNode { int data; DNode* prev; DNode* next; };

DNode sentinel;                // a permanent, dataless node
sentinel.next = sentinel.prev = &sentinel; // empty list = sentinel alone

// insert v before position p – no branches, no null checks, ever
void insertBefore(DNode* p, int v) {
    DNode* n = new DNode{v, p->prev, p};
    p->prev->next = n;
    p->prev      = n;
}

// pushBack(v) is insertBefore(&sentinel, v)
// pushFront(v) is insertBefore(sentinel.next, v)

```

THE PAYOFF: EVERY SPECIAL CASE DISAPPEARS

Compare that four-line **insertBefore** with Topic 35's version and its **if (p->prev != nullptr) ... else head = n;**. With a sentinel, the empty list, the head, and the tail are all just "some node with two neighbours," so *there are no boundary branches at all*. This is why **std::list::end()** is a real iterator you can decrement: it points at the sentinel. It is also why **--end()** gives you the last element for free.

NEVER DELETE THE SENTINEL

clear() must free every node *except* the sentinel, then restore **sentinel.next = sentinel.prev = &sentinel;**. Freeing it, or forgetting to re-point it, leaves the list in a state where the very next insertion writes through a dangling pointer. Keep the sentinel as a by-value member of the list class so it cannot be deleted by accident.

CDLL with sentinel – $O(1)$ insert/delete anywhere, zero null checks, zero boundary branches

42. Circular Insertion and Deletion

Delete a node from a circular singly linked list

```
void deleteNode(Node*& last, int v) {
    if (last == nullptr) return;           // empty

    // Single-node circle
    if (last->next == last) {
        if (last->data == v) { delete last; last = nullptr; }
        return;
    }

    Node* prev = last;
    Node* curr = last->next;               // start at the head

    do {
        if (curr->data == v) {
            prev->next = curr->next;       // bridge over curr
            if (curr == last) last = prev; // deleted the last node
            delete curr;
            return;
        }
        prev = curr;
        curr = curr->next;
    } while (curr != last->next);
}
```

THE THREE CASES THAT BREAK NAIVE CODE

- **Single-node circle.** `last->next == last`. Deleting it must set `last = nullptr`, or you keep a dangling pointer to freed memory that every later call will dereference.
- **Deleting the head.** The head has a predecessor — the `last` node — so unlike a plain SLL there is no special case, provided you started `prev` at `last`.
- **Deleting `last` itself.** The `last` handle must be moved back to `prev` *before* the delete, or it dangles.

Insert after a given value

```
void insertAfterValue(Node*& last, int target, int v) {
    if (last == nullptr) return;
    Node* curr = last->next;
    do {
        if (curr->data == target) {
            curr->next = new Node(v, curr->next);
            // inserted at the very end
            if (curr == last) last = curr->next;
            return;
        }
        curr = curr->next;
    } while (curr != last->next);
}
```

Note the recurring shape: the splice itself is identical to the linear case, and all the extra code exists purely to keep the `last` handle honest. In a circular list, *the structure is robust and the*

handle is fragile — the opposite of a linear list, where the structure has ends that need guarding.

circular insert/delete — $O(1)$ at the ends with a `last` pointer · $O(n)$ by value

43. Traversal Termination

The single most important behavioural difference in this section. In a linear list you stop at `nullptr`. In a circular list **there is no `nullptr`**, so you must stop when you come back to where you started.

```
// WRONG — never terminates, and never crashes either. It just hangs.
for (Node* c = head; c != nullptr; c = c->next) visit(c);

// WRONG — visits nothing. The condition is false on the very first check.
Node* c = head;
while (c != head) { visit(c); c = c->next; }

// CORRECT — do-while: act first, test afterwards.
if (last != nullptr) {
    Node* c = last->next;           // the head
    do {
        visit(c);
        c = c->next;
    } while (c != last->next);
}
```

WHY IT MUST BE A `do-while`

The stop condition — "I am back at the head" — is *true at the start*. A `while` loop tests before the first iteration and exits immediately, visiting nothing. A `do-while` performs the visit first and tests afterwards, so the start position is visited exactly once and the loop ends on the return trip. Circular list, circular reasoning: **act, then check**.

The alternative: stop on the tail

```
Node* c = last->next;
while (true) {
    visit(c);
    // reached the tail — stop before wrapping
    if (c == last) break;
    c = c->next;
}
```

CACHE THE STOP MARKER BEFORE YOU MUTATE

If the loop body can delete or insert nodes, `last->next` may change mid-traversal and the condition `c != last->next` will then compare against a moving target — usually an infinite loop, occasionally a use-after-free. Save the start node into a local (`Node* start = last->next;`) and compare against that, and re-derive it if the head is removed.

Debugging a suspected circular structure

If you are unsure whether a list is circular or accidentally cyclic, do **not** print it — you will fill the terminal. Use Floyd's cycle detection (Topic 50), or bound the traversal: `for (int i = 0; c`

`&& i < 1000; ++i)`. An unintended cycle from a bad relink and a deliberate circular list are structurally identical; only your intent distinguishes them.

circular traversal – Time $O(n)$ · Space $O(1)$ · `do-while`, and cache the stop marker

44. Josephus Problem

n people stand in a circle. Counting from person 1, every k -th person is eliminated, and counting resumes from the next survivor. Which position survives?

Simulation with a circular list — $O(n \cdot k)$

```
int josephusSim(int n, int k) {
    if (n <= 0 || k <= 0) return -1;

    Node* last = nullptr;           // build 1..n
    for (int i = 1; i <= n; ++i) insertEnd(last, i);

    Node* prev = last;
    Node* curr = last->next;        // start at person 1

    while (curr->next != curr) {    // until one remains
        for (int step = 1; step < k; ++step) { // advance k-1 times
            prev = curr;
            curr = curr->next;
        }
        prev->next = curr->next;    // eliminate curr
        delete curr;
        // resume from the next
        curr = prev->next;
    }
    int survivor = curr->data;
    delete curr;
    return survivor;
}
```

The recurrence — $O(n)$ time, $O(1)$ space

Let $J(n, k)$ = the 0-based index of the survivor among n people.

$$J(1, k) = 0$$

$$J(n, k) = (J(n-1, k) + k) \bmod n$$

Why: after the first elimination, $n-1$ people remain in a circle that has been

ROTATED so that counting restarts at the person after the victim. Solve the smaller problem, then rotate the answer back by k positions.

```
int josephus(int n, int k) {
    int r = 0;                       // J(1,k) = 0
    for (int i = 2; i <= n; ++i)
        r = (r + k) % i;
    return r + 1;                   // convert to 1-based
}
```

THE TWO ANSWERS ARE BOTH THE POINT

The simulation shows you *understand the structure*; the recurrence shows you *understand the problem*. In an interview, write the simulation first — it is obviously correct — then say "but this is $O(n \cdot k)$ and there is an $O(n)$ recurrence," and derive it. The special case $k = 2$ has a closed form: writing $n = 2^m + 1$ with $0 \leq l < 2^m$, the survivor is $2l + 1$.

Check by hand: $n=5, k=2$. Eliminations are 2, 4, 1, 5; survivor is **3**. The loop gives $r = 0 \rightarrow (0+2)\%2 = 0 \rightarrow (0+2)\%3 = 2 \rightarrow (2+2)\%4 = 0 \rightarrow (0+2)\%5 = 2$, and $2 + 1 = 3$. ✓

Josephus – simulation $O(n \cdot k)$ time, $O(n)$ space · recurrence $O(n)$ time, $O(1)$ space

45. Split a Circular List

Split a circular singly linked list into two *separate circular lists* of equal size (the first gets the extra node when n is odd). Classic slow/fast application.

```
void splitCircular(Node* head, Node*& first, Node*& second) {
    if (head == nullptr) { first = second = nullptr; return; }

    Node* slow = head;
    Node* fast = head;

    // advance until fast reaches the end of the circle
    while (fast->next != head && fast->next->next != head) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // even count: push fast to the last node
    if (fast->next->next == head) fast = fast->next;

    first = head;
    second = slow->next;

    // close the SECOND circle (last -> second head)
    fast->next = slow->next;
    // close the FIRST circle (mid -> first head)
    slow->next = head;
}
```

circle of 6: 1 2 3 4 5 6

loop: slow 1->2->3 fast 1->3->5
 fast->next->next == head, so fast advances to [6]

first = [1]	second = slow->next = [4]
fast->next = [4]	6 -> 4 closes 4 5 6
slow->next = [1]	3 -> 1 closes 1 2 3

result: (1 2 3) and (4 5 6), both circular, independent.

THE ODD/EVEN ASYMMETRY

For an **odd** count the loop exits with `fast->next == head`, so `fast` is already the last node. For an **even** count it exits with `fast->next->next == head`, so `fast` is one short and must be nudged forward. Omitting that single line silently drops the final node out of the second circle — and because the result is still a valid circle, nothing crashes and no test fails unless it checks the sizes.

THE PATTERN UNDERNEATH

Two pointers moving at different speeds locate the midpoint in one pass with $O(1)$ memory — no length computation, no second traversal. That is the entire content of Topic 46, and this problem is its first non-trivial application. The circular variant differs from the linear one only in the stop condition: compare against `head` instead of `nullptr`.

Both halves must be closed into proper circles before returning. Leaving either `next` pointing into the other half produces two lists that share nodes — a structure that looks fine when printed from one head and corrupts on the first deletion.

split a circular list – Time $O(n)$ single pass · Space $O(1)$ · slow/fast midpoint

PRACTICE SET — Topics 34-45

A. Conceptual (answer in your own words)

1. State the two DLL invariants. Then describe a bug that satisfies one and violates the other, and explain why it can survive weeks of testing.
2. "A doubly linked list uses 50% more memory than a singly linked list." Give the numbers that make this false for an `int` payload, and a payload size for which it becomes true.
3. In `insertAfter`, why must `p->next = n;` be the last of the four writes? Describe the resulting structure if it is done first.
4. Deleting a known node is $O(1)$ in a DLL and $O(n)$ in an SLL. Name the exact piece of information the SLL lacks, and name one real data structure that is impossible to build efficiently without it.
5. In `reverseDLL`, explain why the loop advances with `curr = curr->prev`. What happens if you use `curr->next` instead?
6. Describe the $O(1)$ alternative to physically reversing a DLL, and name the standard library feature built on that idea.
7. Why should a circular singly linked list be held by a `last` pointer rather than a `head` pointer? Give the two complexities that change.
8. Why must circular traversal use `do-while`? Give the exact output of the `while` version.
9. Explain how a sentinel node removes every boundary branch in a CDLL, and connect it to why `--std::list::end()` is valid.
10. An unintended cycle from a bad relink and a deliberate circular list are structurally identical. What practical consequence does this have for debugging, and what tool resolves it?

B. Pointer tracing (pen and paper)

Where a DLL is needed, start from `null ← [10] ⇄ [20] ⇄ [30] ⇄ [40] → null`.

1. Trace `unlink(head, tail, [10])`. Which branch of each `if/else` fires, and what are `head` and `tail` afterwards? Repeat for `[40]` and for a one-node list.
2. Trace `reverseDLL` fully. Give `curr->prev` and `curr->next` after the swap at every node, and the final `head` and `tail`.
3. Someone writes `insertAfter` with the four writes in the order 4, 1, 2, 3. Draw the resulting structure and say how many nodes are orphaned.
4. A CSLL holds $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ with `last = [4]`. Trace `insertEnd(last, 5)` step by step, giving `last` and `last->next` at each stage.
5. Same CSLL. Trace `deleteNode(last, 4)`. Which branch moves the `last` handle, and what breaks if that line is removed?
6. Compute `josephus(7, 3)` using the recurrence, showing `r` after each iteration. Then verify by writing out the elimination order by hand.
7. Compute `josephus(8, 2)` with the recurrence, then check it against the closed form $n = 2^m + 1 \Rightarrow \text{survivor} = 2l + 1$.
8. Trace `splitCircular` on a circle of 5 nodes. Give `slow` and `fast` at each iteration, state whether the nudge line fires, and give both resulting circles.
9. Repeat for a circle of 2 nodes, and for 1 node. What does the function produce, and is it correct?

C. Code tasks

1. Implement a full `DoublyLinkedList` class with cached `head`, `tail` and `size`. Write a `checkInvariants()` that verifies both directions agree and that `size` matches a forward count *and* a backward count. Call it after every mutation in a debug build.
2. Rebuild the same class using a **sentinel** node instead of null-terminated ends. Count how many `if` statements disappear.
3. Implement an LRU cache with capacity C using a DLL plus `unordered_map<int, DNode*>`. Verify `get` and `put` are both $O(1)$, and confirm the map entry is erased before the node is deleted. (This is Topic 77 — attempt it now; you have every piece.)
4. Implement `josephusSim` and `josephus`. Verify they agree for all $n \in [1, 200]$ and $k \in [1, 10]$, then time both at $n = 100\,000$, $k = 7$.
5. Implement `splitCircular` and write a test that counts the nodes in each resulting circle. Then delete the even-count nudge line and confirm your test — and only your test — catches it.
6. Write `toCircular` and `toLinear` converters for an SLL, plus an `isCircular` check that uses $O(1)$ space.

ANSWER KEY — Topics 34-45

A. Conceptual

1. For every node `x`: `x->next->prev == x` and `x->prev->next == x` (where non-null). A bug that sets `p->next = n` but forgets `p->next->prev = n` leaves the list perfectly traversable *forwards*. Every forward test passes; the corruption appears only when something walks backwards or deletes an interior node — possibly weeks later, in unrelated code.
2. SLL node 16 bytes, DLL node 24 — but glibc's minimum chunk is 32 and chunks round to a multiple of 16, so **both cost 32 bytes**. It becomes true at a payload that straddles a boundary: a 24-byte payload gives SLL 32 (chunk 32) versus DLL 40 (chunk 48) — a real 50% increase.
3. Because steps 1 and 3 read `p->next` to find the successor. Writing it first destroys that reference: `n->next` then reads back `n` itself, giving a self-loop, and every node after `p` is **orphaned and leaked**.
4. The SLL lacks the **predecessor**, and cannot recover it without a walk from the head. An **LRU cache** is impossible to build efficiently without it: the hash map hands you a node pointer and you must unlink that node in $O(1)$.
5. Because the swap already happened — this node's `prev` field now holds what used to be its `next`, i.e. the onward direction. Using `curr->next` reads the *old prev*, sending you back to the node you just came from: an infinite two-node oscillation.
6. Keep a `bool reversed` in the list object and have iterators follow `next` or `prev` according to it — reversal becomes one boolean flip, $O(1)$. This is exactly `std::reverse_iterator`, exposed as `rbegin()` / `rend()`.
7. From `last` the head is `last->next`, reachable in $O(1)$; from `head` the last node needs an $O(n)$ walk. So insert-at-end goes from **$O(n)$ to $O(1)$** , and delete-at-end from **$O(n)$ to $O(n)$ but with a correct handle** — the decisive change is insertion at the back.
8. Because the stop condition ("back at the start") is true before the first iteration. The `while` version **prints nothing at all** and returns immediately, which reads as an empty list rather than as a bug.
9. With a sentinel, the empty list, the head and the tail are all "a node with two valid neighbours," so no operation needs a null check or a head/tail special case. `end()` is an iterator *at the sentinel*, which is a real node — so `--end()` follows its `prev` to the last element, legally.
10. You cannot detect the difference by inspection, and printing either one hangs the terminal. The resolution is Floyd's cycle detection (Topic 50) for an $O(1)$ -space answer, or a bounded traversal (`i < 1000`) as a quick diagnostic.

B. Pointer tracing

1. `[10]`: first `else` fires (`head = [20]`), first branch of the second pair bridges backward (`[20]->prev = nullptr`); tail unchanged. `[40]`: first branch bridges (`[30]->next = nullptr`), second `else` fires (`tail = [30]`). One-node list: **both** `else` branches fire, leaving `head == tail == nullptr`.
2. `[10]`: `prev=[20]`, `next=null`. `[20]`: `prev=[30]`, `next=[10]`. `[30]`: `prev=[40]`, `next=[20]`. `[40]`: `prev=null`, `next=[30]`. Advance path `10→20→30→40→null`. Finally `head=[40]`, `tail=[10]`.
3. `p->next = n` first, so step 1 sets `n->next = n` — a self-loop — and step 3 sets `n->prev = n` as well. **Every node after `p` is orphaned** (all of them, however many there were), and traversal from `p` spins on `n` forever.

4. `insertFront`: `n=[5]`, `n->next = last->next = [1]`, `last->next = [5]` — the circle is now `4→5→1→2→3→4` with `last` still `[4]`, so `[5]` is the *head*. Then `last = last->next = [5]`, making `[5]` the tail and `[1]` the head again: `1→2→3→4→5→1`. ✓
5. `prev` walks to `[3]`, `curr` reaches `[4]` which equals `last`, so `if (curr == last) last = prev;` fires, moving the handle to `[3]`. Removing that line leaves `last` pointing at freed memory — a dangling pointer, and the next `insertEnd` writes through it.
6. `r`: `i=2 → (0+3)%2 = 1`; `i=3 → (1+3)%3 = 1`; `i=4 → (1+3)%4 = 0`; `i=5 → (0+3)%5 = 3`; `i=6 → (3+3)%6 = 0`; `i=7 → (0+3)%7 = 3`. Answer **3 + 1 = 4**. Elimination order for `n=7,k=3`: 3, 6, 2, 7, 5, 1 — survivor **4**. ✓
7. `r`: `0 → 0 → 0 → 0 → 0 → 0 → 0 → 0` (every step is `(r+2) % i` with `r` staying 0 only at `i=2`; compute carefully: `i=2→0`, `i=3→2`, `i=4→0`, `i=5→2`, `i=6→4`, `i=7→6`, `i=8→0`). Answer **0 + 1 = 1**. Closed form: `8 = 23 + 0`, so `l = 0` and survivor = `2(0) + 1 = 1`. ✓ (For any power of two, person 1 survives.)
8. `n=5`: start `slow=[1]`, `fast=[1]`. Iter 1: `fast->next=[2]≠head` and `fast->next->next=[3]≠head` → `slow=[2]`, `fast=[3]`. Iter 2: `[4]≠head`, `[5]≠head` → `slow=[3]`, `fast=[5]`. Now `fast->next == head`, loop exits; the nudge does **not** fire. `first=[1]`, `second=[4]`; `[5]->next=[4]`, `[3]->next=[1]` → circles **(1 2 3)** and **(4 5)**.
9. `n=2`: the loop body never runs (`fast->next->next == head`), the nudge fires moving `fast` to `[2]`; `first=[1]`, `second=[2]`, giving two one-node circles. Correct. `n=1`: `fast->next == head` so the loop is skipped and the nudge does not fire; `first=[1]` and `second=slow->next=[1]` — **both halves are the same node**. This is the degenerate case worth guarding explicitly.

C. Code tasks — what to verify

- **C1**. `checkInvariants()` must count in both directions; a forward-only count passes even when the `prev` chain is broken. That asymmetry is the whole lesson of question A1.
- **C2**. Typically **8-12** `if` statements vanish: two per insertion variant and two per deletion variant, plus the empty-list guards. The sentinel version is usually 30-40% shorter.
- **C3**. On eviction, erase the map entry keyed by the *victim node's* key *before delete* — reading `victim->key` after the delete is the classic use-after-free here. A sentinel-based DLL makes the move-to-front logic branch-free.
- **C4**. They must agree on every pair. At `n = 100 000`, `k = 7` expect the simulation to take on the order of a second ($O(n \cdot k)$ with allocator traffic) and the recurrence to be microseconds — roughly a 10^5 - $10^6 \times$ gap.
- **C5**. Without the nudge, an even split yields sizes $(n/2, n/2 - 1)$ with one node lost from the second circle. Both results are valid circles, so nothing crashes: only a size assertion catches it.
- **C6**. `isCircular`: walk with `fast` two steps and `slow` one; if they meet, a cycle exists. To confirm it is circular *from the head* rather than merely cyclic, check that the meeting point leads back to `head`.

HANDNOTE SUMMARY CARD

DOUBLY LINKED LIST

```
struct DNode { int data; DNode* prev; DNode* next; };
sizeof 24 · real heap chunk 32 · SAME AS AN SLL NODE for an int payload
```

INVARIANTS (assert these in debug builds)

```
x->next->prev == x      x->prev->next == x
head->prev == null     tail->next == null
```

INSERT AFTER p – 4 writes, and the ORDER is fixed

```
n->next = p->next;
n->prev = p;
if (p->next) p->next->prev = n; else tail = n;
p->next = n;                               <- LAST. it destroys what 1 and 3 read.
```

UNLINK d – O(1) FROM THE NODE ALONE. the headline feature.

```
if (d->prev) d->prev->next = d->next; else head = d->next;
if (d->next) d->next->prev = d->prev; else tail = d->prev;
delete d;
-> middle / head / tail / only-node ALL covered by the two if-elses
```

REVERSE

```
swap(curr->prev, curr->next); curr = curr->prev; // prev is the OLD next
then swap(head, tail).
O(1) alternative: keep a `reversed` flag. == std::reverse_iterator.
```

SLL vs DLL

backward traverse	O(n)+O(n) stack	->	O(n), O(1) space
insert BEFORE node	O(n)	->	O(1)
delete known node	O(n)	->	O(1)
delete tail	O(n)	->	O(1)
memory	same 32 B chunk.	cost is INVARIANT SURFACE, not RAM.	

CIRCULAR LINKED LIST

CSLL last->next == head. NO NULL ANYWHERE.

HOLD THE `last` POINTER, NOT `head`:

```
head = last->next      O(1)
insertFront: n->next = last->next; last->next = n;
insertEnd : insertFront(...); last = last->next; // just relabel!
one-node circle: n->next = n
```

TRAVERSAL – MUST BE do-while (stop condition is TRUE at the start)

```
Node* c = last->next;
do { visit(c); c = c->next; } while (c != last->next);
while(...) version visits NOTHING. cache the stop marker if you mutate.
```

DELETION TRAPS

```
single-node circle -> last = nullptr (else dangling handle)
deleting `last`    -> last = prev BEFORE the delete
deleting the head  -> no special case: its predecessor IS `last`
```

CDLL + SENTINEL = how std::list is actually built

```
sentinel.next = sentinel.prev = &sentinel (empty list)
zero null checks, zero boundary branches, end() is a real node,
--end() is the last element. NEVER free the sentinel.
```

JOSEPHUS

```
sim on a circle      O(n*k) time, O(n) space
J(1,k)=0; J(n,k)=(J(n-1,k)+k)%n      O(n) time, O(1) space
int r=0; for(i=2..n) r=(r+k)%i; return r+1;
k=2 closed form: n = 2^m + l -> survivor = 2l+1
```

SPLIT A CIRCLE – slow/fast midpoint, stop on `head` not `nullptr`

```
while (fast->next != head && fast->next->next != head) { slow++, fast+=2 }
if (fast->next->next == head) fast = fast->next; // EVEN count nudge
fast->next = slow->next; slow->next = head; // close BOTH circles
```

CONSOLIDATED CHEATSHEET — PART II

REVERSAL — ONE LOOP, FOUR WRAPPERS

```
core      nxt = curr->next; curr->next = prev; prev = curr; curr = nxt;
          return prev.          SAVE before you FLIP.
```

```
29 full list      0(n) / 0(1)    old head becomes the TAIL — fix cached tail
30 recursive      0(n) / 0(n)    head->next->next = head; head->next = null;
31 groups of k    0(n) / 0(n/k)  check k exist; head->next = recurse(curr,k)
32 sublist m..n   0(n) / 0(1)    dummy + pull-to-front, n-m pulls
33 alternate k    0(n) / 0(n/2k) skip k-1 after the reversal, NOT k
```

STRUCTURE SELECTOR

```
SLL      front-only work: stacks, adjacency lists, hash chains, free lists
DLL      interior deletion, bidirectional cursors: LRU, undo, history, sched
CSLL     round-robin, ring buffers, turn order, Josephus
CDLL     std::list, kernel list_head — sentinel kills every special case
```

COMPLEXITY — SIDE BY SIDE

	SLL	DLL	CSLL(last*)	CDLL+sentinel
insert front	0(1)	0(1)	0(1)	0(1)
insert back	0(1)*	0(1)	0(1)	0(1)
insert before a node	0(n)	0(1)	0(n)	0(1)
delete a KNOWN node	0(n)	0(1)	0(n)	0(1)
delete back	0(n)	0(1)	0(n)	0(1)
traverse backward	0(n) stack	0(n)/0(1)	0(n)	0(n)/0(1)
reverse	0(n)	0(n) or 0(1) flag		
node bytes (int)	16/32heap	24/32heap	16/32heap	24/32heap

* with a cached tail pointer

THE FIVE BUGS OF THIS VOLUME

1 flipping curr->next before saving nxt	remainder leaked
2 p->next = n written FIRST in DLL insert	self-loop, tail orphaned
3 updating next but not prev	passes every forward test
4 while() instead of do-while on a circle	visits nothing, looks empty
5 not moving `last` when deleting the last	dangling handle, later crash

ALWAYS-TEST MATRIX

```
[] · [x] · two nodes · head · tail · only node
k=1 · k=n · k>n · m==n · m==1 · odd n · even n · single-node circle
```

End of Volume II — Topics 29-45 of 88. Next: Section 7, Two-Pointer Patterns (46-55), which formalises the slow/fast technique you have already used twice, in Topic 45 and in the palindrome exercise. Solve 12-15 problems on reversal and doubly linked lists before starting it — Floyd's cycle detection and the reorder problems assume both are automatic.

PART III

Algorithms

Two-Pointer Patterns · Sorting & Merging — Topics 46-62

SECTION 7 — TWO-POINTER PATTERNS

46. Slow and Fast Pointers

A linked list gives you no length, no index, and no way back. The two-pointer family exists to recover all three — in **one pass**, with **$O(1)$ memory**, by running two cursors over the same list at different speeds or with a fixed gap between them.

The two variants, and what each one computes

Variant	Setup	What it gives you
Different speed	<code>slow += 1, fast += 2</code>	The midpoint (47); cycle detection (50–53); the split for merge sort (57)
Fixed gap	<code>fast</code> starts n nodes ahead, both move by 1	The n -th node from the end (48); single-pass deletion from the end (49)

THE IDEA IN ONE SENTENCE

If `fast` moves at twice the speed of `slow`, then when `fast` reaches the end, `slow` has reached the middle — the ratio of their *speeds* converts into a ratio of their *positions*, and you never had to know the length. If instead `fast` stays exactly n nodes ahead, then when `fast` falls off the end, `slow` is exactly n nodes from the end. Speed gives you fractions of the list; gap gives you offsets from the end.

The skeleton — every problem in this section is a variation of it

```
Node* slow = head;
Node* fast = head;

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;          // 1 step
    fast = fast->next->next;     // 2 steps
    // (cycle detection adds: if (slow == fast) ... )
}
// list length even -> fast == nullptr,      slow == second middle
// list length odd  -> fast->next == nullptr, slow == exact middle
```

THE GUARD ORDER IS LOAD-BEARING

`fast != nullptr && fast->next != nullptr` — in that order. C++ short-circuits `&&`, so the second test is only evaluated when `fast` is non-null. Swap them and an even-length list dereferences `nullptr` on the final iteration. This single condition appears in seven of the ten topics in this section; get it wrong once and you have written seven bugs.

two-pointer family — Time $O(n)$ single pass · Space $O(1)$ · no length computation, ever

47. Find Middle Node

```
Node* findMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

```
ODD length (5):  [1] [2] [3] [4] [5]
slow:            1  2  3
fast:            1  3  5    loop ends: fast->next == null
=> slow = [3], the exact middle.

EVEN length (6): [1] [2] [3] [4] [5] [6]
slow:            1  2  3  4
fast:            1  3  5  null loop ends: fast == null
=> slow = [4], the SECOND of the two middles.
```

The two conventions — and why it matters more than it looks

Initialisation	Even-length result	Use it for
<code>slow = head; fast = head;</code>	Second middle ([4] of 6)	LeetCode 876; "the middle node"
<code>slow = head; fast = head->next;</code>	First middle ([3] of 6)	Merge-sort split (57); palindrome (54)

WHY THE SPLIT VERSION MUST RETURN THE FIRST MIDDLE

Merge sort cuts the list at `slow`, giving `[head ... slow]` and `[slow->next ... end]`. On a **two-node** list, the second-middle convention puts `slow` on node 2, so the split is `[1,2]` and `[]` — the recursion never shrinks and you get **infinite recursion and a stack overflow**. Starting `fast = head->next` puts `slow` on node 1, splitting into `[1]` and `[2]`. One character of initialisation is the difference between a working sort and a crash.

Also worth knowing

Nothing here requires the length, and nothing requires a second pass. The obvious alternative — count the nodes, then walk $n/2$ — is also $O(n)$ but takes **two** traversals, which on a cache-hostile structure (Topic 10) is genuinely twice the cost, not a constant factor to wave away.

find middle – Time $O(n)$ one pass · Space $O(1)$ · pick your convention deliberately

48. Nth Node from End

The naive solution computes the length, then walks `length - n` nodes: two passes. The gap method does it in one.

```

Node* nthFromEnd(Node* head, int n) {
    if (head == nullptr || n <= 0) return nullptr;

    Node* fast = head;
    for (int i = 0; i < n; ++i) {          // open a gap of exactly n
        if (fast == nullptr) return nullptr; // list shorter than n
        fast = fast->next;
    }

    Node* slow = head;
    // move together until fast falls off
    while (fast != nullptr) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}

```

```

n = 2 on  [1] [2] [3] [4] [5]

open the gap:  slow=[1]                fast=[3]
               <-- 2 apart -->

move together: slow=[2] fast=[4]
               slow=[3] fast=[5]
               slow=[4] fast=null    <- stop

slow = [4] = the 2nd node from the end. ONE pass.

```

THE INVARIANT THAT MAKES IT WORK

The distance between **slow** and **fast** is fixed at n and never changes, because both advance by one. So the moment **fast** steps *off* the list — i.e. lands *one past* the last node — **slow** must be sitting n nodes back from that same imaginary position, which is exactly the n -th node from the end. State the invariant, not the trace; that is what an interviewer is listening for.

TWO OFF-BY-ONES LIVE HERE

- The gap loop runs **n times, not $n-1$** . Get this wrong and you return the $(n\pm 1)$ -th node — a silent wrong answer, never a crash.
- The null check *inside* the gap loop is what handles $n > \text{length}$. Without it, **`fast = fast->next`** dereferences null and the function crashes on the exact input a test suite is most likely to try.

n -th from end — Time $O(n)$ one pass · Space $O(1)$ · gap of n , then move in lockstep

49. Remove Nth from End

Same gap technique, but now you need the **predecessor** of the target — so open the gap one node wider and add a dummy node to absorb the "remove the head" case.

```

Node* removeNthFromEnd(Node* head, int n) {
    Node dummy(0, head);
    Node* fast = &dummy;
    Node* slow = &dummy;

    // note: n+1 steps, from the dummy
    for (int i = 0; i <= n; ++i) {
        // n larger than the list – reject
        if (fast == nullptr) return head;
        fast = fast->next;
    }

    while (fast != nullptr) {                // slow lands on the PREDECESSOR
        slow = slow->next;
        fast = fast->next;
    }

    Node* doomed = slow->next;
    // the unlink primitive, Topic 22
    slow->next = doomed->next;
    delete doomed;

    return dummy.next;                       // head may have changed
}

```

```

remove n=2 from [1] [2] [3] [4] [5]

dummy -> 1 -> 2 -> 3 -> 4 -> 5

gap of n+1 = 3 from dummy:  slow=dummy  fast=[3]
move together:              slow=[1]     fast=[4]
                           slow=[2]     fast=[5]
                           slow=[3]     fast=null  <- stop

slow=[3] is the PREDECESSOR of [4].  unlink [4].

result: 1 -> 2 -> 3 -> 5

```

WHY THE DUMMY IS NOT OPTIONAL HERE

Removing the *last* node from the end ($n = \text{length}$) means removing the head — which has no predecessor. Without a dummy that needs an entire separate branch. With one, the dummy is the head's predecessor, *slow* stops on it naturally, and *dummy.next* carries the new head out. This is the cleanest illustration in the whole book of why Topic 74 exists.

remove n-th from end – Time $O(n)$ one pass · Space $O(1)$ · gap $n+1$, dummy required

50. Cycle Detection (Floyd)

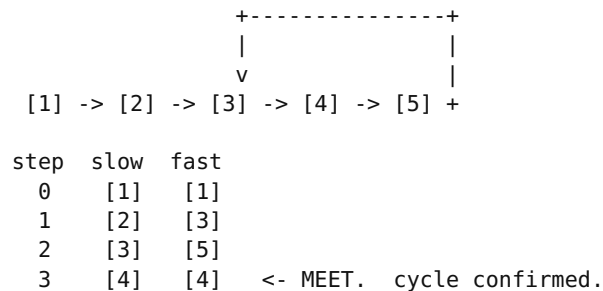
A cycle means some node's *next* points back to an earlier node. Traversal then never terminates: the loop hangs rather than crashing, which is why this is diagnosed rather than observed.

```

bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;    // they met – a cycle exists
    }
    // fast reached the end – no cycle
    return false;
}

```



WHY THEY ARE GUARANTEED TO MEET

Once both pointers are inside the cycle, consider the gap from **slow** forward to **fast** around the loop. Each step, **fast** gains exactly one position on **slow**, so that gap shrinks by exactly 1 per step. A quantity that decreases by 1 each step and lives in the range $[0, C)$ must hit 0 — they cannot "jump over" each other, because the gap changes by one, never two. Meeting is therefore certain within C steps of both being in the loop.

This is also why the speeds must differ by exactly $1 \times (1 \text{ and } 2)$. With speeds 1 and 3 the gap shrinks by 2 per step and can skip past 0 on a cycle of even length — the pointers can miss each other indefinitely.

The alternatives, and why Floyd wins

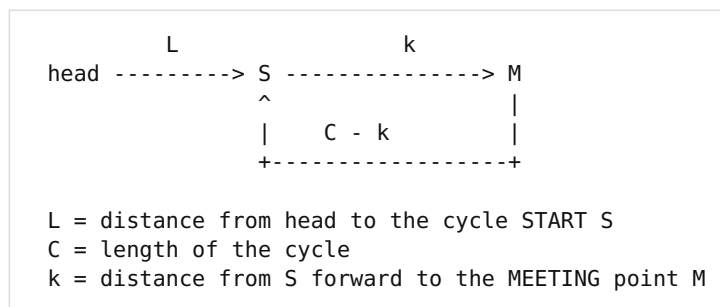
Method	Time	Space	Notes
Hash set of visited nodes	$O(n)$	$O(n)$	Easier to explain; fine if memory is free
Mark nodes as visited	$O(n)$	$O(1)$	Mutates the list — usually disallowed
Floyd's tortoise and hare	$O(n)$	$O(1)$	Non-destructive, and extends to Topics 51-53

cycle detection – Time $O(n)$ · Space $O(1)$ · meets within C steps of both entering the loop

51. Find Cycle Start Node

Detection tells you a cycle exists. This tells you *where it begins* — the node that two different predecessors point at. The result looks like magic and is four lines of algebra.

The proof — derive it, do not memorise it



When the two pointers meet:

- **slow** has travelled **L + k**.
- **fast** has travelled **L + k + nC** for some integer $n \geq 1$ — the same position, after n extra full laps.
- **fast** travelled exactly twice as far: **$2(L + k) = L + k + nC$** .

$$\begin{aligned} 2(L + k) &= L + k + nC \\ L + k &= nC \\ L &= nC - k \\ L &= (n-1)C + (C - k) \end{aligned}$$

READ THE LAST LINE OUT LOUD

*The distance from the head to the cycle start equals the distance from the meeting point onward to the cycle start, plus some whole number of extra laps. Laps do not change where you land. So: put one pointer at the head, leave the other at the meeting point, advance **both one step at a time** — they arrive at the cycle start together.*

```
Node* findCycleStart(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != nullptr && fast->next != nullptr) {    // phase 1: detect
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {                                // phase 2: locate
            slow = head;
            // BOTH move by ONE now
            while (slow != fast) {
                slow = slow->next;
                fast = fast->next;
            }
            // == the cycle start
            return slow;
        }
    }
    return nullptr;                                        // no cycle
}
```

PHASE 2 IS ONE STEP AT A TIME — BOTH POINTERS

The most common error is leaving **fast** on two steps in the second loop. The algebra says the two *distances* are equal, so the two pointers must travel at the *same* speed to cover them in the same number of moves. Keep **fast** at double speed and it laps past the start and the loop may never terminate.

Special case worth checking by hand: when the entire list is one big cycle, $L = 0$ and the meeting point satisfies $k = 0$ too, so phase 2 exits immediately with `slow == fast == head`. Correct, and it needs no special-casing.

find cycle start – Time $O(n)$ · Space $O(1)$ · phase 1 detects, phase 2 walks both at speed 1

52. Find Cycle Length

Once the pointers have met, both are inside the cycle. Freeze one and walk the other around until it comes back — the number of steps is the cycle length.

```
int cycleLength(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        // meeting point is inside the cycle
        if (slow == fast) {
            int len = 1;
            Node* c = fast->next;
            while (c != fast) { ++len; c = c->next; }
            return len;
        }
    }
    return 0; // no cycle
}
```

WHY THE MEETING POINT IS A VALID STARTING LINE

You do not need the cycle *start* to measure the cycle. Any node inside the loop works, because a cycle has no beginning from the inside — walking forward from any of its nodes returns to that same node after exactly C steps. The meeting point is guaranteed to be inside, so it is the cheapest starting line available.

Putting the three together

A full diagnosis of a suspected-cyclic list, all $O(1)$ space:

<code>hasCycle()</code>	-> yes / no	(Topic 50)
<code>findCycleStart()</code>	-> the node S where it closes	(Topic 51)
<code>cycleLength()</code>	-> C	(Topic 52)
tail length L	-> walk head to S , counting	= the acyclic prefix
total nodes	-> $L + C$	

cycle length – Time $O(n)$ · Space $O(1)$ · any node inside the loop is a valid starting line

53. Remove Cycle

Breaking a cycle means finding the **last node of the loop** — the one pointing back at the start — and setting its `next` to `nullptr`.

```

void removeCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    bool found = false;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) { found = true; break; }
    }
    if (!found) return;

    slow = head;
    // L == 0: the whole list is the loop
    if (slow == fast) {
        while (fast->next != slow) fast = fast->next;
    } else {
        // stop one node BEFORE the start
        while (slow->next != fast->next) {
            slow = slow->next;
            fast = fast->next;
        }
    }
    fast->next = nullptr;          // cut it
}

```

THE L = 0 CASE NEEDS ITS OWN BRANCH

If the cycle starts at the head, phase 2 would terminate immediately with both pointers on the head — and you would then have no handle on the *last* node of the loop. That branch walks *fast* all the way round until *fast->next == head*, which is the node that must be cut. Nearly every published solution to this problem omits it, and nearly every one is wrong on a fully circular list.

THE TRICK IN THE MAIN BRANCH

Topic 51 advances until *slow == fast* (landing *on* the start). Here the condition is *slow->next != fast->next*, which stops one node earlier — leaving *fast* on the loop's *last* node, exactly the pointer that needs rewriting. Same walk, shifted by one, to get the predecessor instead of the node. That is the "look one ahead" pattern from Topic 19 again.

After cutting, verify: *hasCycle(head)* must now return false, and the node count must equal *L + C* from Topic 52. In production, a cycle is a symptom — something relinked a node incorrectly — so find the writer, not just the loop.

remove cycle – Time $O(n)$ · Space $O(1)$ · cut at the loop's LAST node, mind the $L = 0$ case

54. Palindrome Linked List

A singly linked list cannot be read backwards — so you make half of it readable backwards. Three phases: find the middle, reverse the second half, compare.

```

bool isPalindrome(Node* head) {
    if (head == nullptr || head->next == nullptr) return true;

    // 1. FIRST middle (fast = head->next), so the halves split cleanly
    Node* slow = head;
    Node* fast = head->next;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // 2. reverse the second half (Topic 29)
    Node* second = slow->next;
    slow->next = nullptr;
    Node* prev = nullptr;
    while (second != nullptr) {
        Node* nxt = second->next;
        second->next = prev;
        prev = second;
        second = nxt;
    }

    // 3. compare the two halves
    bool ok = true;
    Node* a = head;
    Node* b = prev;
    Node* secondHead = prev;
    while (b != nullptr) {
        if (a->data != b->data) { ok = false; break; }
        a = a->next;
        b = b->next;
    }

    // 4. RESTORE the list before returning
    Node* p = nullptr;
    while (secondHead != nullptr) {
        Node* nxt = secondHead->next;
        secondHead->next = p;
        p = secondHead;
        secondHead = nxt;
    }
    slow->next = p;

    return ok;
}

```

1 -> 2 -> 3 -> 2 -> 1

split at the FIRST middle: [1 2 3] and [2 1]
reverse the second half: [1 2 3] and [1 2]
compare pairwise: 1==1, 2==2 -> palindrome
(the shorter half controls the loop, so the odd middle is skipped)

THE STEP EVERYONE SKIPS

Phase 4. The function was asked a *question*, so it must not leave the caller's list mutated. Reversing the second half back and re-attaching it is what separates a correct implementation from one that returns the right answer and quietly destroys the input. Interviewers watch for this specifically; if you are short on time, at minimum say out loud "and I would restore the list here."

WHAT NOT TO DO

Copying the values into a **vector** and using two indices is $O(n)$ space and passes every test — but it sidesteps the entire point of the exercise. Similarly, recursing to the end and comparing on the way back up (Topic 15's trick) is elegant but $O(n)$ stack. The reverse-half method is the only one that is $O(n)$ time *and* $O(1)$ space.

palindrome – Time $O(n)$ · Space $O(1)$ · find middle → reverse half → compare → RESTORE

55. Intersection of Two Lists

Two lists *intersect* when they share actual nodes — the same addresses, not merely equal values. From the intersection point on, they are one list: a Y shape, never an X.

```
A: [4] -> [1] --+
                   v
                   [8] -> [4] -> [5] -> null      intersection node = [8]
                   ^
B: [5] -> [6] -> +

Once they merge they can never diverge – a node has only ONE next pointer.
```

Method 1 — align by length difference

```
Node* getIntersection(Node* a, Node* b) {
    int la = 0, lb = 0;
    for (Node* p = a; p; p = p->next) ++la;
    for (Node* p = b; p; p = p->next) ++lb;

    // trim the longer list's head start
    while (la > lb) { a = a->next; --la; }
    while (lb > la) { b = b->next; --lb; }

    // now aligned – walk in lockstep
    while (a != b) { a = a->next; b = b->next; }
    // nullptr if they never meet
    return a;
}
```

Method 2 — the pointer switch (no length computation)

```
Node* getIntersection(Node* headA, Node* headB) {
    if (!headA || !headB) return nullptr;
    Node* a = headA;
    Node* b = headB;
    while (a != b) {
        // at the end, jump to the OTHER list
        a = (a == nullptr) ? headB : a->next;
        b = (b == nullptr) ? headA : b->next;
    }
    return a; // the node, or nullptr
}
```

WHY THE SWITCH WORKS — THE ONE-LINE PROOF

Let the exclusive prefixes have lengths p and q , and the shared tail length c . Pointer **a** travels $p + c + q$ to reach the intersection on its second list; pointer **b** travels $q + c + p$. **Those are the same number.** Both pointers arrive at the intersection node simultaneously, after at most $m + n$ steps. If there is no intersection, both become `nullptr` at the same moment and the loop exits with the correct answer — `nullptr` — for free.

COMPARE POINTERS, NOT VALUES

`while (a != b)` compares *addresses*. Writing `a->data != b->data` finds the first position where the values happen to match, which is a different — and almost always wrong — question. Two lists holding `[1,2,3]` and `[1,2,3]` as separate nodes do **not** intersect.

The switch method is $O(1)$ space and one pass in spirit, but note it does traverse up to $m + n$ nodes — the same total work as the length method, in less code. Prefer it for elegance, not for speed.

intersection — Time $O(m + n)$ · Space $O(1)$ · compare addresses; switch lists at the end

PRACTICE SET — Topics 46-55

A. Conceptual (answer in your own words)

1. Why must the loop guard be `fast != nullptr && fast->next != nullptr` in that order? Give the exact input that breaks the reversed order.
2. Give the two initialisations of `fast` for finding the middle, state which middle each returns on an even-length list, and name the algorithm that *requires* the first-middle version and would otherwise infinitely recurse.
3. State the invariant that makes the gap method correct, in one sentence, without tracing an example.
4. Why does `removeNthFromEnd` need a dummy node while `nthFromEnd` does not?
5. Prove that Floyd's pointers must meet once both are inside the cycle. Then explain why speeds of 1 and 3 break the proof.
6. Derive $L = nC - k$ from scratch, defining every symbol. Then state in plain English what it licenses you to do.
7. In phase 2 of `findCycleStart`, why do *both* pointers move one step at a time? What happens if `fast` keeps its double speed?
8. Why can the cycle length be measured from the meeting point, without first finding the cycle start?
9. Why does `isPalindrome` have a fourth phase, and what breaks in the caller if you omit it?
10. Explain why two lists containing `[1,2,3]` and `[1,2,3]` may not intersect, and give the one-line proof that the pointer-switch method terminates correctly when they do not.

B. Tracing (pen and paper)

1. Trace `findMiddle` on lists of length 1, 2, 3 and 6, for *both* initialisations. Tabulate all eight results.
2. Trace `nthFromEnd([1,2,3,4,5,6], 3)`. Give `slow` and `fast` at every step and the returned node.
3. Trace `removeNthFromEnd([1,2], 2)`. Give `slow` when the second loop exits, and state precisely which line the dummy node saves you from writing.
4. A list of 8 nodes has a cycle starting at node 3 (so $L = 2$, $C = 6$). Simulate Floyd's phase 1 and give the meeting node's index. Then verify $L = nC - k$ with your numbers, stating n .
5. Same list. Run phase 2 by hand and confirm both pointers arrive at node 3.
6. A list is *entirely* a cycle ($L = 0$, $C = 5$). Give the meeting node, the result of phase 2, and explain what `removeCycle` does differently here.
7. Trace `isPalindrome([1,2,2,1])`. Give the two halves after the split, after the reversal, and the sequence of comparisons. Repeat for `[1,2,3,2,1]`.
8. Lists $A = [4,1,8,4,5]$ and $B = [5,6,1,8,4,5]$ intersect at the node holding 8. Trace the pointer switch: give the step number at which `a` and `b` first become equal, and confirm it equals $p + c + q$.
9. Two non-intersecting lists of lengths 3 and 5. After how many steps does the pointer-switch loop terminate, and what are `a` and `b` at that moment?

C. Code tasks

1. Implement all of Topics 47-53 as free functions over a shared `Node`. Write a harness that builds a list with a cycle at a chosen index and verifies `hasCycle`, `findCycleStart`, `cycleLength` and `removeCycle` against the known answer, for every $L \in [0, 10]$ and $C \in [1, 10]$.

2. Implement `isPalindrome` and assert with a saved copy of the original values that the list is byte-identical after the call. Then delete phase 4 and watch the assertion fire.
3. Implement both intersection methods. Test with: shared tail, no intersection, one list empty, intersection at the head of both, and two lists with equal values but no shared nodes.
4. Write `removeCycle` without the $L = 0$ branch, then test it on a fully circular list. Describe the failure precisely.
5. Benchmark `findMiddle` (one pass) against count-then-walk (two passes) on 10 000 000 nodes. Report both times and explain the ratio using Topic 10.

ANSWER KEY — Topics 46-55

A. Conceptual

1. `&&` short-circuits, so `fast->next` is only evaluated once `fast` is known non-null. Reversed, any **even-length** list crashes: on the final iteration `fast` is `nullptr` and `fast->next` is a null dereference. Length 2 is the smallest failing case.
2. `fast = head` → the **second** middle; `fast = head->next` → the **first** middle. **Merge sort** (Topic 57) requires the first-middle version: with the other, a two-node list splits into `[1,2]` and `[]`, the sub-problem never shrinks, and the recursion overflows the stack.
3. The distance between the pointers is fixed at n and both advance by one, so when `fast` reaches one-past-the-end, `slow` is n nodes back from one-past-the-end — which is the n -th node from the end.
4. Because deletion needs the target's **predecessor**, and when n equals the list length the target is the head, which has none. The dummy supplies one, so no separate branch is needed and `dummy.next` carries out the possibly-changed head. `nthFromEnd` only *reads*, so it never needs a predecessor.
5. Once both are in the cycle, the forward gap from `slow` to `fast` shrinks by exactly 1 per step (fast gains 2, slow gains 1). A non-negative integer decreasing by exactly 1 must reach 0, so they meet within C steps. With speeds 1 and 3 the gap shrinks by 2 per step, so on an even-length cycle it can alternate between odd values and skip 0 forever.
6. L = head to cycle start; C = cycle length; k = cycle start to meeting point. At the meeting, slow has gone $L+k$ and fast $L+k+nC$; fast went twice as far, so $2(L+k) = L+k+nC$, giving $L = nC - k = (n-1)C + (C-k)$. Plain English: **the walk from the head to the cycle start is the same length as the walk from the meeting point onward to the cycle start** (plus whole laps, which change nothing). So advance one pointer from each, at equal speed, and they meet there.
7. Because the algebra equates two *distances*. Equal distances are covered in equal step counts only at equal speeds. At double speed `fast` overshoots the start, laps the cycle, and the equality `slow == fast` may never hold at the right node — potentially an infinite loop.
8. Because a cycle has no distinguished beginning when viewed from inside: from *any* node in the loop, walking forward returns to that same node after exactly C steps. The meeting point is guaranteed to be in the loop, so it is a free starting line.
9. Because the function mutates the caller's list to answer a read-only question. Omitting phase 4 leaves the second half reversed and detached — the caller's list is silently truncated at the middle, and every later traversal, length or delete operates on half the data.
10. Intersection is defined by **shared nodes** (identical addresses), not equal values; two independently allocated lists with the same contents share nothing. Termination proof: if there is no shared node, `a` traverses m then n , and `b` traverses n then m — after $m+n+1$ steps both are `nullptr` simultaneously, so `a == b` holds and the loop exits returning `nullptr`.

B. Tracing

1. With `fast = head`: len 1 → [1]; len 2 → [2]; len 3 → [2]; len 6 → [4]. With `fast = head->next`: len 1 → [1]; len 2 → [1]; len 3 → [2]; len 6 → [3]. The two conventions differ only on even lengths.
2. Gap: fast [1]→[2]→[3]→[4]. Then together: slow [1]→[2], fast [4]→[5]; slow→[3], fast→[6]; slow→[4], fast→null. Returns **[4]**, the 3rd from the end. ✓

3. Gap of $n+1 = 3$ from the dummy: `fast = dummy→[1]→[2]→null`. Second loop never runs, so `slow` is still **the dummy**. It unlinks `dummy.next = [1]` — the head. The dummy saves you from writing `if (n == length) { head = head->next; ... }` as a separate branch.
4. Nodes indexed 1..8, cycle start at 3, so the cycle is {3,4,5,6,7,8} and node 8 → node 3. `slow/fast`: (1,1)→(2,3)→(3,5)→(4,7)→(5,3)→(6,5)→(7,7) — **meet at node 7**. So $k = 4$ (3→4→5→6→7). Check: $L = 2, C = 6, nC - k = n \cdot 6 - 4 = 2$ when $n = 1$. ✓
5. `slow = head = node 1, fast = node 7`. Step: (2, 8); step: (3, 3) — both at **node 3**, the cycle start. ✓ Two steps, matching $L = 2$.
6. $L = 0, C = 5$, nodes 1..5 with 5→1. `slow/fast`: (1,1)→(2,3)→(3,5)→(4,2)→(5,4)→(1,1) — **meet at node 1**, the head. Phase 2 then exits immediately (`slow == fast == head`), so `removeCycle` has no predecessor handle and must take its special branch: walk `fast` until `fast->next == head`, landing on node 5, and cut there.
7. [1,2,2,1]: first middle → `slow = [2]` (index 2), halves [1,2] and [2,1]; reversed second → [1,2]; compare `1==1, 2==2` → **true**. [1,2,3,2,1]: `slow = [3]`? No — first-middle on length 5 gives `slow` at index 3 (the node holding 3), halves [1,2,3] and [2,1]; reversed → [1,2]; loop runs while `b` is non-null, so it compares `1==1, 2==2` and stops — the middle 3 is never compared, which is exactly right. **true**.
8. $p = 2$ (4,1), $q = 3$ (5,6,1), $c = 3$ (8,4,5). Both pointers reach the intersection after $p+c+q = 5$ steps from their switch-adjusted start — `a` covers 2+3+3 and `b` covers 3+3+2, both = 8 nodes traversed, meeting at [8]. ✓
9. Both become `nullptr` after $m+n+1 = 9$ steps, at which point `a == b == nullptr`, the loop condition fails, and `nullptr` is returned.

C. Code tasks — what to verify

- **C1.** The $L = 0$ and $C = 1$ (self-loop) combinations are where implementations break. If your harness passes all 110 combinations, the code is genuinely correct.
- **C2.** Without phase 4 the assertion fires on the very first non-trivial input: the list is truncated at the middle and the second half is reversed and orphaned.
- **C3.** The "equal values, no shared nodes" case is the one that catches a `a->data != b->data` comparison. It must return `nullptr`.
- **C4.** On a fully circular list, phase 2 exits immediately with both pointers on the head, so `fast->next = nullptr` cuts the link *out of the head* — destroying the list down to a single node instead of removing the cycle.
- **C5.** Expect the two-pass version to take close to **2×** the one-pass time, not a negligible amount more: each pass is a full set of non-prefetchable cache misses (Topic 10), so the second traversal costs as much as the first.

HANDNOTE SUMMARY CARD

THE SKELETON – SEVEN OF THESE TEN TOPICS ARE THIS LOOP

```
while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
```

GUARD ORDER IS LOAD-BEARING. fast first, then fast->next.
reversed => every EVEN-length list null-derefs.

SPEED gives you FRACTIONS of the list. GAP gives you OFFSETS from the end.

FIND MIDDLE – TWO CONVENTIONS, PICK DELIBERATELY

fast = head -> SECOND middle on even (LeetCode 876)
fast = head->next -> FIRST middle on even (merge sort, palindrome)
wrong one in merge sort => [1,2] splits to [1,2]+[] => INFINITE RECURSION

GAP METHOD

nth from end open gap of n, then lockstep. slow = answer.
remove nth open gap of n+1 FROM A DUMMY. slow = PREDECESSOR.
invariant: gap never changes, so fast falling off => slow is n from the end

FLOYD – DETECT

meet inside the loop because the gap shrinks by EXACTLY 1 per step.
speeds 1 and 3 shrink it by 2 -> can skip 0 forever on an even cycle.

FLOYD – LOCATE THE START

L = dist head->start
2(L+k) = L+k+nC C = cycle length
L = nC - k k = dist start->meeting point
L = (n-1)C + (C-k)
=> head-to-start == meetingpoint-to-start. Move BOTH at speed ONE.

FLOYD – CYCLE LENGTH

freeze one at the meeting point, walk the other round, count. Any node
inside the loop is a valid starting line.

FLOYD – REMOVE

stop ONE EARLY: while (slow->next != fast->next) -> fast = loop's LAST node
SPECIAL CASE L == 0 (whole list is the loop): walk until fast->next == head
then fast->next = nullptr.

PALINDROME – 4 PHASES, AND PHASE 4 IS THE ONE PEOPLE SKIP

1 first middle 2 reverse 2nd half 3 compare 4 REVERSE BACK + reattach
O(n) time, O(1) space. vector copy = O(n) space, misses the point.

INTERSECTION – COMPARE ADDRESSES, NOT VALUES

switch method: a = a ? a->next : headB; b = b ? b->next : headA;
a walks p+c+q, b walks q+c+p – SAME NUMBER, so they arrive together.
no intersection => both hit nullptr at the same step => returns nullptr.
lists are Y-shaped, never X: a node has only one next pointer.

ALL OF THESE: O(n) time, O(1) space, ONE pass, no length computed.

SECTION 8 — SORTING & MERGING

56. Merge Two Sorted Lists

The foundation of the whole section: given two ascending lists, produce one ascending list by **relinking existing nodes** — no allocation, no copying.

Iterative, with a dummy node

```
Node* mergeTwo(Node* a, Node* b) {
    Node dummy(0);
    Node* tail = &dummy;

    while (a != nullptr && b != nullptr) {
        if (a->data <= b->data) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    // attach whatever remains, in one write
    tail->next = (a != nullptr) ? a : b;

    return dummy.next;
}
```

TWO IDEAS DOING ALL THE WORK

The dummy removes the "which list supplies the first node?" special case — you always append to **tail**, never assign a head.

The tail attach is a single pointer write, not a loop. When one list is exhausted, the *entire* remainder of the other is already sorted and already linked, so you splice it wholesale. Writing a second **while** loop to copy it node by node is the most common way to make this function twice as long for no benefit.

Recursive

```
Node* mergeTwoRec(Node* a, Node* b) {
    if (a == nullptr) return b;
    if (b == nullptr) return a;

    if (a->data <= b->data) { a->next = mergeTwoRec(a->next, b); return a; }
    else { b->next = mergeTwoRec(a, b->next); return b; }
}
```

Shorter, but $O(m + n)$ stack frames — it will overflow on large inputs. Use it to explain the idea; ship the iterative one.

<= IS NOT THE SAME AS <

Using **a->data <= b->data** takes from **a** when the values are equal, which preserves the relative order of equal elements — the definition of a **stable** merge. Change it to **<** and equal elements swap lists. This is invisible for plain **ints** and immediately visible when the payload is a struct sorted on one field.

merge two sorted — Time $O(m + n)$ · Space $O(1)$ iterative, $O(m + n)$ stack recursive · stable with **<=**

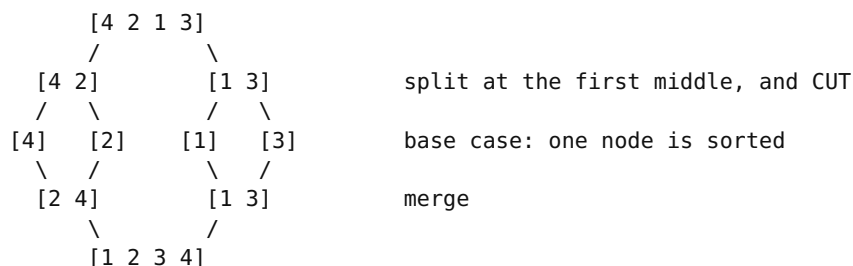
57. Merge Sort on Linked List

The right sort for a linked list, and the reason is worth stating precisely: merge sort needs only *sequential* access and can merge by relinking, so the $O(n)$ auxiliary array that makes it a poor choice for arrays **does not exist here**.

```
// returns the head of the second half
Node* split(Node* head) {
    Node* slow = head;
    Node* fast = head->next;           // FIRST middle – see Topic 47
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    Node* mid = slow->next;
    // CUT – this is what makes it terminate
    slow->next = nullptr;
    return mid;
}

Node* mergeSort(Node* head) {
    // 0 or 1 node
    if (head == nullptr || head->next == nullptr) return head;

    Node* second = split(head);
    Node* left = mergeSort(head);
    Node* right = mergeSort(second);
    return mergeTwo(left, right);      // Topic 56
}
```



split at the first middle, and CUT

base case: one node is sorted

merge

depth = $\log n$, each level touches every node once $\rightarrow O(n \log n)$

THE THREE LINES THAT MATTER MOST

fast = head->next — the first-middle convention. With **fast = head**, a two-node list splits into itself plus nothing and the recursion never terminates.

slow->next = nullptr — the cut. Without it both halves still point at the same tail and the "halves" are not independent lists.

head == nullptr || head->next == nullptr — both base cases. Dropping the null check crashes inside `split`.

Why not quicksort, and why not the array intuition

	On an array	On a linked list
Merge sort space	$O(n)$ auxiliary array	$O(\log n)$ stack only
Quicksort space	$O(\log n)$	$O(\log n)$ – but no random pivot
Random access needed	Quicksort: yes	Impossible — $O(n)$ per access
Cache behaviour	Quicksort wins	Both are cache-hostile anyway
Verdict	Quicksort usually	Merge sort, always

`std::list::sort()` is a merge sort for exactly these reasons, and it is *stable* and allocation-free. Note also that `std::sort` cannot be used on a `std::list` at all — it requires random-access iterators.

merge sort – Time $O(n \log n)$ guaranteed · Space $O(\log n)$ stack · stable · the correct default

58. Insertion Sort on Linked List

Build a new sorted list by removing nodes from the input one at a time and splicing each into its correct position (Topic 19).

```
Node* insertionSort(Node* head) {
    // head of the growing sorted list
    Node dummy(0);

    Node* curr = head;
    while (curr != nullptr) {
        // SAVE – curr->next is about to change
        Node* nxt = curr->next;

        Node* p = &dummy;                // find the insertion point
        while (p->next != nullptr && p->next->data < curr->data)
            p = p->next;

        curr->next = p->next;              // splice curr in after p
        p->next = curr;

        curr = nxt;
    }
    return dummy.next;
}
```

input 4 -> 2 -> 1 -> 3

```
take 4: sorted = 4
take 2: scan 4, 2 < 4 -> insert at front    sorted = 2 4
take 1: scan 2, 1 < 2 -> insert at front    sorted = 1 2 4
take 3: scan 1,2 then 4 > 3 -> insert here  sorted = 1 2 3 4
```

WHERE IT ACTUALLY BEATS MERGE SORT

$O(n^2)$ in general, so never the default — but on a list that is already **nearly sorted** the inner scan terminates almost immediately and the whole thing runs in close to $O(n)$ with $O(1)$ space and no recursion. For a small list (say under 30 nodes) that is genuinely faster than merge sort's recursive splitting, which is why production merge sorts often switch to insertion sort at small sizes.

SAVE `next` BEFORE SPLICING

`curr->next` is overwritten by the splice. Read the rest of the input *first* or the remainder of the list is lost — the same discipline as Topics 22, 24 and 29. If a linked-list operation writes to a `next` field, the value that field held must already be in a local variable.

insertion sort – Time $O(n^2)$, $\sim O(n)$ if nearly sorted · Space $O(1)$ · stable with `<` in the scan

59. Quick Sort on Linked List

Quicksort's array formulation depends on swapping elements by index, which a linked list cannot do. The list version partitions into **three separate lists** and concatenates.

```
Node* quickSort(Node* head) {
    if (head == nullptr || head->next == nullptr) return head;

    // simplest choice: the head's value
    int pivot = head->data;
    Node lessD(0), equalD(0), greaterD(0);    // three dummy heads
    Node *lt = &lessD, *eq = &equalD, *gt = &greaterD;

    for (Node* c = head; c != nullptr; ) {
        Node* nxt = c->next;
        c->next = nullptr;
        if (c->data < pivot) { lt->next = c; lt = c; }
        else if (c->data > pivot) { gt->next = c; gt = c; }
        else { eq->next = c; eq = c; }
        c = nxt;
    }

    Node* left = quickSort(lessD.next);
    Node* right = quickSort(greaterD.next);

    // concatenate: left ++ equal ++ right
    Node* result = concat(left, concat(equalD.next, right));
    return result;
}
```

WHY THE THREE-WAY PARTITION, NOT TWO

A two-way partition (`<` and `>=`) puts the pivot itself into one of the halves, so a list of all-equal values never shrinks and the recursion is infinite. Splitting equals into their own list guarantees each recursive call is strictly smaller, and it makes duplicate-heavy input $O(n)$ rather than $O(n^2)$. This is Dutch-national-flag partitioning, and on a linked list it is easier than on an array.

WHY THIS IS NOT THE ANSWER TO "SORT A LINKED LIST"

Choosing a good pivot requires sampling — and a linked list has no $O(1)$ random access, so median-of-three costs $O(n)$ per level. With the head as pivot, an already-sorted list degrades to $O(n^2)$, which is exactly the input you are most likely to be handed. Quicksort is also **not stable** here. Know this implementation to discuss the trade-off; use merge sort.

The `concat` helper must walk to the end of the left list to attach the right — $O(n)$ per call — unless you return a (head, tail) pair from each recursion. That refinement is worth doing if you implement this properly.

quicksort — Time $O(n \log n)$ average, $O(n^2)$ worst · Space $O(\log n)$ – $O(n)$ stack · unstable

60. Merge K Sorted Lists

Given k sorted lists with N total nodes, produce one sorted list. Three approaches, and the gap between them is large.

Approach 1 — sequential merging: $O(N \cdot k)$

```
Node* mergeKSequential(std::vector<Node*>& lists) {
    Node* result = nullptr;
    for (Node* l : lists) result = mergeTwo(result, l);
    return result;
}
```

The accumulated `result` is re-traversed on every merge, so the first list is walked k times. Correct, but quadratic in k .

Approach 2 — divide and conquer: $O(N \log k)$

```
Node* mergeKDivide(std::vector<Node*>& lists, int lo, int hi) {
    if (lo > hi) return nullptr;
    if (lo == hi) return lists[lo];
    int mid = lo + (hi - lo) / 2;
    return mergeTwo(mergeKDivide(lists, lo, mid),
                    mergeKDivide(lists, mid + 1, hi));
}
```

Approach 3 — min-heap: $O(N \log k)$

```
struct Cmp {
    bool operator()(Node* a, Node* b) const { return a->data > b->data; }
};

Node* mergeKHeap(std::vector<Node*>& lists) {
    std::priority_queue<Node*, std::vector<Node*>, Cmp> pq;    // min-heap
    for (Node* l : lists) if (l) pq.push(l);

    Node dummy(0);
    Node* tail = &dummy;
    while (!pq.empty()) {
        Node* n = pq.top(); pq.pop();
        tail->next = n;
        tail = n;
        if (n->next) pq.push(n->next);           // refill from the same list
    }
    tail->next = nullptr;
    return dummy.next;
}
```

Approach	Time	Extra space	Notes
Sequential	$O(N \cdot k)$	$O(1)$	Simple; unacceptable for large k
Divide and conquer	$O(N \log k)$	$O(\log k)$ stack	No heap needed; usually fastest in practice
Min-heap	$O(N \log k)$	$O(k)$	Streams naturally; the right answer if lists arrive lazily

THE INTERVIEW ANSWER

Say both. "Divide and conquer gives $O(N \log k)$ with no auxiliary structure, because each node is touched once per level and there are $\log k$ levels. A min-heap gives the same bound with $O(k)$ space, and I would prefer it if the lists were streams rather than fully materialised." That sentence demonstrates you understand *why* the bound is $\log k$ and not $\log N$.

merge k sorted — $O(N \log k)$ by heap or divide-and-conquer · $O(N \cdot k)$ if merged sequentially

61. Sort 0s, 1s and 2s

A list containing only the values 0, 1 and 2. Sorting it does not need a comparison sort at all.

The relinking method — one pass, nodes preserved

```
Node* sort012(Node* head) {
    Node zeroD(0), oneD(0), twoD(0);
    Node *z = &zeroD, *o = &oneD, *t = &twoD;

    for (Node* c = head; c != nullptr; c = c->next) {
        if (c->data == 0) { z->next = c; z = c; }
        else if (c->data == 1) { o->next = c; o = c; }
        else { t->next = c; t = c; }
    }

    // 0s -> 1s (or straight to 2s)
    z->next = oneD.next ? oneD.next : twoD.next;
    o->next = twoD.next; // 1s -> 2s
    t->next = nullptr; // terminate

    return zeroD.next;
}
```

THE JOIN IS WHERE THIS BREAKS

If there are no 1s, `o` is still `&oneD` and the zero-list must skip straight to the twos — hence the conditional. Test all eight presence combinations: only 0s, only 1s, only 2s, 0s+1s, 0s+2s, 1s+2s, all three, and empty. The `0s + 2s` case is the one that fails on naive code, and it fails silently by dropping every 2.

The counting method — and why it may be disallowed

```
Node* sort012Counting(Node* head) {
    int cnt[3] = {0, 0, 0};
    for (Node* c = head; c; c = c->next) ++cnt[c->data];

    Node* c = head;
    for (int v = 0; v < 3; ++v)
        // OVERWRITES data
        while (cnt[v]-- > 0) { c->data = v; c = c->next; }
    return head;
}
```

READ THE PROBLEM STATEMENT

Counting is two passes, $O(1)$ space, and simpler — but it **rewrites the payloads** rather than moving nodes. If the node carries anything besides the sort key, or if any external pointer refers to a particular node, the counting method silently corrupts it. This is the same "swap links, not data" distinction from Topic 27, and interviewers ask this problem partly to see whether you notice.

sort 0s/1s/2s — Time $O(n)$ · Space $O(1)$ · relink for correctness; count only if payloads are pure keys

62. Check if List is Sorted

```
bool isSortedAscending(Node* head) {
    if (head == nullptr) return true;           // empty is sorted
    for (Node* c = head; c->next != nullptr; c = c->next)
        // strict violation found
        if (c->data > c->next->data) return false;
    return true;
}
```

Note the loop condition is `c->next != nullptr`, not `c != nullptr` — you are comparing *pairs*, so the walk must stop on the last node (Topic 14). The empty-list guard on the first line is what makes that safe.

A generic version worth writing once

```
template <typename Cmp>
bool isSortedBy(Node* head, Cmp inOrder) {
    if (head == nullptr) return true;
    for (Node* c = head; c->next != nullptr; c = c->next)
        if (!inOrder(c->data, c->next->data)) return false;
    return true;
}

// ascending, duplicates OK
isSortedBy(head, [](int a, int b){ return a <= b; });
// strictly increasing
isSortedBy(head, [](int a, int b){ return a < b; });
isSortedBy(head, [](int a, int b){ return a >= b; }); // descending
```

USE IT AS A TEST ORACLE

This function is worth more as a *verifier* than as an answer to an interview question. After every sort you implement in this section, assert `isSortedAscending(result)` and that the node count is unchanged. Sorts that lose or duplicate nodes still produce sorted output — the ordering assertion alone will not catch a dropped node, and dropping nodes is the most common merge-sort bug.

check if sorted – Time $O(n)$, early exit on the first violation · Space $O(1)$ · empty list is sorted

PRACTICE SET — Topics 56-62

A. Conceptual (answer in your own words)

1. In `mergeTwo`, why is the leftover attached with a single pointer write rather than a second loop? What property of the remaining list makes that valid?
2. What exactly changes if `a->data <= b->data` becomes `<`? Give an input where the difference is observable and one where it is not.
3. Merge sort is a poor default for arrays but the correct default for linked lists. Give the specific resource that differs, and explain why it disappears for lists.
4. Which line of `split` makes the recursion terminate, and what is the exact failure mode if it is removed?
5. Why does merge sort on a linked list use $O(\log n)$ space and not $O(n)$, given that merge sort on an array uses $O(n)$?
6. Give two reasons quicksort is a bad choice for a linked list, and name the input that triggers its worst case with a head pivot.
7. Why does the linked-list quicksort partition three ways instead of two? Give the input that makes a two-way partition loop forever.
8. Merging k lists sequentially is $O(N \cdot k)$ but divide-and-conquer is $O(N \log k)$. Explain where the extra factor comes from in the first, in one sentence.
9. Why is the min-heap bound $O(N \log k)$ and not $O(N \log N)$?
10. When is the counting method for sorting 0s/1s/2s *incorrect* rather than merely inelegant?
11. Why is `isSortedAscending` insufficient on its own as a test for a sort implementation?

B. Tracing and analysis (pen and paper)

1. Trace `mergeTwo([1,3,5], [2,3,6])`. Give the value appended at each iteration, note which list supplies each equal value, and give the final list.
2. Draw the full recursion tree for `mergeSort([4,2,1,3])`: every `split`, every base case, every `mergeTwo`. Count the total number of node comparisons.
3. Run `split` by hand on lists of length 1, 2, 3 and 4. Give both halves in each case, and identify which length would infinitely recurse under the `fast = head` convention.
4. Trace `insertionSort([4,2,1,3])`. Give the sorted list after each node is spliced, and the number of comparisons in each inner scan. Total them and compare with $n^2/4$.
5. Trace `quickSort([3,1,3,2,3])` with the head as pivot. Give the three partitions at the top level and explain why the all-equal case would not terminate with a two-way split.
6. For $k = 8$ lists of 100 nodes each, compute the total node-visits for sequential merging and for divide-and-conquer. Give both numbers and the ratio.
7. Trace `sort012([2,0,2,1,0])`. Give the three intermediate lists before the join and the final result. Then trace `[2,0,2,0]` and state which line of the join matters.
8. An $n = 1\,000\,000$ list is already sorted. Give the approximate comparison count for merge sort and for insertion sort, and the practical consequence.

C. Code tasks

1. Implement `mergeSort` and write a verification harness that checks *three* properties after every run: the output is sorted, the node count is unchanged, and the multiset of values is unchanged. Test on random lists of length 0-200.

2. Deliberately write `split` with `fast = head`. Run it on a two-node list and record the crash, then explain the stack trace.
3. Implement `mergeSort` so it switches to `insertionSort` below a threshold of 16 nodes. Benchmark against plain merge sort on 1 000 000 random nodes and report both times.
4. Implement all three merge- k approaches. Benchmark with $k = 500$ lists of 200 nodes each and confirm the $O(N \cdot k)$ versus $O(N \log k)$ gap empirically.
5. Implement `sort012` by relinking and test all eight presence combinations from Topic 61. Then break the conditional in the join and confirm which combination fails.
6. Write a stability test: sort a list of `{key, originalIndex}` pairs by key with both merge sort and quicksort, then assert that equal keys retain ascending `originalIndex`. Confirm merge sort passes and quicksort fails.

ANSWER KEY — Topics 56-62

A. Conceptual

1. Because when one list is exhausted, everything left in the other is already sorted *and already linked together*. Appending it is one write; copying it node by node would be $O(\text{remaining})$ work for no change in the result.
2. `<=` takes from `a` on ties, preserving the relative order of equal elements — a **stable** merge. Not observable for plain `ints` (equal ints are indistinguishable); immediately observable for `{key, index}` structs sorted on `key`, where `<` reverses the order of equal keys.
3. The **$O(n)$ auxiliary array**. An array merge must write results somewhere, because overwriting the source destroys unread elements. A list merge produces its output by rewriting `next` pointers of nodes that already exist, so no scratch storage is needed at all.
4. `slow->next = nullptr;` — the cut. Without it, the "first half" still runs through to the end of the whole list, so the sub-problem is never smaller than the original: infinite recursion and a stack overflow.
5. Because the $O(n)$ in the array version is the auxiliary array, which lists do not need. What remains is the recursion depth, $\log n$ frames, so $O(\log n)$.
6. (a) A good pivot needs sampling, and sampling needs random access, which costs $O(n)$ per probe on a list. (b) It is **not stable**. The worst case is triggered by an **already-sorted list** (or reverse-sorted), where the head pivot splits off one node per level $\rightarrow O(n^2)$.
7. A two-way split ($<$ and $\geq$) leaves the pivot in one of the halves, so a list of **all-equal values** produces a sub-problem the same size as the input and never terminates. The three-way split isolates the equals, guaranteeing strict shrinkage.
8. Because the accumulated result is re-traversed on every one of the k merges, so the first list's nodes are walked k times — the merges cost roughly $N/k + 2N/k + \dots + N \approx N \cdot k/2$.
9. Because the heap holds at most **one node per list** — k entries, never N . Each of the N nodes is pushed and popped exactly once at a cost of $\log k$, giving $O(N \log k)$.
10. When the node carries anything besides the sort key, or when an external pointer refers to a specific node. Counting overwrites `data` in place, so it moves *values* and leaves every other field attached to the wrong record. If `data` is the entire payload and nothing holds node pointers, it is merely a stylistic choice.
11. Because a sort that **drops** or **duplicates** nodes still returns sorted output. You must also assert that the node count and the multiset of values are unchanged — dropping a node is the most common merge-sort bug and the ordering check is blind to it.

B. Tracing and analysis

1. Appends: 1 (a), 2 (b), 3 (**from a**, because `<=`), 3 (b), 5 (a), then `a` is exhausted and `[6]` is attached in one write. Result `1,2,3,3,5,6`.
2. `split` $\rightarrow$ `[4,2]` + `[1,3]`; each splits into singletons. Merges: `[4]+[2]` $\rightarrow$ `[2,4]` (1 comparison), `[1]+[3]` $\rightarrow$ `[1,3]` (1), then `[2,4]+[1,3]` $\rightarrow$ `[1,2,3,4]` (3 comparisons: 2vs1, 2vs3, 4vs3). **Total 5 comparisons.**
3. `len 1` $\rightarrow$ `split` is never called (base case). `len 2` $\rightarrow$ `[1]` + `[2]`. `len 3` $\rightarrow$ `[1,2]` + `[3]`. `len 4` $\rightarrow$ `[1,2]` + `[3,4]`. **Length 2** is the one that infinitely recurses under `fast = head`, splitting into `[1,2]` + `[]`.
4. `4` $\rightarrow$ `[4]` (0 comparisons). `2` $\rightarrow$ `[2,4]` (1). `1` $\rightarrow$ `[1,2,4]` (1). `3` $\rightarrow$ `[1,2,3,4]` (3: vs 1, vs 2, vs 4). **Total 5.** $n^2/4 = 4$, so the two are the same order — insertion sort's cost is genuinely quadratic in expectation.

5. `pivot = 3`. `less = [1,2]`, `equal = [3,3,3]`, `greater = []`. With a two-way split ($<$ and $\geq$), the input `[3,3,3]` would give `less = []` and `$\geq = [3,3,3]$` — the same size as the input, forever.
6. $N = 800$. Sequential: merges cost $100 + 200 \cdot \dots \approx 100(1+2+\dots+8)$ — adjusting, total node-visits ≈ 3600 . Divide and conquer: $\log_2 8 = 3$ levels $\times 800$ nodes = **2400**. Ratio $\approx 1.5\times$ at $k = 8$; at $k = 500$ the ratio is roughly $500/2 \div \log_2 500 \approx 28\times$.
7. `[2,0,2,1,0]`: zeros `[0,0]`, ones `[1]`, twos `[2,2]` $\rightarrow$ `0,0,1,2,2`. `[2,0,2,0]`: zeros `[0,0]`, ones **empty**, twos `[2,2]`. The line that matters is `z->next = oneD.next ? oneD.next : twoD.next;` — without the conditional the zeros link to an empty one-list and **both 2s are dropped**.
8. Merge sort: $\approx n \log_2 n \approx 10^6 \times 20 = 2 \times 10^7$ comparisons regardless of input order. Insertion sort on already-sorted input: the inner scan exits on the first comparison every time, so $\approx 10^6$ — twenty times fewer. The practical consequence is that production sorts detect or exploit existing runs (this is the core idea behind Timsort).

C. Code tasks — what to verify

- **C1.** The multiset check is the one that earns its keep. Test length 0 and 1 explicitly; both are base cases and both are commonly mishandled.
- **C2.** The stack trace shows `mergeSort` recursing into itself with an unchanged argument until the stack is exhausted — typically 100 000+ identical frames.
- **C3.** Expect a **10-25%** improvement. The gain comes from avoiding recursion overhead and cache misses at the leaves, not from a better asymptotic bound.
- **C4.** With $k = 500$ expect roughly a **25-30 $\times$** gap between sequential and the other two. Divide-and-conquer usually edges out the heap because it does no allocation and has better locality.
- **C5.** The failing combination is **0s and 2s with no 1s**; the 2s vanish and the result is silently short.
- **C6.** Merge sort preserves the original index order among equal keys; quicksort with a three-way partition does not, because the `greater` and `less` lists are built independently and concatenated. This is the cleanest demonstration of stability you can write in fifteen lines.

HANDNOTE SUMMARY CARD

MERGE TWO SORTED – THE FOUNDATION OF THE SECTION

```
Node dummy(0); Node* tail = &dummy;
while (a && b) {
    if (a->data <= b->data) { tail->next = a; a = a->next; }
    else { tail->next = b; b = b->next; }
    tail = tail->next;
}
tail->next = a ? a : b; // ONE write. the rest is already sorted+linked
return dummy.next;

<= keeps it STABLE. < swaps equal elements between lists.
```

MERGE SORT – THE CORRECT DEFAULT FOR A LINKED LIST

```
split: fast = head->next      <- FIRST middle. head => infinite recursion
      ... slow/fast walk ...
      mid = slow->next;
      slow->next = nullptr;    <- THE CUT. without it, never terminates
sort:  if (!head || !head->next) return head;
      return mergeTwo(mergeSort(head), mergeSort(split(head)));

O(n log n) GUARANTEED · O(log n) stack · STABLE · no auxiliary array
(arrays need O(n) scratch to merge; lists just rewrite next pointers)
std::list::sort() is exactly this. std::sort CANNOT take a list iterator.
```

SORT COMPARISON – LINKED LIST

	time	space	stable	verdict
merge sort	$O(n \log n)$	$O(\log n)$	yes	ALWAYS use this
insertion sort	$O(n^2)$, $O(n)$ ish when near-sorted	$O(1)$	yes	only if nearly sorted or $n < \sim 30$
quick sort	$O(n \log n)$ avg $O(n^2)$ worst	$O(\log n)$	no	no random pivot -> sorted input = $O(n^2)$

quicksort on a list: THREE-way partition (<, ==, >).
two-way loops forever on all-equal input.

MERGE K SORTED – N total nodes, k lists

sequential	$O(N * k)$	re-walks the accumulator every time
divide & conquer	$O(N \log k)$	no aux structure, usually fastest
min-heap	$O(N \log k)$	$O(k)$ space, right choice for streams

$\log k$ not $\log N$: the heap holds ONE node per list, never N.

SORT 0s/1s/2s

RELINK into three dummy lists, then join. $O(n)$, $O(1)$, nodes preserved.
join trap: `z->next = oneD.next ? oneD.next : twoD.next;`
(no 1s => zeros must skip straight to the 2s, or 2s are LOST)
counting method OVERWRITES data – wrong if the node carries anything else.

CHECK IF SORTED – and use it as a TEST ORACLE

loop on `c->next != nullptr` (comparing PAIRS), guard the empty list first.
after any sort assert THREE things:
sorted · node count unchanged · multiset of values unchanged
ordering alone will not catch a DROPPED node – the #1 merge sort bug.

CONSOLIDATED CHEATSHEET — PART III

THE ONE LOOP BEHIND SECTION 7

```
while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
```

guard order: fast FIRST. reversed => every even-length list null-derefs.

SPEED -> fractions of the list GAP -> offsets from the end

47 middle	fast=head -> 2nd middle · fast=head->next -> 1st middle
48 nth from end	gap n, lockstep, slow = answer
49 remove nth	gap n+1 FROM A DUMMY, slow = predecessor
50 has cycle	meet: gap shrinks by exactly 1 per step
51 cycle start	L = nC - k => head-to-start == meet-to-start
	phase 2: BOTH pointers move ONE step
52 cycle length	freeze at meeting point, walk round, count
53 remove cycle	stop one early (slow->next != fast->next); L==0 special case
54 palindrome	1st middle -> reverse half -> compare -> RESTORE
55 intersection	a = a ? a->next : headB (compare ADDRESSES not values)
	a walks p+c+q, b walks q+c+p – same number

SECTION 8 IN ONE BLOCK

56 merge two	dummy + tail; leftover attached in ONE write; <= = stable
57 merge sort	first middle + THE CUT; $O(n \log n)$, $O(\log n)$, stable
58 insertion sort	$O(n^2)$; genuinely good on nearly-sorted and tiny lists
59 quick sort	three-way partition; $O(n^2)$ on sorted input; unstable
60 merge k	$O(N \log k)$ by heap OR divide-and-conquer; $O(N*k)$ naive
61 sort 012	relink three lists; mind the no-1s join
62 is sorted	pair loop; use it as a test oracle, with a COUNT check

THE FIVE BUGS OF THIS VOLUME

1 fast->next tested before fast	crash on every even-length list
2 fast = head in the merge-sort split	infinite recursion on 2 nodes
3 missing slow->next = nullptr	halves not independent, no shrink
4 palindrome without the restore	caller's list silently truncated
5 sort012 join with no 1s present	every 2 silently dropped

UNIVERSAL POST-SORT ASSERTION

```
isSorted(result) && count(result) == count(input)
&& multiset(result) == multiset(input)
```

End of Volume III — Topics 46–62 of 88. Next: Section 9, Rearrangement Problems (63–73), which combines everything so far — rotation and reorder use the midpoint and reversal together, and "add two numbers as lists" uses the backward-recursion trick from Topic 15. Solve 15–20 problems on two-pointer patterns and merge sort before continuing; these two sections carry more interview weight than any others in the book.

PART IV

Rearrangement and Design

Rearrangement Problems · Advanced Variants — Topics 63–81

SECTION 9 — REARRANGEMENT PROBLEMS

63. Rotate List by K

Rotate right by k : $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ with $k=2$ becomes $4 \rightarrow 5 \rightarrow 1 \rightarrow 2 \rightarrow 3$. The last k nodes move to the front.

THE TRICK: CLOSE THE CIRCLE, THEN CUT IT

Rather than moving k nodes one at a time, join the tail to the head to make a circular list (Section 6), then walk to the new tail and break there. Rotation becomes **two pointer writes** regardless of k .

```
Node* rotateRight(Node* head, int k) {
    if (head == nullptr || head->next == nullptr || k == 0) return head;

    // 1. measure and find the old tail
    int n = 1;
    Node* tail = head;
    while (tail->next != nullptr) { tail = tail->next; ++n; }

    // 2. normalise k – rotating by n is the identity
    k %= n;
    if (k == 0) return head;

    // 3. close the circle
    tail->next = head;

    // 4. the new tail is at position n - k - 1 (0-based) from the old head
    Node* newTail = head;
    for (int i = 0; i < n - k - 1; ++i) newTail = newTail->next;

    Node* newHead = newTail->next;
    newTail->next = nullptr;      // 5. cut
    return newHead;
}
```

1 -> 2 -> 3 -> 4 -> 5, k = 2, n = 5

close: 1 -> 2 -> 3 -> 4 -> 5 --+
 ^ |
 +-----+-----+-----+-----+

n - k - 1 = 2 steps: newTail = [3]
newHead = [4], cut [3]->next

result: 4 -> 5 -> 1 -> 2 -> 3

$k \% n$ IS NOT OPTIONAL

With $k = 1\,000\,000$ on a 5-node list, the walk in step 4 would run a million times — or, without the modulo, $n - k - 1$ is negative, the loop body never executes, and you return the list rotated by zero. Rotating by n is the identity, so only $k \bmod n$ matters. Interviewers routinely test $k \geq n$.

Rotating **left** by k is the same code with the walk length changed to $k - 1$ — or simply `rotateRight(head, n - k % n)`.

64. Odd-Even Node Split

Group nodes by **position**, not value: all odd-indexed nodes first, then all even-indexed, each group keeping its relative order. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ becomes $1 \rightarrow 3 \rightarrow 5 \rightarrow 2 \rightarrow 4$.

```
Node* oddEvenList(Node* head) {
    if (head == nullptr || head->next == nullptr) return head;

    Node* odd      = head;          // tail of the odd chain
    Node* even      = head->next;    // tail of the even chain
    Node* evenHead = even;          // save it – we must reattach later

    while (even != nullptr && even->next != nullptr) {
        odd->next = even->next;      // odd skips over the even node
        odd      = odd->next;
        even->next = odd->next;      // even skips over the new odd node
        even      = even->next;
    }
    odd->next = evenHead;           // stitch the two chains together
    return head;
}
```

```
1 -> 2 -> 3 -> 4 -> 5      odd=[1] even=[2] evenHead=[2]

iter 1:  odd->next = [3]      1 -> 3
         odd      = [3]
         even->next = [4]      2 -> 4
         even      = [4]

iter 2:  odd->next = [5]      1 -> 3 -> 5
         odd      = [5]
         even->next = null     2 -> 4 -> null
         even      = null     loop ends

stitch:  odd->next = evenHead 1 -> 3 -> 5 -> 2 -> 4
```

THE TWO-CHAIN PATTERN

Grow two independent chains by their tails, then join them once at the end. It appears again in Topics 65, 69 and 61 — and it is the reason the loop needs `evenHead`: once `even` starts advancing, the head of the even chain is unreachable unless you saved it on the first line.

THE GUARD IS `even && even->next`, NOT `odd->next`

The even chain always ends first or simultaneously, so testing it covers both parities. An odd-length list exits when `even->next` is null; an even-length list exits when `even` itself is null. Testing `odd->next` instead dereferences null on even-length input.

65. Segregate Even and Odd Values

Same shape as Topic 64, but grouping by **value parity** rather than position: all even values first, then all odd values.

```
Node* segregateEvenOdd(Node* head) {
    Node evenD(0), oddD(0);           // two dummy heads
    Node* et = &evenD;
    Node* ot = &oddD;

    for (Node* c = head; c != nullptr; c = c->next) {
        if (c->data % 2 == 0) { et->next = c; et = c; }
        else { ot->next = c; ot = c; }
    }
    ot->next = nullptr;                // terminate the odd chain FIRST
    et->next = oddD.next;               // then join
    return evenD.next ? evenD.next : oddD.next;
}
```

TERMINATE BEFORE YOU JOIN

`ot->next = nullptr;` must come before `et->next = oddD.next;`. The last odd node still points at whatever followed it in the original list — often an even node that is now earlier in the result. Joining first and terminating second cuts the chain in the wrong place, or leaves a **cycle**. Verify with Topic 50 after implementing.

Note `c->data % 2 == 0` is wrong for negative values in C++ — `-3 % 2` is `-1`, not `1`. Use `c->data % 2 != 0` to test odd, or `(c->data & 1) == 0` for even, which is correct for two's-complement negatives and faster besides.

segregate by value – Time $O(n)$ · Space $O(1)$ · stable within each group; mind negative operands

66. Reorder List

Interleave the list with its own reverse: $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ becomes $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$. For $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ the answer is $1 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow 3$.

THREE TECHNIQUES YOU ALREADY HAVE

Find the middle (Topic 47) → reverse the second half (Topic 29) → merge the two halves alternately (a variation of Topic 56). This problem is the clearest demonstration in the book that the sections compose: nothing new is required, only assembly.

```

void reorderList(Node* head) {
    if (head == nullptr || head->next == nullptr) return;

    // 1. first middle
    Node* slow = head;
    Node* fast = head->next;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // 2. cut and reverse the second half
    Node* second = slow->next;
    slow->next = nullptr;
    Node* prev = nullptr;
    while (second != nullptr) {
        Node* nxt = second->next;
        second->next = prev;
        prev = second;
        second = nxt;
    }

    // 3. alternate-merge first and prev
    Node* a = head;
    Node* b = prev;
    while (b != nullptr) {
        Node* an = a->next;
        Node* bn = b->next;
        a->next = b;
        b->next = an;    // may be nullptr on the last splice – correct
        a = an;
        b = bn;
    }
}

```

1 2 3 4 5

middle (first): [1 2 3] and [4 5]
 reverse second: [1 2 3] and [5 4]
 alternate merge: 1 -> 5 -> 2 -> 4 -> 3

the first half is always $\geq$ the second, so `a` never runs out early
 -- that is exactly why the FIRST middle convention is required here.

WHY while (b) **AND NOT** while (a && b)

Splitting at the first middle guarantees the first half has the same length as the second, or one more. So the second half is exhausted first, and driving the loop on **b** is both sufficient and correct. Using the second-middle convention makes the second half longer, **a** becomes null first, and **a->next** crashes.

reorder list – Time $O(n)$ · Space $O(1)$ · middle + reverse + alternate merge

67. Swap Nodes in Pairs

Swap every two adjacent nodes: 1→2→3→4 becomes 2→1→4→3. The point of the exercise is to **swap the nodes, not their values**.

```

Node* swapPairs(Node* head) {
    Node dummy(0, head);
    Node* prev = &dummy;

    while (prev->next != nullptr && prev->next->next != nullptr) {
        Node* first = prev->next;
        Node* second = first->next;

        first->next = second->next;    // 1. first points past second
        second->next = first;          // 2. second points back at first
        prev->next = second;           // 3. predecessor adopts second

        prev = first;                 // first is now the pair's tail
    }
    return dummy.next;
}

```

```

dummy -> 1 -> 2 -> 3 -> 4

prev=dummy  first=[1]  second=[2]

first->next = [3]      1 -> 3
second->next = [1]     2 -> 1 -> 3
prev->next   = [2]     dummy -> 2 -> 1 -> 3

prev = [1]            (the tail of the swapped pair)

next round swaps [3] and [4]:  dummy -> 2 -> 1 -> 4 -> 3

```

WHY NOT JUST SWAP THE DATA?

`std::swap(first->data, second->data)` is one line and produces identical output. It is also the wrong answer whenever anything outside the list holds a pointer to a node — an LRU cache's hash map, an iterator, an intrusive kernel list. Those pointers would now refer to nodes holding different values. The problem is stated as "swap nodes" precisely to test whether you notice the difference (compare Topic 27).

An odd trailing node is handled by the loop guard: `prev->next->next` is null, the loop stops, and the last node stays where it is. This is the $k = 2$ case of reverse-in-groups (Topic 31) — recognising that connection is worth more than memorising this function.

swap in pairs – Time $O(n)$ · Space $O(1)$ iterative · relink, never swap payloads

68. Swap Kth from Start and End

Swap the k -th node from the start with the k -th from the end. The straightforward version swaps values; the honest version swaps nodes, and is considerably harder.

```

Node* swapKth(Node* head, int k) {
    int n = 0;
    for (Node* c = head; c; c = c->next) ++n;
    if (k < 1 || k > n) return head;
    if (2 * k - 1 == n) return head;          // same node – nothing to do

    // predecessors of the two targets (nullptr means "the head itself")
    Node *xPrev = nullptr, *x = head;
    for (int i = 1; i < k; ++i) { xPrev = x; x = x->next; }

    Node *yPrev = nullptr, *y = head;
    for (int i = 1; i < n - k + 1; ++i) { yPrev = y; y = y->next; }

    if (xPrev) xPrev->next = y; else head = y;
    if (yPrev) yPrev->next = x; else head = x;

    std::swap(x->next, y->next);              // exchange the outgoing links
    return head;
}

```

THE THREE CASES THAT BREAK NAIVE CODE

- **One of the targets is the head** ($k = 1$). Its predecessor is null, so `head` itself must be reassigned — hence the `else head = ...` branches.
- **The two targets are the same node** (n odd and k the exact middle, i.e. $2k - 1 = n$). Swapping a node with itself through this code produces a self-loop. Return early.
- **The two targets are adjacent.** After the predecessor rewrites, `std::swap(x->next, y->next)` still gives the right structure — trace $n = 4, k = 2$ by hand to convince yourself, because it looks wrong and is not.

If the problem permits swapping values, the whole thing collapses to finding both nodes and calling `std::swap(x->data, y->data)`. Ask which is wanted; the node version exists to test edge-case discipline, not cleverness.

swap k-th from both ends – Time $O(n)$ · Space $O(1)$ · guard head, self-swap, and adjacency

69. Partition List Around Value

Move all nodes with value less than x before all nodes with value $\geq x$, **preserving the relative order within each group**. That stability requirement is what makes this a linked-list problem rather than a two-pointer array swap.


```

Node* partition(Node* head, int x) {
    Node lessD(0), geD(0);           // two dummy heads
    Node* lt = &lessD;
    Node* ge = &geD;

    for (Node* c = head; c != nullptr; c = c->next) {
        if (c->data < x) { lt->next = c; lt = c; }
        else             { ge->next = c; ge = c; }
    }
    ge->next = nullptr;               // TERMINATE before joining
    lt->next = geD.next;               // then join
    // works even if the less-chain is empty
    return lessD.next;
}

```

partition [1,4,3,2,5,2] around x = 3

less chain: 1 -> 2 -> 2
ge chain: 4 -> 3 -> 5

join: 1 -> 2 -> 2 -> 4 -> 3 -> 5

order WITHIN each group is unchanged – 4 still precedes 3.

An array two-pointer swap gives 1,2,2,5,3,4: same partition, not stable.

WHY THE LIST VERSION IS EASIER THAN THE ARRAY VERSION

A stable array partition needs $O(n)$ scratch space, because moving an element means shifting everything after it. A linked list partitions stably in $O(1)$ space by *relinking* into two chains and joining them once. This is the same structural advantage that makes merge sort natural on lists (Topic 57) — and it is a genuinely good answer to "when would you actually choose a linked list?"

`return lessD.next` handles the case where nothing is less than `x`: `lessD.next` was set by `lt->next = geD.next` in the join, because `lt` is still `&lessD`. No branch needed. Note also the equal case goes into the `ge` chain — `<`, not `<=` — which matches the standard problem statement.

partition – Time $O(n)$ one pass · Space $O(1)$ · stable, unlike the array two-pointer swap

70. Remove Duplicates (Sorted)

In a sorted list all duplicates are adjacent, so one pass with no extra memory suffices.

Keep one copy of each value

```
Node* removeDuplicates(Node* head) {
    for (Node* c = head; c != nullptr && c->next != nullptr; ) {
        if (c->data == c->next->data) {
            Node* doomed = c->next;
            c->next = doomed->next;    // unlink
            delete doomed;
            // do NOT advance – the new c->next is still unexamined
        } else {
            c = c->next;
        }
    }
    return head;
}
```

Remove every node that has a duplicate

```
Node* removeAllDuplicates(Node* head) {
    Node dummy(0, head);
    Node* prev = &dummy;           // last confirmed-unique node
    Node* curr = head;

    while (curr != nullptr) {
        if (curr->next != nullptr && curr->data == curr->next->data) {
            int v = curr->data;
            while (curr != nullptr && curr->data == v) {
                Node* doomed = curr;
                curr = curr->next;
                delete doomed;    // free the whole run
            }
            prev->next = curr;    // bridge over it
        } else {
            prev = curr;
            curr = curr->next;
        }
    }
    return dummy.next;
}
```

input 1 -> 2 -> 2 -> 3 -> 3 -> 3 -> 4

keep one of each -> 1 -> 2 -> 3 -> 4

remove ALL duplicated -> 1 -> 4 (2 and 3 vanish entirely)

Read the problem statement. These are different questions.

THE NON-ADVANCE RULE, AGAIN

After unlinking, `c->next` already points at the next unexamined node. Advancing anyway skips it, which fails on three or more consecutive equal values — `[1,1,1]` is the minimal failing case. This is the same discipline as Topic 23; if you have internalised it there, both functions above write themselves.

remove duplicates (sorted) – Time $O(n)$ one pass · Space $O(1)$ · dummy needed for the remove-all form

71. Remove Duplicates (Unsorted)

Without sorting, duplicates can be anywhere, so you trade memory for a single pass.

```
#include <unordered_set>

Node* removeDuplicatesUnsorted(Node* head) {
    if (head == nullptr) return head;

    std::unordered_set<int> seen;
    seen.insert(head->data);

    Node* prev = head;
    Node* curr = head->next;

    while (curr != nullptr) {
        if (seen.count(curr->data)) {
            prev->next = curr->next;    // unlink the duplicate
            Node* doomed = curr;
            curr = curr->next;
            delete doomed;
        } else {
            seen.insert(curr->data);
            prev = curr;
            curr = curr->next;
        }
    }
    return head;
}
```

Approach	Time	Space	When
Hash set	$O(n)$ average	$O(n)$	Default. The right answer almost always.
Nested loops	$O(n^2)$	$O(1)$	Only if memory is genuinely unavailable
Sort first, then Topic 70	$O(n \log n)$	$O(\log n)$	If the output may be reordered

THE FOLLOW-UP QUESTION IS THE REAL QUESTION

"Now do it without extra memory." The answer is the $O(n^2)$ nested loop: for each node, walk the rest of the list and unlink matches. State the trade-off explicitly — hash set is $O(n)$ time and $O(n)$ space; the nested loop is $O(1)$ space and $O(n^2)$ time; sorting first is $O(n \log n)$ but *destroys the original order*, which may or may not be allowed. Naming all three and asking which constraint dominates is a better answer than producing any one of them.

remove duplicates (unsorted) – Time $O(n)$ with a hash set · Space $O(n)$ · $O(n^2)/O(1)$ without

72. Add Two Numbers as Lists

Each list holds the digits of a number. Add them and return the sum as a list.

Reverse order (least significant digit first) — the easy direction

```
Node* addTwoNumbers(Node* a, Node* b) {
    Node dummy(0);
    Node* tail = &dummy;
    int carry = 0;

    while (a != nullptr || b != nullptr || carry != 0) {
        int sum = carry;
        if (a) { sum += a->data; a = a->next; }
        if (b) { sum += b->data; b = b->next; }

        carry = sum / 10;
        tail->next = new Node(sum % 10);
        tail = tail->next;
    }
    return dummy.next;
}
```

THE LOOP CONDITION DOES THREE JOBS

`while (a || b || carry)` handles unequal lengths *and* the final carry in one expression. `[9,9] + [1] = 99 + 1 = 100` → the loop runs three times, the third driven purely by `carry`, producing the leading 1. Writing the loop as `while (a && b)` with two clean-up loops afterwards is three times the code and where the dropped-carry bug lives.

Forward order (most significant digit first) — the hard direction

Addition must proceed from the least significant digit, but a singly linked list can only be read forward. Three ways out:

1. **Reverse both lists**, add as above, reverse the result. Three passes, $O(1)$ extra space, and it mutates the inputs unless you restore them.
2. **Use two stacks**. Push both lists, pop together — the pops arrive least-significant first. $O(m + n)$ space, and the cleanest to write under time pressure.
3. **Recurse to the end** and add on the way back up — Topic 15's backward-traversal trick, using the call stack as the storage. Elegant; $O(n)$ stack, so unsafe for long lists.

reverse order	<code>[2,4,3]</code>	+	<code>[5,6,4]</code>	=	<code>342 + 465 = 807</code>	->	<code>[7,0,8]</code>
forward order	<code>[3,4,2]</code>	+	<code>[4,6,5]</code>	=	<code>342 + 465 = 807</code>	->	<code>[8,0,7]</code>

same numbers, and the second needs a stack or a reversal to reach the ones.

BUILD THE RESULT, DO NOT REUSE THE INPUTS

It is tempting to write the sum digits back into list *a*. That breaks when *b* is longer and when a final carry adds a digit, and it destroys an input the caller may still need. Allocate fresh nodes onto a dummy — the cost is *n* allocations you were going to make anyway for the result.

add two numbers — Time $O(\max(m,n))$ · Space $O(\max(m,n))$ for the result · `while (a || b || carry)`

73. Flatten a Multilevel List

Each node has an extra **child** pointer that may head a whole sublist, which itself may have children. Flatten everything into a single level, depth-first, with the child list spliced in immediately after its parent.

```
struct MNode {
    int    data;
    MNode* prev;
    MNode* next;
    MNode* child;
};

MNode* flatten(MNode* head) {
    for (MNode* c = head; c != nullptr; c = c->next) {
        if (c->child == nullptr) continue;

        MNode* nxt = c->next;          // 1. remember what follows

        // 2. splice the child list in after c
        c->next      = c->child;
        c->next->prev = c;
        // the child pointer must be cleared
        c->child      = nullptr;

        // 3. walk to the end of the newly spliced-in run
        MNode* tail = c->next;
        while (tail->next != nullptr) tail = tail->next;

        // 4. reattach the remembered remainder
        tail->next = nxt;
        if (nxt != nullptr) nxt->prev = tail;
    }
    return head;
}
```

```
1 -- 2 -- 3 -- 4
  |
  7 -- 8 -- 9
    |
    11 -- 12
```

flatten (depth first): 1 2 7 8 11 12 9 3 4

The outer loop keeps advancing into the spliced-in nodes, so nested children are found and flattened on later iterations – no recursion needed.

WHY THE ITERATIVE VERSION HANDLES ARBITRARY DEPTH

After splicing a child list in, **c** continues walking *into* it. If one of those nodes has its own child, the same branch fires again on a later iteration. The nesting is handled by the fact that the list grows in front of the cursor — no stack, no recursion, $O(1)$ space. A recursive version is easier to write but costs $O(\text{depth})$ stack.

THREE THINGS THAT ARE EASY TO FORGET

- **Clear `child`.** A flattened list with dangling child pointers still passes a forward traversal test but is structurally wrong, and re-flattening it duplicates nodes.
- **Fix every `prev`.** This is a doubly linked list; both spliced boundaries need their back-links repaired (Topic 34's invariant).
- **Guard `next` for null.** When the parent was the last node, `next->prev` is a null dereference.

flatten multilevel – Time $O(n)$, each node visited once · Space $O(1)$ iterative, $O(\text{depth})$ recursive

PRACTICE SET — Topics 63-73

A. Conceptual (answer in your own words)

1. Why does `rotateRight` close the list into a circle instead of moving k nodes individually? State the complexity of both approaches.
2. Give two distinct failures that occur if `k % n` is omitted from the rotation.
3. Topic 64 groups by position and Topic 65 by value. Both use two chains — name the one extra variable Topic 64 needs and explain why.
4. In the odd-even split, why is the loop guard written on `even` rather than on `odd`?
5. In `segregateEvenOdd`, why must the terminate step precede the join step? Describe the exact structure produced by the wrong order.
6. Reorder List requires the *first* middle. Explain what breaks with the second-middle convention, naming the line that crashes.
7. "Swap nodes" and "swap values" produce identical output for Topic 67. Name a structure that breaks under the wrong choice and say why.
8. List the three edge cases in `swapKth` and say what each produces if unguarded.
9. Why is a stable partition $O(1)$ space on a linked list but $O(n)$ space on an array?
10. Give the minimal input that distinguishes "keep one of each duplicate" from "remove every duplicated value", and both outputs.
11. State the three approaches to removing duplicates from an unsorted list with their trade-offs, and say which constraint decides between them.
12. Why does `while (a || b || carry)` replace three separate loops? Give an input where the `carry` term alone drives the final iteration.
13. Explain why the iterative flatten handles arbitrary nesting depth without a stack.

B. Tracing (pen and paper)

1. Trace `rotateRight([1,2,3,4,5], 2)`: give n , the normalised k , the walk length, `newTail`, and the result. Repeat for $k = 5$ and $k = 7$.
2. Trace `oddEvenList([1,2,3,4,5,6])`. Give `odd` and `even` after each iteration and the final list. Confirm which guard term ends the loop.
3. Trace `reorderList([1,2,3,4,5,6])`: the two halves after the cut, after the reversal, and the sequence of splices. State how many times the merge loop runs.
4. Trace `swapPairs([1,2,3])`. Give `prev`, `first`, `second` each round, and explain how the odd trailing node survives untouched.
5. Trace `swapKth([1,2,3,4], 2)` — the adjacent case. Give `xPrev`, `x`, `yPrev`, `y`, and the list after each of the three writes.
6. Trace `partition([1,4,3,2,5,2], 3)`. Give both chains before the join and the final list. Then give what an array two-pointer swap would produce, and name the property that differs.
7. Trace `removeAllDuplicates([1,2,2,3,3,3,4])`. Give `prev` and `curr` at each outer iteration and the final list.
8. Compute `addTwoNumbers([9,9,9], [1])` digit by digit, giving `sum` and `carry` each round, and the final list. State how many iterations run and which condition drove the last one.
9. Flatten the structure in Topic 73's diagram by hand, writing the order in which nodes are visited and the iteration on which each child splice occurs.

C. Code tasks

1. Implement `rotateRight` and test $k = 0, 1, n-1, n, n+1$ and 10^6 on a 5-node list. All six must be correct and none may loop more than n times.
2. Implement `segregateEvenOdd` with the join and terminate steps swapped, then run `hasCycle` (Topic 50) on the result. Report what it finds.
3. Implement `reorderList` using the second-middle convention and record the crash. Then explain the stack trace in one sentence.
4. Implement `swapPairs` both by relinking and by swapping data. Then build an `unordered_map<int, Node*>` over the original nodes, run both versions, and show that the map is still correct after one and corrupted after the other.
5. Implement all three unsorted-duplicate-removal approaches. Benchmark on 100 000 nodes with 10% distinct values and report all three times.
6. Implement `addTwoNumbers` for the **forward** order using two stacks, then again by reversing and restoring. Verify both against $342 + 465$, $999 + 1$, and two 200-digit numbers.
7. Implement `flatten` and write a validator that checks every `child` is null, every `prev` agrees with its `next`, and the node count is unchanged.

ANSWER KEY — Topics 63-73

A. Conceptual

1. Because closing the circle makes rotation independent of k : one walk of $n-k-1$ and two pointer writes. Moving k nodes one at a time is $O(n \cdot k)$ — for each of k rotations you must find the tail again. The circle method is **$O(n)$** regardless of k .
2. (a) For $k > n$, $n - k - 1$ is negative, the walk loop never runs, and the function returns the list rotated by zero — a silent wrong answer. (b) For very large k without the negative case, the walk would run k times, turning $O(n)$ into $O(k)$.
3. **evenHead**. The odd chain is reachable from **head**, but once **even** starts advancing, nothing points at the *first* even node — and it is needed for the final stitch. Topic 65 uses dummy heads instead, which serve the same purpose.
4. Because the even chain always ends first or at the same time. An odd-length list exits on **even->next == nullptr**; an even-length list exits on **even == nullptr**. Testing **odd->next** misses the second case and dereferences null.
5. The last odd node still points at whatever followed it originally — usually an even node now placed earlier in the result. Joining first leaves that stale link, producing either a truncated list or a **cycle** (the odd tail pointing back into the even chain).
6. With the second middle, the second half is *longer* than the first, so in the merge loop **a** becomes **nullptr** while **b** is still valid, and **Node* an = a->next;** dereferences null.
7. An **LRU cache**. Its hash map stores node pointers; swapping payloads leaves those pointers aimed at nodes now holding different keys, so every lookup silently returns the wrong value. Relinking preserves node identity.
8. (a) $k = 1$ — a target is the head, whose predecessor is null, so **head** itself must be re-assigned; unguarded you write through a null pointer. (b) $2k-1 = n$ — the two targets are the same node, and the swap produces a **self-loop**. (c) Adjacent targets — correct as written, but must be traced to confirm.
9. Because a list partitions by **relinking** into two chains and joining once — no element ever moves. An array must physically shift elements to keep order, and shifting in place destroys unread values, so a stable array partition needs an $O(n)$ scratch buffer.
10. **[1,1,2]**. Keep-one-of-each $\rightarrow$ **[1,2]**. Remove-every-duplicated $\rightarrow$ **[2]**. Both are common problem statements and they are different questions.
11. Hash set: $O(n)$ time, $O(n)$ space, order preserved. Nested loops: $O(n^2)$ time, $O(1)$ space, order preserved. Sort first: $O(n \log n)$ time, $O(\log n)$ space, **order destroyed**. The deciding constraint is whether extra memory is available and whether the original order must be preserved.
12. Because it covers unequal lengths and the final carry in one expression rather than a main loop plus two clean-up loops. **[9,9] + [1]**: iterations give $(9+1=10, \text{carry } 1)$, $(9+0+1=10, \text{carry } 1)$, then **a third iteration driven only by carry**, emitting the leading $1 \rightarrow$ **[0,0,1]**.
13. Because the spliced-in child nodes are placed *in front of the cursor*, so the same outer loop walks into them and fires the same branch again on any node that has its own child. The list grows ahead of the cursor, which is the iterative equivalent of a recursion stack.

B. Tracing

1. $n = 5$. $k = 2 \rightarrow$ walk $n-k-1 = 2 \rightarrow$ newTail = [3], result **4,5,1,2,3**. $k = 5 \rightarrow k \% n$ gives 0 $\rightarrow$ early return, unchanged. $k = 7 \rightarrow k = 2 \rightarrow$ identical to the first case, **4,5,1,2,3**.

2. odd/even after each iteration: $([3],[4]) \rightarrow ([5],[6])$. Then `even->next` is null so the loop exits — the `even->next != nullptr` term ends it. Stitch gives `1,3,5,2,4,6`.
3. Cut $\rightarrow [1,2,3]$ and `[4,5,6]`; reversed $\rightarrow [6,5,4]$. Splices: 1 $\rightarrow$ 6, 2 $\rightarrow$ 5, 3 $\rightarrow$ 4 giving `1,6,2,5,3,4`. The merge loop runs **3** times, once per node of the second half.
4. Round 1: `prev = dummy`, `first = [1]`, `second = [2]` $\rightarrow$ `2,1,3`, `prev = [1]`. Round 2: `prev->next = [3]` but `prev->next->next` is null, so the guard fails and the loop stops. `[3]` is never touched and remains the tail.
5. $n = 4$, $k = 2$: `xPrev = [1]`, `x = [2]`; `yPrev = [2]`, `y = [3]`. Writes: `[1]->next = [3]`; `yPrev->next = x` i.e. `[2]->next = [2]` (temporarily a self-loop); then `swap(x->next, y->next)` exchanges `[2]`'s and `[3]`'s outgoing links, giving `[3]->next = [2]` and `[2]->next = [4]`. Final `1,3,2,4`. ✓
6. `less = 1,2,2`; `ge = 4,3,5`; joined `1,2,2,4,3,5`. An array two-pointer swap gives `1,2,2,5,3,4` — the same partition but the `ge` group's internal order is destroyed. The property that differs is **stability**.
7. `curr = [1]` (unique) $\rightarrow$ `prev = [1]`. `curr = [2]`: duplicate run, both 2s freed, `curr = [3]`, `[1]->next = [3]`. `curr = [3]`: duplicate run, all three 3s freed, `curr = [4]`, `[1]->next = [4]`. `curr = [4]` unique $\rightarrow$ `prev = [4]`, `curr = null`. Result `1  $\rightarrow$  4`.
8. `i1`: `sum = 9+1 = 10`, digit 0, carry 1. `i2`: `sum = 9+0+1 = 10`, digit 0, carry 1. `i3`: `sum = 9+0+1 = 10`, digit 0, carry 1. `i4`: `sum = 0+1 = 1`, digit 1, carry 0 — driven **only by carry**. **4 iterations**, result `[0,0,0,1] = 1000 = 999 + 1`. ✓
9. Visit order `1, 2, 7, 8, 11, 12, 9, 3, 4`. The child of 2 splices on iteration 2; the child of 8 splices on iteration 4, *after* the cursor has already walked into the first spliced run — which is exactly the nesting mechanism.

C. Code tasks — what to verify

- **C1.** $k = n$ and $k = 10^6$ are the two that fail without the modulo; both must return the list unchanged ($10^6 \bmod 5 = 0$).
- **C2.** `hasCycle` returns **true** — the odd tail still points back into the even chain. This is the cleanest demonstration in the book of why terminate-before-join matters.
- **C3.** The trace shows a null dereference at `Node* an = a->next`; because the second half was longer and `a` ran out first.
- **C4.** After the relink version the map is still correct; after the data-swap version every entry points at a node holding its neighbour's value.
- **C5.** Expect roughly: hash set ~10 ms, sort-then-dedupe ~40 ms, nested loops **minutes** (10^{10} comparisons at 10% distinct). The gap is the point.
- **C6.** The two-stack version is shorter and does not touch the inputs; the reverse version needs a restore pass or it mutates the caller's lists. Both must produce the leading carry digit for `999 + 1`.
- **C7.** The `child != nullptr` check is the one people fail — a forward traversal looks perfect while the child pointers are still dangling.

HANDNOTE SUMMARY CARD

THE TWO-CHAIN PATTERN – TOPICS 61, 64, 65, 69 ARE ALL THIS

```
Node aD(0), bD(0); Node* at = &aD; Node* bt = &bD;
for (Node* c = head; c; c = c->next)
    if (test(c)) { at->next = c; at = c; }
    else        { bt->next = c; bt = c; }
bt->next = nullptr; // TERMINATE FIRST
at->next = bD.next;  // then JOIN
return aD.next;
```

join before terminate => stale link => truncated list or a CYCLE
order within each chain is preserved => STABLE, in $O(1)$ space
(an array needs $O(n)$ scratch for a stable partition – this is a real reason to choose a linked list)

63 ROTATE BY K – close the circle, then cut

```
n = count, tail->next = head; k %= n; (k %= n IS NOT OPTIONAL)
walk n-k-1 -> newTail; newHead = newTail->next; newTail->next = null
 $O(n)$  regardless of k. moving k nodes one at a time is  $O(n*k)$ .
```

64 ODD-EVEN by POSITION guard on `even`, save `evenHead` on line 1

65 SEGREGATE by VALUE $(x \& 1) \text{ not } (x \% 2)$ – negatives break %

66 REORDER $L_0 L_n L_1 L_{n-1} \dots$ = middle + reverse + alternate merge
FIRST middle required. second middle => `a` runs out => a->next crashes
drive the merge loop on `b`, never on `a`

67 SWAP IN PAIRS dummy; first->next=second->next; second->next=first;

prev->next=second; prev=first. == k-groups with k=2

68 SWAP KTH BOTH ENDS guard: $k==1$ (head) · $2k-1==n$ (same node) · adjacent

70 DUPES (SORTED) one pass, no memory. do NOT advance after unlinking.
"keep one" vs "remove all duplicated" are DIFFERENT:
[1,1,2] -> [1,2] vs [2]

71 DUPES (UNSORTED) hash set $O(n)/O(n)$ · nested $O(n^2)/O(1)$ · sort $O(n \log n)$
but sorting DESTROYS the original order

72 ADD TWO NUMBERS while (a || b || carry) <- one loop does all 3 jobs
digit = sum % 10; carry = sum / 10; build a NEW list
forward order => two stacks, or reverse + restore

73 FLATTEN splice child after parent, walk to its tail, reattach remainder
clear c->child · fix BOTH prev links · guard nxt for null
iterative handles any depth: list grows in front of the cursor

UNIVERSAL CHECKS AFTER ANY REARRANGEMENT

hasCycle(result) == false · node count unchanged · no lost tail

SECTION 10 — ADVANCED VARIANTS

74. Dummy / Sentinel Node

A dataless node placed before the real head, so that the first node stops being a special case. It has appeared in roughly a third of this book; this topic states the rule behind it.

```
Node dummy(0, head);           // stack-allocated: cannot leak, cannot be lost
Node* prev = &dummy;

// ... operate uniformly: every real node now HAS a predecessor ...

return dummy.next;             // the head, possibly changed
```

Before and after

```
// WITHOUT a dummy – the head needs its own branch
if (head != nullptr && head->data == v) {
    Node* d = head; head = head->next; delete d;
}
Node* prev = head;
while (prev && prev->next) { ... }

// WITH a dummy – one uniform loop
Node dummy(0, head);
for (Node* prev = &dummy; prev->next; )
    if (prev->next->data == v) { /* unlink */ } else prev = prev->next;
head = dummy.next;
```

THE RULE

Use a dummy whenever an operation **can change which node is first**. That covers delete-by-value, remove-n-th-from-end, partition, merge, reverse-sublist, swap-pairs, and every build-a-new-list problem. The dummy is the head's predecessor, so the head loses its special status entirely.

Property	Dummy (local, temporary)	Sentinel (permanent member)
Lifetime	One function call	The whole list's lifetime
Storage	Stack — <code>Node dummy(0, head);</code>	By-value member of the list class
Purpose	Remove one function's edge case	Remove <i>all</i> edge cases, permanently
Seen in	Topics 23, 32, 49, 56, 67, 69	Topic 41, <code>std::list</code>

ALLOCATE IT ON THE STACK

`Node* dummy = new Node(0, head);` requires a matching `delete` on every return path — including early returns and exception paths. `Node dummy(0, head);` is destroyed automatically and cannot leak. There is no reason to heap-allocate a dummy.

dummy node — Space $O(1)$, one stack node · removes head special-casing · use it by default

75. Tail Pointer Optimization

Caching a pointer to the last node turns append from $O(n)$ into $O(1)$. The cost is a maintenance obligation that most bugs in hand-written lists come from.

```
class List {
    Node* head = nullptr;
    Node* tail = nullptr;
    int    sz    = 0;
public:
    void pushBack(int v) {
        Node* n = new Node(v);
        if (!head) head = tail = n;
        else      { tail->next = n; tail = n; }
        ++sz;
    }
};
```

THE AUDIT LIST — EVERY ONE OF THESE MUST UPDATE `tail`

Operation	What must happen to <code>tail</code>
<code>pushBack</code>	Becomes the new node
<code>pushFront</code> on an empty list	Becomes the new node too
<code>popBack</code> / delete tail	Moves back to the predecessor
<code>popFront</code> on a one-node list	Becomes <code>nullptr</code>
delete by value / by position	Moves back if the tail was removed
<code>reverse</code>	Swaps with <code>head</code>
<code>clear</code>	Becomes <code>nullptr</code>
rotate, reorder, partition, sort	Must be recomputed — the last node changes

A stale tail is a dangling pointer, and the crash surfaces later, inside an unrelated `pushBack`.

THE DEFENCE

Write a `checkInvariants()` that asserts `tail->next == nullptr`, `sz` equals a forward count, and `tail` is reachable from `head`. Call it after every mutation in debug builds. Cached state is only as correct as your least careful mutator — this applies equally to a cached `size` (Topic 26).

tail pointer – append $O(n) \rightarrow O(1)$ · delete-tail stays $O(n)$ in an SLL · audit every mutator

76. Clone List with Random Pointer

Each node has `next` plus a `random` pointer to any node in the list, or null. Produce a **deep copy**: same structure, entirely new nodes.

```
struct RNode { int data; RNode* next; RNode* random; };
```

The difficulty: when you copy node *A* and it points randomly at *D*, the copy of *D* may not exist yet — so you cannot set `random` during a single forward pass.

Method 1 — hash map, $O(n)$ space

```
RNode* cloneWithMap(RNode* head) {
    std::unordered_map<RNode*, RNode*> m;
    for (RNode* c = head; c; c = c->next)
        m[c] = new RNode{c->data, nullptr, nullptr};    // pass 1: all nodes

    for (RNode* c = head; c; c = c->next) {            // pass 2: all links
        m[c]->next = c->next ? m[c->next] : nullptr;
        m[c]->random = c->random ? m[c->random] : nullptr;
    }
    return head ? m[head] : nullptr;
}
```

Method 2 — interleaving, $O(1)$ space

```
RNode* cloneInterleaved(RNode* head) {
    if (!head) return nullptr;

    // 1. weave a copy in after every original: A A' B B' C C'
    for (RNode* c = head; c; c = c->next->next)
        c->next = new RNode{c->data, c->next, nullptr};

    // 2. set random on the copies — the copy of X is always X->next
    for (RNode* c = head; c; c = c->next->next)
        c->next->random = c->random ? c->random->next : nullptr;

    // 3. unweave: restore the original list and extract the copy
    RNode* copyHead = head->next;
    for (RNode* c = head; c; c = c->next) {
        RNode* copy = c->next;
        c->next = copy->next;
        copy->next = c->next ? c->next->next : nullptr;
    }
    return copyHead;
}
```

```
original      A --> B --> C
weave         A -> A' -> B -> B' -> C -> C'

the copy of X is ALWAYS X->next.  so if A.random == C,
then A'.random = A.random->next = C'.  No map needed.

unweave       A --> B --> C      and      A' -> B' -> C'
```

THE IDEA WORTH REMEMBERING

The interleaving *encodes the map inside the list itself*: the relation "original $\rightarrow$ copy" becomes the relation "node $\rightarrow$ its next", which costs nothing to store. That is the whole trick, and it generalises — whenever you need a temporary association between old and new nodes, consider whether the structure can hold it.

STEP 3 MUST RESTORE THE ORIGINAL

If you only extract the copy and do not repair the originals' `next` pointers, the caller's list is left interleaved with foreign nodes. Same discipline as the palindrome restore (Topic 54): a function asked to *copy* must not mutate its input.

clone with random – map: $O(n)$ time, $O(n)$ space · interleaved: $O(n)$ time, $O(1)$ space

77. LRU Cache (DLL + Hash Map)

The single most-asked design question built on linked lists, and the payoff for Section 5. Requirement: `get` and `put` both in **$O(1)$** , evicting the least recently used key when full.

WHY THIS EXACT PAIR OF STRUCTURES

A hash map gives $O(1)$ lookup but no notion of order. A doubly linked list gives $O(1)$ unlink and $O(1)$ move-to-front but $O(n)$ lookup. Combine them: the map stores `key → DNode*`, and the list keeps recency order. Every operation is then a map lookup plus a constant number of pointer writes. It must be *doubly* linked — unlinking an interior node from the stored pointer alone is impossible in an SLL (Topic 36).


```

class LRUCache {
    struct DNode {
        int key, val;
        DNode *prev, *next;
        DNode(int k, int v)
            : key(k), val(v), prev(nullptr), next(nullptr) {}
    };

    int capacity;
    std::unordered_map<int, DNode*> map;
    DNode* head;           // sentinel: head->next is most recent
    DNode* tail;           // sentinel: tail->prev is least recent

    void unlink(DNode* n) {
        n->prev->next = n->next;
        n->next->prev = n->prev;
    }
    void pushFront(DNode* n) {
        n->next = head->next;
        n->prev = head;
        head->next->prev = n;
        head->next = n;
    }

public:
    LRUCache(int cap) : capacity(cap) {
        head = new DNode(0, 0);
        tail = new DNode(0, 0);
        head->next = tail;
        tail->prev = head;
    }

    int get(int key) {
        auto it = map.find(key);
        if (it == map.end()) return -1;
        unlink(it->second);
        pushFront(it->second);           // touch: move to most recent
        return it->second->val;
    }

    void put(int key, int value) {
        auto it = map.find(key);
        if (it != map.end()) {
            it->second->val = value;
            unlink(it->second);
            pushFront(it->second);
            return;
        }
        if ((int)map.size() == capacity) {
            DNode* victim = tail->prev; // least recently used
            unlink(victim);
            map.erase(victim->key);     // ERASE BEFORE DELETE
            delete victim;
        }
        DNode* n = new DNode(key, value);
        map[key] = n;
        pushFront(n);
    }

    ~LRUCache() {
        DNode* c = head;

```

```

    while (c) { DNode* nx = c->next; delete c; c = nx; }
}
};

```

THE TWO BUGS THAT ALWAYS APPEAR

- **Erasing the map entry after the delete.** `map.erase(victim->key)` reads `victim`, so it must run *before* `delete victim`. Reversed, it is a use-after-free that usually appears to work.
- **Omitting the two sentinels.** Without `head` and `tail` sentinels, `unlink` and `pushFront` need null checks on every boundary, and the empty-cache and single-entry cases each need their own branch. The sentinels are why both helpers are four unconditional lines (Topic 41).

An LFU cache is the natural follow-up: a map from frequency to its own DLL, plus a `minFreq` counter. Same principle, one more level of indexing.

LRU cache – get $O(1)$ · put $O(1)$ · space $O(\text{capacity})$ · sentinel-bounded DLL + hash map

78. XOR Linked List

A doubly linked list using **one** pointer field instead of two, by storing the bitwise XOR of the two neighbours' addresses.

node	A	B	C	D
link	$\emptyset \wedge B$	$A \wedge C$	$B \wedge D$	$C \wedge \emptyset$

traverse forward, knowing the PREVIOUS address:
`next = prev XOR curr->link`

at B with `prev = A`: $A \wedge (A \wedge C) = C$ (since $A \wedge A == \emptyset$)

```

struct XNode {
    int data;
    XNode* link;          // prev XOR next, pointer-sized value
};

XNode* XOR(XNode* a, XNode* b) {
    return reinterpret_cast<XNode*>(
        reinterpret_cast<uintptr_t>(a) ^ reinterpret_cast<uintptr_t>(b));
}

void traverse(XNode* head) {
    XNode* prev = nullptr;
    XNode* curr = head;
    while (curr != nullptr) {
        std::cout << curr->data << " ";
        XNode* next = XOR(prev, curr->link);
        prev = curr;
        curr = next;
    }
}

```

KNOW IT; DO NOT SHIP IT

The saving is 8 bytes per node — and Topic 34 showed that on glibc a DLL node usually occupies the same 32-byte chunk as an SLL node anyway, so **the saving is frequently zero in real memory**. Against that:

- **Undefined behaviour in standard C++.** Round-tripping a pointer through arithmetic on `uintptr_t` is only conditionally supported, and the XOR of two valid pointers is not a valid pointer.
- **Breaks garbage collectors and leak detectors.** No stored value is a real address, so tools like ASan and Valgrind cannot follow the list; every node looks unreachable.
- **Breaks debuggers.** You cannot inspect the list in gdb.
- **No random node deletion.** You cannot unlink a node from a pointer to it alone — you need a neighbour, which defeats the main reason to use a DLL.

It remains a good interview *discussion*: it shows you understand that XOR is self-inverse ($a \oplus a = 0$, $a \oplus 0 = a$) and that pointer identity is a language-level concept, not just a number.

XOR list — Space 8 B/node saved (often 0 in practice) · UB in standard C++ · traversal needs the previous node

79. Unrolled Linked List

Store an *array* of values in each node instead of a single value. This directly attacks the two weaknesses established in Section 2: pointer overhead (Topic 9) and cache behaviour (Topic 10).

```
template <int K>
struct UNode {
    int    values[K];
    int    count = 0;           // how many slots are in use
    UNode* next  = nullptr;
};
```

```
plain SLL    [1|•] -> [2|•] -> [3|•] -> ... 32 real bytes per element

unrolled    [1 2 3 4 5 6 7 8 | cnt=8 | •] -> [9 10 ... | cnt=8 | •]
             K = 8 ints (32 B) + count (4) + pad (4) + next (8) = 48 B
             48 / 8 elements = 6 bytes per element    (vs 32)
```

Property	Singly linked	Unrolled (K = 16)
Real bytes per element	32	~5
Cache misses per scan	1 per element	1 per 16 elements
Allocator calls	1 per element	1 per 16
Insert in the middle	O(1) given the node	O(K) — shift within the node
Pointer stability	Permanent	Lost — elements shift inside a node

THE TRADE-OFF, STATED HONESTLY

An unrolled list is a deliberate step back toward the array: it recovers locality and amortises the link, at the cost of $O(K)$ shifting on insertion and the loss of pointer stability. Choosing K so that one node fills one or two cache lines ($K \approx 14$ ints for a 64-byte line) is the usual rule. If you find yourself wanting an unrolled list, check first whether `std::deque` or `std::vector` already solves your problem — they usually do.

unrolled list – $\sim 6\times$ less memory, $\sim 16\times$ fewer misses · insertion $O(K)$ · no pointer stability

80. Skip List

The structure that answers the question Topic 25 left open: a sorted linked list searchable in **$O(\log n)$ expected**, by layering express lanes above the base list.

```
L3  1 -----> 9
L2  1 -----> 4 -----> 9
L1  1 ---> 3 ---> 4 -----> 6 -----> 9
L0  1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 9      (the full sorted list)

search 7:  L3: 1, next is 9 > 7 -> drop
           L2: 1, next is 4 <= 7 -> move to 4; next is 9 -> drop
           L1: 4, next is 6 <= 7 -> move to 6; next is 9 -> drop
           L0: 6, 7 found.      4 comparisons instead of 7.
```

```
struct SNode {
    int    key;
    std::vector<SNode*> forward;      // forward[i] = next node at level i
    SNode(int k, int level) : key(k), forward(level + 1, nullptr) {}
};

int randomLevel(int maxLevel) {      // geometric distribution, p = 1/2
    int lvl = 0;
    while ((rand() & 1) && lvl < maxLevel) ++lvl;
    return lvl;
}

bool search(SNode* head, int maxLevel, int key) {
    SNode* c = head;
    for (int i = maxLevel; i >= 0; --i)
        while (c->forward[i] && c->forward[i]->key < key)
            c = c->forward[i];        // walk right as far as possible
    c = c->forward[0];                // drop to the base list
    return c != nullptr && c->key == key;
}
```

WHY RANDOMNESS, RATHER THAN BALANCING

Each inserted node is promoted to the next level with probability $\frac{1}{2}$, so level i holds about $n/2^i$ nodes and the expected height is $\log_2 n$. That gives $O(\log n)$ expected search **with no rebalancing logic at all** — no rotations, no colour flips. The result is far simpler to implement correctly than a red-black tree and, crucially, much easier to make lock-free, which is why skip lists are used in Redis sorted sets, LevelDB and RocksDB memtables, and `java.util.concurrent.ConcurrentSkipListMap`.

Operation	Expected	Worst case
Search	$O(\log n)$	$O(n)$
Insert / delete	$O(\log n)$	$O(n)$
Space	$O(n) - \sim 2n$ pointers	$O(n \log n)$

skip list – $O(\log n)$ expected search on a linked structure · probabilistic, no rebalancing

81. Static Linked List Using Arrays

Replace pointers with array indices. The list lives in one contiguous block and the "links" are `ints`.

```
const int MAXN = 100005;
int data[MAXN];
int next_[MAXN];          // next_[i] = index of the next node, -1 = end
int head = -1;
int freeIdx = 0;          // simple bump allocator

int newNode(int v) {
    int i = freeIdx++;
    data[i] = v;
    next_[i] = -1;
    return i;
}

void pushFront(int v) {
    int i = newNode(v);
    next_[i] = head;
    head = i;
}

void traverse() {
    for (int i = head; i != -1; i = next_[i])
        std::cout << data[i] << " ";
}
```

WHY THIS IS NOT A CURIOSITY

- **Competitive programming.** No allocator calls at all — the arrays are declared globally and zero-initialised for free. Building 10^6 nodes costs microseconds instead of the 20–100 ms of Topic 11, and the nodes are contiguous, so cache behaviour approaches an array's.
- **Serialisation.** Indices survive being written to disk or sent over a network; pointers do not. Any on-disk linked structure — a filesystem's free list, a database's row chain — uses offsets, which is exactly this.
- **Shared memory.** Two processes map the same region at different virtual addresses, so a stored pointer is meaningless in the other process. An index is identical in both.
- **Embedded systems.** Many safety standards forbid dynamic allocation outright; a fixed-size static list is the only permitted form.

To support deletion, replace the bump allocator with a **free list** — which is itself a static linked list threaded through the unused slots. This closes a loop opened in Topic 11: the structure that manages memory is the structure being studied.

```
int freeHead = 0;                                // thread all slots initially
void initFreeList() {
    for (int i = 0; i < MAXN - 1; ++i) next_[i] = i + 1;
    next_[MAXN - 1] = -1;
}
int acquire()      { int i = freeHead; freeHead = next_[i]; return i; }
void release(int i) { next_[i] = freeHead; freeHead = i; }
```

static linked list – no allocator, contiguous, serialisable · fixed capacity, indices not pointers

PRACTICE SET — Topics 74-81

A. Conceptual (answer in your own words)

1. State the one-line rule for when to use a dummy node. Then list four operations from earlier sections that satisfy it.
2. Why should a dummy be stack-allocated rather than created with `new`? Give a specific code path where the heap version leaks.
3. Distinguish a dummy from a sentinel by lifetime and purpose, and name the standard container built on the second.
4. A tail pointer makes append $O(1)$ but does *not* make delete-tail $O(1)$ in a singly linked list. Explain why, and name the change that would.
5. Name five operations that must update a cached `tail`, and describe the symptom of a stale one.
6. Why can the random pointers of a clone not be set during a single forward pass?
7. Explain how the interleaving method encodes the old→new mapping without a map, in one sentence.
8. Why must an LRU cache use a doubly linked list rather than a singly linked one? Point to the exact operation that fails.
9. In `put`, why must `map.erase` precede `delete victim`?
10. Give three concrete reasons not to ship an XOR linked list, and state how much memory it actually saves for an `int` payload on glibc.
11. An unrolled linked list improves memory and cache behaviour. Name the two properties it gives up.
12. Why does a skip list achieve $O(\log n)$ expected search with no rebalancing logic? Name two production systems that use one.
13. Give four situations where indices are strictly better than pointers for a linked structure.

B. Analysis (pen and paper)

1. Rewrite `deleteFirstOccurrence` (Topic 23) with a dummy and count how many lines and branches disappear.
2. A list caches `head`, `tail` and `size`. For each of `reverse`, `rotateRight`, `partition` and `mergeSort`, state what happens to all three fields.
3. Trace `cloneInterleaved` on `A→B→C` where `A.random = C`, `B.random = A`, `C.random = nullptr`. Give the woven list, the value assigned to each copy's `random`, and both lists after unweaving.
4. Simulate an LRU cache of capacity 2 on: `put(1,1)`, `put(2,2)`, `get(1)`, `put(3,3)`, `get(2)`, `put(4,4)`, `get(1)`, `get(3)`, `get(4)`. Give every return value and the list order after each operation.
5. For an XOR list `A B C D`, give the stored `link` value of every node, then show the arithmetic that recovers `D` when standing at `C` with `prev = B`.
6. An unrolled list with $K = 16$ ints. Compute `sizeof` the node, the real heap chunk, bytes per element, and the ratio against a plain SLL's 32.
7. For the skip list diagram in Topic 80, trace searches for 5, 9 and 10, counting comparisons at each level. Compare with the base-list scan count.
8. With $p = \frac{1}{2}$ and $n = 1\,000\,000$, give the expected number of nodes at levels 0, 1, 10 and 20, and the expected height.

9. A static list has `MAXN = 8` with the free list initialised. Trace three `acquire()` calls, one `release(1)`, then two more `acquire()` calls, giving `freeHead` and the indices returned.

C. Code tasks

1. Take any three functions you wrote in Sections 3 or 9 and rewrite them with dummy nodes. Diff the line counts and the number of `if` statements.
2. Build a `List` class with cached `head`, `tail` and `size`, plus `checkInvariants()`. Implement `reverse` and `rotate` *without* updating the caches first, and confirm the assertions catch both.
3. Implement both clone methods. Verify with an assertion that no copy node's address appears in the original list and that the original is bit-identical afterwards.
4. Implement the LRU cache. Test capacity 1, capacity 0, repeated `put` of the same key, and `get` of a missing key. Then run it under `-fsanitize=address` with the `map.erase` and `delete` lines swapped.
5. Implement an XOR list with `build`, `traverse-forward` and `traverse-backward`. Run it under ASan and report what the tool says about reachability.
6. Implement an unrolled list with $K = 16$. Benchmark a full scan of 10^6 elements against `std::list` and `std::vector`, and report all three.
7. Implement a skip list with `insert` and `search`. Insert 100 000 random keys, then measure the average number of comparisons per search and compare with $\log_2(100\ 000) \approx 17$.
8. Implement a static linked list with a free list. Build 10^6 nodes and time it against the same build using `new`. Report the speedup.

ANSWER KEY — Topics 74-81

A. Conceptual

1. Use a **dummy** whenever the operation can change which node is first. Examples: delete-by-value (23), reverse-sublist (32), remove-n-th-from-end (49), merge two sorted (56), swap-pairs (67), partition (69).
2. A heap dummy needs a matching `delete` on every return path. Any early `return head;` inside a guard clause — or an exception thrown by `new Node` later in the function — leaks it. A stack dummy is destroyed automatically.
3. A **dummy** is local and temporary, existing for one function call to remove that function's head special-case. A **sentinel** is a permanent member of the list, removing *all* boundary cases for the list's whole lifetime. `std::list` is built on a sentinel.
4. Because unlinking the tail requires writing `nullptr` into the **second-to-last** node's `next`, and an SLL cannot reach a node's predecessor without walking from the head. The change that fixes it is adding a `prev` pointer — i.e. a doubly linked list.
5. `pushBack`, `pushFront` on an empty list, `popBack`, `popFront` on a one-node list, delete-by-value, `reverse` (swaps with head), `clear`, and any rearrangement. Symptom: `tail` dangles at freed memory, and the crash appears later inside an unrelated `pushBack`.
6. Because a node's `random` may point *forward* to a node whose copy has not been created yet. You need all copies to exist before any random link can be resolved — hence two passes, or an encoding trick.
7. Because after weaving, the copy of any node `X` is exactly `X->next`, so `X'->random = X->random->next` resolves every link with no external storage.
8. Because `get` must move an *interior* node to the front, which means unlinking it from the pointer the map handed you. In an SLL that needs the predecessor — an $O(n)$ search — so `get` would no longer be $O(1)$. The failing operation is **unlink from a node pointer**.
9. Because `map.erase(victim->key)` reads a field of `victim`. After `delete` that memory is returned to the allocator, so the read is a use-after-free — one that usually appears to work, because the bytes are not yet overwritten.
10. (a) Undefined behaviour: the XOR of two pointers is not a valid pointer, and the round trip through `uintptr_t` is only conditionally supported. (b) Leak detectors and garbage collectors cannot follow the list, so every node looks unreachable. (c) You cannot delete a node given only a pointer to it, which removes the main reason to use a DLL. Real saving on glibc for an `int`: **zero**, because SLL and DLL nodes both land in a 32-byte chunk (Topic 34).
11. (a) **$O(1)$ insertion** — inserting inside a node shifts up to K elements. (b) **Pointer stability** — elements move within a node, so a pointer to an element is no longer permanent.
12. Because each node is promoted with probability $\frac{1}{2}$, so level i holds about $n/2^i$ nodes and the expected height is $\log_2 n$ — the balance is produced by the distribution rather than by rotations. Used in **Redis** sorted sets and **LevelDB/RocksDB** memtables (also `java.util.concurrent.ConcurrentSkipListMap`).
13. (a) Competitive programming — no allocator calls, contiguous, zero-initialised globals. (b) Serialisation — indices survive being written to disk; pointers do not. (c) Shared memory — two processes map the region at different addresses, so only offsets are meaningful. (d) Embedded or safety-critical code where dynamic allocation is forbidden.

B. Analysis

1. The entire `if (head->data == value) { ... }` block disappears — typically **6 lines and 2 branches**, leaving a single uniform loop plus `head = dummy.next;`.
2. **reverse**: head and tail *swap*; size unchanged. **rotateRight**: both head and tail change to new nodes; size unchanged. **partition**: head may change, tail is the last node of the $\geq$ chain — both must be recomputed; size unchanged. **mergeSort**: head almost certainly changes, tail becomes the maximum element; size unchanged. In all four, size is safe and *both* pointers are suspect.
3. Woven: `A A' B B' C C'`. Randoms: `A'.random = A.random->next = C'`; `B'.random = B.random->next = A'`; `C'.random = nullptr`. After unweaving: original `A→B→C` exactly as before, copy `A'→B'→C'` with `A'.random = C'` and `B'.random = A'`. ✓
4. `put(1,1) → [1]`. `put(2,2) → [2,1]`. `get(1) →` returns **1**, list `[1,2]`. `put(3,3) →` evicts 2, list `[3,1]`. `get(2) → -1`. `put(4,4) →` evicts 1, list `[4,3]`. `get(1) → -1`. `get(3) → 3`, list `[3,4]`. `get(4) → 4`, list `[4,3]`.
5. `A: 0 ^ B`. `B: A ^ C`. `C: B ^ D`. `D: C ^ 0`. At C with `prev = B`: `B ^ C.link = B ^ (B ^ D) = (B ^ B) ^ D = 0 ^ D = D`. ✓
6. 16 ints = 64 B, + `count` 4 B + 4 B pad + `next` 8 B = **80**. Chunk rounds to a multiple of 16 → **80 B**. Per element `80/16 = 5 bytes`, against 32 → **6.4× better**.
7. Search 5: L3 drop, L2 → 4, L1 → 4 (next 6 > 5) drop, L0 from 4 → 5. About **4 comparisons** vs 5 on the base list. Search 9: L3 → 9 found in **1** vs 9. Search 10: walks to 9 at every level and falls off — about **5 comparisons**, returns false.
8. L0: 1 000 000. L1: 500 000. L10: ≈ 977 . L20: ≈ 1 . Expected height $\approx \log_2 10^6 \approx 20$.
9. init: `freeHead = 0`, chain `0→1→2→...→7`. `acquire → 0` (`freeHead` 1), `acquire → 1` (`freeHead` 2), `acquire → 2` (`freeHead` 3). `release(1) → next_[1] = 3`, `freeHead = 1`. `acquire → 1` (`freeHead` 3), `acquire → 3` (`freeHead` 4). Freed slots are reused last-in-first-out.

C. Code tasks — what to verify

- **C1**. Expect roughly a 25-40% reduction in branches. If a function got *longer*, you probably kept the old head branch as well as the dummy.
- **C2**. Both assertions must fire. The reverse case is the more instructive: `tail->next == nullptr` fails immediately because tail now points at the new head.
- **C3**. The "original unchanged" assertion is what catches a missing step 3 in the interleaved method — the most common bug in it.
- **C4**. Capacity 0 must not crash. With the two lines swapped, ASan reports `heap-use-after-free READ of size 4` at the `map.erase` line.
- **C5**. ASan and Valgrind report the nodes as **definitely lost** even while the list works correctly, because no stored value is a real address. That output is the practical argument against the structure.
- **C6**. Expect roughly: vector ~ 5 ms, unrolled ~ 15 -25 ms, `std::list` ~ 80 -150 ms. The unrolled list should land much closer to the vector than to the list.
- **C7**. Average comparisons should be on the order of **15-25**, matching the predicted $\sim 2 \cdot \log_2 n$. If you see hundreds, your `randomLevel` is not geometric.
- **C8**. Expect a **10-50×** speedup on build — no allocator calls at all, and the "nodes" are contiguous, so the cache behaves as it would for an array.

HANDNOTE SUMMARY CARD

- 74 DUMMY NODE – USE IT WHENEVER THE FIRST NODE CAN CHANGE
Node dummy(0, head); Node* prev = &dummy;
... uniform loop, every real node now HAS a predecessor ...
return dummy.next;
STACK-ALLOCATE IT. a heap dummy leaks on every early return.
dummy = local, one call sentinel = permanent member (std::list)
- 75 TAIL POINTER – append $O(n) \rightarrow O(1)$, and a maintenance debt
must update on: pushBack · pushFront(empty) · popBack · popFront(1 node)
 delete by value/pos · REVERSE (swaps with head) · clear
 rotate · reorder · partition · sort (recompute)
delete-tail is STILL $O(n)$ in an SLL – needs the predecessor. Use a DLL.
defence: checkInvariants() asserting tail->next == nullptr, size, reach
- 76 CLONE WITH RANDOM POINTER
map method 2 passes, $O(n)$ space, trivial
interleave $O(1)$ space: A A' B B' C C'
 1 weave a copy after each original
 2 X'->random = X->random->next (copy of X IS X->next)
 3 UNWEAVE – restore the original or you mutate the input
- 77 LRU CACHE = HASH MAP + DOUBLY LINKED LIST, both $O(1)$
map: key -> DNode* list: recency order, head = most recent
get : find -> unlink -> pushFront -> return val
put : exists? update+touch. full? evict tail->prev
MUST be doubly linked: unlink from a node pointer alone is $O(n)$ in an SLL
MUST map.erase(victim->key) BEFORE delete victim (use-after-free)
use TWO sentinels -> unlink/pushFront become 4 unconditional lines
- 78 XOR LIST link = prev XOR next; next = prev XOR curr->link
a^a==0, a^0==a. KNOW IT, DO NOT SHIP IT:
UB in standard C++ · invisible to ASan/GC/debuggers · no random delete
real saving for an int on glibc: ZERO (both are 32-byte chunks)
- 79 UNROLLED LIST K values per node
K=16 ints -> 80 B/node -> 5 B/element (vs 32) and 1 miss per 16
GIVES UP: $O(1)$ insertion ($O(K)$ shift) and pointer stability
pick K so a node fills 1-2 cache lines (~14 ints)
- 80 SKIP LIST – the answer to "binary search a linked list"
promote each node with $p=1/2 \rightarrow$ level i has $n/2^i$ nodes -> height $\log n$
search: walk right while next->key < key, else drop a level
 $O(\log n)$ EXPECTED, no rotations, easy to make lock-free
used by Redis sorted sets, LevelDB / RocksDB memtables
- 81 STATIC LIST – indices instead of pointers
int data[N], next_[N]; head = -1 means empty
no allocator · contiguous · SERIALISABLE · works in shared memory
deletion needs a FREE LIST – itself a static linked list (see Topic 11)

CONSOLIDATED CHEATSHEET — PART IV

REARRANGEMENT — WHICH TECHNIQUE EACH PROBLEM NEEDS

63 rotate by k	close the circle, $k \% n$, walk $n-k-1$, cut
64 odd-even position	two chains, guard on `even`, save evenHead
65 even-odd value	two chains, $(x \& 1)$, terminate before join
66 reorder list	MIDDLE + REVERSE + ALTERNATE MERGE (composes 47+29+56)
67 swap in pairs	dummy, 3 writes per pair; == k-groups with $k=2$
68 swap kth both ends	guard $k==1$, $2k-1==n$, adjacency
69 partition	two chains -> STABLE in $O(1)$ (an array needs $O(n)$)
70 dupes sorted	one pass, do not advance after unlinking
71 dupes unsorted	hash $O(n)/O(n)$ · nested $O(n^2)/O(1)$ · sort loses order
72 add two numbers	while (a b carry) — one loop, all three jobs
73 flatten	splice, clear child, fix both prev, guard nxt

THE TWO-CHAIN PATTERN (61, 64, 65, 69)

build two chains by their tails -> TERMINATE the second -> then JOIN
join-before-terminate leaves a stale link => truncation or a CYCLE

DESIGN TOOLKIT — WHEN TO REACH FOR WHAT

need to remove the head special case	-> dummy node	(74)
need $O(1)$ append	-> tail pointer	(75)
need $O(1)$ interior delete from a node pointer	-> doubly linked	(36)
need $O(1)$ lookup AND recency order	-> LRU: map + DLL	(77)
need less memory / better cache	-> unrolled list	(79)
need $O(\log n)$ search on a linked structure	-> skip list	(80)
need no allocator, or serialisable links	-> static list	(81)
need a deep copy with tangled links	-> interleave	(76)

COMPLEXITY QUICK TABLE — PART IV

rotate / odd-even / segregate / partition	$O(n)$ time, $O(1)$ space
reorder / swap pairs / dupes sorted / flatten	$O(n)$ time, $O(1)$ space
dupes unsorted (hash)	$O(n)$ time, $O(n)$ space
add two numbers	$O(\max(m,n))$
clone with random (interleave)	$O(n)$ time, $O(1)$ space
LRU get / put	$O(1)$ both
skip list search / insert	$O(\log n)$ expected

UNIVERSAL CHECKS AFTER ANY STRUCTURAL CHANGE

hasCycle(result) == false · node count unchanged
cached head / tail / size still correct · no dangling child pointers

End of Part Two — Topics 63–81 of 88. Remaining: Section 11, C++ Tools & Safety (82–86), and Section 12, Real-World & Interview (87–88) — seven topics, plus the final consolidated cheatsheet for the whole book. Before continuing, implement the LRU cache from memory: it uses the dummy node, the sentinel, the DLL unlink primitive and a hash map together, and if you can write it cold you have Sections 5, 9 and 10 secured.

PART V

Tools, Safety and Practice

C++ Standard Containers · Bugs & Ownership · Real-World Uses — Topics 82–88

SECTION 11 — C++ TOOLS & SAFETY

82. `std::list`

A **circular doubly linked list with a sentinel node** — exactly the structure built by hand in Topic 41. Knowing that one fact explains almost every part of its interface.

```
#include <list>

std::list<int> l = {3, 1, 2};

l.push_front(0);      // O(1)
l.push_back(4);       // O(1)
l.pop_front();        // O(1)
l.pop_back();         // O(1) <- impossible in an SLL
l.insert(it, 9);       // O(1) given an iterator
l.erase(it);          // O(1) given an iterator
l.size();             // O(1) since C++11
```

The member functions that exist because it is a linked list

```
l.sort();              // merge sort: O(n log n), STABLE, no allocation
l.reverse();           // O(n), relinks — does not move elements
l.unique();            // removes CONSECUTIVE duplicates (sort first)
l.remove(5);           // removes by value, O(n)
l.remove_if(pred);     // removes by predicate
l.merge(other);        // merges two SORTED lists, O(n+m), splices nodes
l.splice(pos, other);  // moves nodes between lists — O(1) for one element
```

WHY THESE ARE MEMBERS AND NOT ALGORITHMS

`std::sort`, `std::reverse` and `std::remove` all work by *moving elements*, which requires random-access iterators or at least element assignment. The list members instead **relink nodes**, so they are cheaper and they preserve node identity — pointers and iterators to elements stay valid. `std::sort(l.begin(), l.end())` does not even compile: a list iterator is bidirectional, not random access.

Iterator invalidation — the guarantee that makes lists worth using

Operation	What is invalidated
insert, push_front, push_back	Nothing. All existing iterators stay valid.
erase, pop_front, pop_back	Only iterators to the erased element.
splice	Nothing — iterators follow the element into the new list.
sort, reverse, merge	Nothing. Elements never move; only links change.

THE SIZE / SPLICE TRADE-OFF

Before C++11, `libstdc++` made `size()` $O(n)$ so that `splice()` of a range could stay $O(1)$ — splicing a sub-range does not reveal how many nodes moved. C++11 mandated $O(1)$ `size()`, so **splicing a sub-range became $O(n)$** . You cannot have both. Splicing a *single element* or an *entire list* is still $O(1)$, because the count is known.

Cost of the convenience: about 32 bytes per `int` element (Topic 9) and one cache miss per element on traversal (Topic 10). For a sequence you mostly iterate, `std::vector` will beat it by 5–20×. Reach for `std::list` when you need **stable references** or **O(1) splicing** — not for "a resizable sequence."

```
std::list – circular DLL + sentinel · O(1) insert/erase/splice · stable iterators · ~32 B per int
```

83. `std::forward_list`

A singly linked list, added in C++11, designed for **zero overhead beyond the nodes themselves**. The container object is a single pointer.

```
#include <forward_list>

std::forward_list<int> f = {1, 2, 3};

static_assert(sizeof(f) == sizeof(void*), "just a head pointer");

f.push_front(0);           // O(1)
f.pop_front();             // O(1)
// f.push_back(4);         // DOES NOT EXIST
// f.size();               // DOES NOT EXIST
// f.back();               // DOES NOT EXIST
```

Everything is `_after`, and that is not an accident

```
auto it = f.begin();
f.insert_after(it, 99);      // insert AFTER it
f.erase_after(it);          // erase the node AFTER it
f.emplace_after(it, 42);

f.insert_after(f.before_begin(), 7); // how you insert at the FRONT
```

TWO DESIGN DECISIONS WORTH UNDERSTANDING

1. Why `_after`? Modifying at a position in a singly linked list requires the *predecessor* (Topic 22), and a forward iterator cannot go back. So the interface asks you for the predecessor directly. `before_begin()` is a fabricated iterator standing one before the head — the standard library's version of the dummy node from Topic 74.

2. Why no `size()`? Maintaining a counter would cost a word per container and a write per mutation, defeating the "no overhead" goal — and O(n) counting was already rejected for `std::list`. Rather than choose, `forward_list` omits it. Use `std::distance(f.begin(), f.end())` if you truly need the count, and note that it is O(n).

	<code>std::forward_list</code>	<code>std::list</code>
Links per node	1	2
Container size	8 bytes	24 bytes (typical)
<code>size()</code>	absent	0(1)
<code>push_back</code> / <code>back()</code>	absent	0(1)
Reverse iteration	impossible	0(n)
Erase a known element	needs the predecessor	0(1)
Use it when	front-only, memory critical	interior edits, bidirectional

`std::forward_list` – SLL, 8-byte container · `_after` interface, `before_begin()` · no size, no back

84. Smart Pointers in Linked Lists

`std::unique_ptr` expresses node ownership exactly: each node owns the next one, and destroying the head destroys the whole chain. It removes every `delete` from your code — and introduces one serious trap.

```
struct Node {
    int data;
    std::unique_ptr<Node> next;          // owns its successor
    explicit Node(int d) : data(d) {}
};

class List {
    std::unique_ptr<Node> head;
public:
    void pushFront(int v) {
        auto n = std::make_unique<Node>(v);
        n->next = std::move(head);      // transfer ownership
        head = std::move(n);
    }
    void print() const {
        // observe with a RAW pointer
        for (const Node* c = head.get(); c; c = c->next.get())
            std::cout << c->data << " ";
    }
};
```

THE TRAP: THE DEFAULT DESTRUCTOR IS RECURSIVE

Destroying `head` destroys `head->next`, which destroys `head->next->next` ... one live stack frame per node. A list of ~100 000 nodes **overflows the stack on destruction** — the same failure as the naive `~Node() { delete next; }` in Topic 24, now hidden inside a "safe" abstraction. Always write the destructor by hand:

```
~List() {
    while (head) head = std::move(head->next);    // iterative unwind
}
```

Doubly linked lists: **shared_ptr** forward, **weak_ptr** back

```
struct DNode {
    int data;
    std::shared_ptr<DNode> next;    // owns
    std::weak_ptr<DNode> prev;    // observes – must NOT own
};
```

WHY **prev** MUST BE WEAK

If both directions were **shared_ptr**, every adjacent pair would hold a reference to the other, so neither refcount ever reaches zero: **the entire list leaks**, silently and without any raw pointer in sight. **weak_ptr** observes without owning, breaking the cycle. This is the canonical example of shared-ownership cycles, and interviewers ask it precisely because "I used smart pointers" is not the same as "I have no leaks."

Choice	Cost per node	Use for
Raw Node*	8 B	Interview code; intrusive/arena-managed lists
unique_ptr	8 B, zero overhead	Single-owner SLL — the default choice
shared_ptr	16 B + control block, atomic refcount	Genuine shared ownership only

smart pointers – **unique_ptr** for SLL (write an iterative destructor) · **weak_ptr** for **prev**

85. Common Bugs

Nearly every linked-list failure is one of these eight. All of them compile cleanly.

Bug	What it looks like	Symptom & fix
Lost head	<pre>void f(Node* head) { head = new ...; }</pre>	Caller unchanged, node leaked. Take Node*& or return the new head.
Self-loop	<pre>head = n; n->next = head;</pre>	Infinite traversal, whole list leaked. Attach before detaching.
Use after free	<pre>delete p; p = p->next;</pre>	Usually "works". Save next first: save → unlink → delete.
Null deref	<pre>while (c->next)</pre>	Crash on the empty list. Guard if (!head) return; first .
Guard order	<pre>fast->next && fast</pre>	Crash on every even-length list. Test fast first.
Advance after unlink	<pre>unlink(); prev = prev->next;</pre>	Skips a node. Fails on [3,3,3] . Do not advance.
Unterminated chain	<pre>join before terminate</pre>	Creates a cycle . Set tail->next = nullptr first.
Stale cache	<pre>reverse() without swap(head, tail)</pre>	Dangling tail ; crash later in an unrelated call.

THE THREE QUESTIONS THAT PREVENT ALL EIGHT

1. Can this change which node is first? → `Node*&`, a returned head, or a dummy.
2. Do I need the predecessor? → stop one early, or use a dummy.
3. Am I removing a node? → save `next`, then unlink, then delete.

They were stated at the opening of Section 3 and they have not changed in eighty topics.

Debugging tools that actually find these

```
g++ -O1 -g -fsanitize=address,undefined prog.cpp // use-after-free, leaks
valgrind --leak-check=full ./a.out                // leaks, invalid reads
```

Neither tool detects a *cycle*, and a cycle hangs rather than crashing. Write `hasCycle()` (Topic 50) into your test harness and call it after every structural change — it is three lines and it catches the one class of bug the sanitisers miss.

common bugs – all eight compile cleanly · ASan + a `hasCycle` assertion catches nearly all

86. Memory Leaks and Double Free

The correct teardown, once more

```
void clear(Node*& head) {
    while (head) {
        Node* nxt = head->next; // 1. SAVE
        delete head;           // 2. FREE
        head = nxt;            // 3. ADVANCE
    }
}                             // head is nullptr – a valid empty list
```

The Rule of Three — skip it and you get a double free

```
class List {
    Node* head = nullptr;
public:
    ~List() { clear(head); }

    List(const List& other) {                // deep copy
        Node dummy(0);
        Node* tail = &dummy;
        for (Node* c = other.head; c; c = c->next) {
            tail->next = new Node(c->data);
            tail = tail->next;
        }
        head = dummy.next;
    }

    List& operator=(const List& other) {
        if (this == &other) return *this;    // SELF-ASSIGNMENT GUARD
        List tmp(other);                     // copy-and-swap
        std::swap(head, tmp.head);
        // tmp's destructor frees the old nodes
        return *this;
    }

    List(List&& other) noexcept               // move: steal, do not copy
        : head(other.head) { other.head = nullptr; }
};
```

WHAT THE COMPILER-GENERATED COPY DOES

It copies the **head** pointer. Now two **List** objects believe they own the same nodes, and when both destructors run, every node is **freed twice** — heap corruption, and the crash is usually far from the copy. If you cannot write a correct deep copy, delete the copy operations explicitly:

```
List(const List&) = delete;
List& operator=(const List&) = delete;
```

A compile error is infinitely better than a double free.

Leak versus double free — the two failure directions

	Memory leak	Double free
Cause	Node unlinked but never deleted; head overwritten	Shallow copy; deleting the same node twice
Symptom	Memory grows; program still correct	Heap corruption; crash, often much later
Detected by	Valgrind, ASan leak check	ASan, glibc "double free or corruption"
Severity	Bad	Worse — undefined behaviour

THE RELIABLE ANSWER

Let RAII own the nodes. A `unique_ptr`-based list (Topic 84) with a hand-written iterative destructor cannot leak and cannot double-free, because ownership is single and explicit in the type. Raw `new/delete` is for interview whiteboards and for allocators; in production code the ownership should be visible in the declaration.

leaks & double free – save/free/advance · Rule of Three or `= delete` · prefer RAII ownership

SECTION 12 — REAL-WORLD & INTERVIEW

87. Real-World Use Cases

Linked lists rarely appear as "a list of things." They appear where **pointer stability**, **O(1) splicing**, or **no allocator** are hard requirements.

Where they genuinely earn their place

System	Variant	Why a linked list
LRU / LFU caches	DLL + hash map	O(1) unlink of an interior node from a stored pointer (Topic 77)
Undo / redo stacks	DLL	Move a cursor both ways; truncate the future in O(1)
Browser history	DLL	Back and forward from an interior position
Music playlist, repeat mode	Circular DLL	Wrap-around with no end-of-list branch
Round-robin schedulers	Circular SLL	<code>curr = curr->next</code> forever; no index arithmetic
Hash table chaining	SLL	Front insertion only; <code>prev</code> would be wasted
Memory allocators	SLL free list	The list lives <i>inside</i> the free blocks — zero overhead
Linux kernel	Intrusive circular DLL	<code>list head</code> embedded in the object; no allocation, no null checks
Filesystems (FAT)	Static list (indices)	Cluster chains on disk; pointers are meaningless there
Redis sorted sets	Skip list	O(log n) ordered queries, easy to make lock-free (Topic 80)
LevelDB / RocksDB	Skip list memtable	Concurrent inserts without rebalancing
Graph adjacency lists	SLL or vector	Per-vertex chains of variable length
Polynomials, big integers	SLL	Sparse terms; digit-wise carry (Topic 72)
Blockchains	Hash-linked list	Each block names its predecessor by hash rather than address

THE PATTERN ACROSS ALL OF THEM

Notice how few are "a sequence of values." Almost every real use is one of three shapes: something already holds a pointer to an element and must move or remove it (*caches, kernel lists*); the structure must wrap around forever (*schedulers, playlists*); or the links must survive outside memory (*filesystems, blockchains*). If your use case is none of these, `std::vector` is very likely the better answer.

THE HONEST CAVEAT

Modern high-performance code uses linked lists *less* than textbooks suggest, because cache behaviour dominates (Topic 10) and `std::vector` wins most benchmarks — including middle insertion at surprisingly large *n*. Being able to say *when not to use one* is a stronger signal in an interview than reciting the advantages.

real-world — pointer stability · $O(1)$ splice · wrap-around · off-memory links. Otherwise: use a vector.

88. Final Cheatsheet

The pattern-recognition table — read the problem, pick the tool

The problem says...	Reach for
"middle", "half", "median position"	Slow/fast pointers (47)
"n-th from the end", "remove from the end"	Gap of n (or $n+1$) (48, 49)
"cycle", "loop", "does it terminate"	Floyd (50–53)
"palindrome"	Middle + reverse half + restore (54)
"intersection", "merge point"	Pointer switch, compare addresses (55)
"the head might change"	Dummy node (74)
"delete", "remove", "unlink"	Save → unlink → delete; you need the predecessor (22)
"in groups of k ", "in pairs", "alternate"	Bounded three-pointer reversal (31, 33, 67)
"partition", "segregate", "group by"	Two chains, terminate then join (61, 64, 65, 69)
"sort a linked list"	Merge sort — first middle, then cut (57)
" k sorted lists"	Heap or divide-and-conquer, $O(N \log k)$ (60)
" $O(1)$ get and put", "least recently used"	DLL + hash map (77)
"deep copy", "random pointer"	Interleave, then unweave (76)
"reorder", " $L_0 L_n L_1 \dots$ "	Middle + reverse + alternate merge (66)
"without extra space"	Two pointers; never a hash set
"search a sorted list faster than $O(n)$ "	Impossible — you need a skip list (80)

The four templates worth having in muscle memory

```
// 1. TRAVERSE
for (Node* c = head; c != nullptr; c = c->next) { /* visit c */ }

// 2. UNLINK (the deletion primitive)
Node* doomed = prev->next;
prev->next = doomed->next;
delete doomed;

// 3. REVERSE (the three-pointer loop)
Node* prev = nullptr;
Node* curr = head;
while (curr) {
    Node* nxt = curr->next; // SAVE before you FLIP
    curr->next = prev;
    prev = curr;
    curr = nxt;
}
return prev;

// 4. SLOW / FAST
Node* slow = head;
Node* fast = head; // head->next for the FIRST middle
while (fast && fast->next) { // guard order matters
    slow = slow->next;
    fast = fast->next->next;
}
```

The test matrix — run every one of these, every time

```
[] · [x] · [x,y] · operate on HEAD · operate on TAIL
target not found · index out of range · all-equal [3,3,3]
k = 0 · k = 1 · k = n · k > n · odd n · even n
```

```
after any structural change, assert THREE things:
    hasCycle(result) == false
    node count unchanged
    cached head / tail / size still correct
```

IF YOU REMEMBER ONLY ONE PAGE OF THIS BOOK

Make it the three questions. *Can the first node change? Do I need the predecessor? Am I removing a node?* Those three cover the dummy node, the stop-one-early walk, and the save-before-delete ordering — which together account for essentially every linked-list bug that compiles.

88 topics · 12 sections · the whole book reduces to 4 templates and 3 questions

PRACTICE SET — Topics 82-88

A. Conceptual (answer in your own words)

1. `std::list` is a circular doubly linked list with a sentinel. Name three visible parts of its interface that this fact explains.
2. Why does `std::sort(l.begin(), l.end())` fail to compile for a `std::list`, and what does `l.sort()` do instead?
3. Explain the $O(1)$ `size()` versus $O(1)$ range-`splice()` trade-off, and state what C++11 decided.
4. Which `std::list` operations invalidate iterators, and why is that guarantee the main reason to choose the container?
5. Why is every mutating member of `std::forward_list` named `_after`? What is `before_begin()` for, and which hand-written idiom does it correspond to?
6. Give two reasons `std::forward_list` has no `size()`.
7. A `unique_ptr`-based list removes every `delete` from your code. Describe the bug it introduces and the exact fix.
8. In a `shared_ptr` doubly linked list, why must `prev` be a `weak_ptr`? Describe precisely what happens if it is not.
9. List the eight common bugs from Topic 85 and state, for each, whether ASan would catch it.
10. Which class of linked-list bug do neither ASan nor Valgrind detect, and what should you put in your test harness instead?
11. What does the compiler-generated copy constructor do to a `List` holding a raw `Node* head`, and what is the resulting failure called?
12. Why is `= delete` on the copy operations a legitimate engineering answer rather than a cop-out?
13. Across the real-world uses in Topic 87, name the three recurring shapes that justify a linked list.
14. Give the strongest one-sentence argument *against* using a linked list, and the strongest argument for.

B. Analysis and short answers

1. For each operation — `push_back`, `splice` one element, `splice` a sub-range, `sort`, `size`, `reverse` — give the complexity for `std::list` and say whether `std::forward_list` supports it at all.
2. Compute the memory for 1 000 000 `ints` in `std::vector`, `std::forward_list` and `std::list` on 64-bit glibc, and give the ratios.
3. Write the four-line iterative destructor for a `unique_ptr` list and explain in one sentence why the default one is unsafe.
4. Draw the reference-count situation for three nodes in a `shared_ptr`-both-ways doubly linked list after the head pointer goes out of scope. Give each node's count.
5. For each of the eight bugs in Topic 85, write the one-line fix from memory.
6. A `List` is passed by value into a function that returns immediately. With only a compiler-generated copy constructor and a destructor, state exactly what happens and when.
7. Given the pattern-recognition table in Topic 88, name the technique for: "return the list with every second node removed"; "detect whether two lists share a suffix"; "rotate a playlist so the current song is first".
8. Write out the four templates from Topic 88 from memory, then check them against the page.

C. Code tasks

1. Write a program that inserts 5 elements into a `std::list`, saves an iterator to the third, then performs `push_front`, `sort`, `reverse` and `splice` — asserting after each that the saved iterator still dereferences to the same value.
2. Implement the same small task with `std::list` and `std::forward_list`: build 1 000 000 elements, sum them, then erase every element equal to 7. Report both times and both memory footprints.
3. Build a `unique_ptr` list of 1 000 000 nodes with the *default* destructor and run it. Record the crash. Add the iterative destructor and confirm it succeeds.
4. Build a doubly linked list with `shared_ptr` in both directions, run it under `valgrind --leak-check=full`, and report the bytes lost. Then change `prev` to `weak_ptr` and re-run.
5. Write a `List` class with a raw `head`, a destructor, and *no* copy constructor. Pass it by value to a function and run under ASan. Paste the first line of the report. Then fix it with the Rule of Three and re-run.
6. Write a reusable test harness: `hasCycle`, `count`, `isSorted` and `checkInvariants`. Run it against every function you have written in this book.
7. Implement an undo/redo stack over a `std::list` with a cursor iterator, supporting `do`, `undo`, `redo`, and truncation of the redo branch on a new action.

ANSWER KEY — Topics 82-88

A. Conceptual

1. (a) `end()` is a real, decrementable iterator — it points at the sentinel. (b) `--end()` legally gives the last element. (c) `insert` and `erase` need no boundary branches, so they are unconditional $O(1)$. (Also: `pop_back` exists at all, which requires `prev`.)
2. Because `std::sort` requires **random-access iterators** and a list iterator is only bidirectional. `l.sort()` performs a **merge sort by relinking nodes** — $O(n \log n)$, stable, no allocation, and it preserves node identity so iterators stay valid.
3. Splicing a sub-range does not reveal how many nodes moved, so you can have an $O(1)$ counter or an $O(1)$ range splice, not both. **C++11 mandated $O(1)$ `size()`**, which made range-splice $O(n)$. Single-element and whole-list splices remain $O(1)$.
4. Only `erase` (and `pop_front/pop_back`) invalidate — and only iterators to the removed element. Insert, splice, sort, reverse and merge invalidate **nothing**. That stability is why an LRU cache can store node pointers, and it is the property a `vector` cannot offer.
5. Because modifying at a position in a singly linked list needs the **predecessor**, and a forward iterator cannot move backwards — so the API takes the predecessor directly. `before_begin()` is a fabricated iterator one before the head, making front insertion uniform; it corresponds to the **dummy node** idiom (Topic 74).
6. (a) A counter costs a word per container and a write per mutation, defeating the zero-overhead design goal — `sizeof(forward_list)` is exactly one pointer. (b) An $O(n)$ `size()` had already been rejected for `std::list`, so rather than pick a bad option it omits the function.
7. The **default destructor is recursive**: destroying the head destroys `next`, which destroys `next->next`, one stack frame per node — a stack overflow at ~100 000 nodes. Fix: `~List() { while (head) head = std::move(head->next); }`.
8. Because two `shared_ptr`s pointing at each other form a reference cycle: each adjacent pair keeps the other's count at 1, so no count ever reaches zero and **the entire list leaks** — with no raw pointer anywhere in the code. `weak_ptr` observes without incrementing the count, breaking the cycle.
9. Lost head — no (a leak; the ASan leak checker catches the leak but not the logic). Self-loop — no. Use after free — **yes**. Null deref — **yes**. Guard order — **yes** (null deref). Advance after unlink — no (silently wrong output). Unterminated chain — no (hangs). Stale cache — **yes**, once the dangling pointer is dereferenced. So ASan catches four of eight.
10. **Cycles**. A cycle causes an infinite loop rather than an invalid memory access, so neither tool fires. Put a `hasCycle()` assertion (Topic 50) in the harness and call it after every structural change.
11. It copies the `head` pointer, so two objects believe they own the same nodes. When both destructors run every node is freed twice — a **double free**, i.e. heap corruption and undefined behaviour.
12. Because a **compile error is strictly better than a double free**. Deleting the copy operations states the ownership model honestly and moves the failure from run time to build time. If copying is genuinely needed, implement it properly; if it is not, forbid it.
13. (a) Something outside already holds a pointer to an element and must move or remove it (caches, kernel lists). (b) The structure must wrap around indefinitely (schedulers, playlists). (c) The links must survive outside process memory (filesystems, blockchains).
14. **Against**: cache behaviour dominates on modern hardware, so `std::vector` wins most real benchmarks — including middle insertion at surprisingly large n — despite the Big-O table.

For: a node's address never changes, so external pointers stay valid and splicing is $O(1)$ no matter how many elements move.

B. Analysis and short answers

1. `push_back` $O(1)$ / absent. `splice` one element $O(1)$ / `splice_after` $O(1)$. `splice` sub-range $O(n)$ / $O(n)$. `sort` $O(n \log n)$ / supported. `size` $O(1)$ / absent. `reverse` $O(n)$ / supported.
2. `vector`: 4 MB (100%). `forward_list`: node = int + pad + 1 pointer = 16 B $\rightarrow$ 32 B chunk $\rightarrow$ **32 MB**. `list`: node = int + pad + 2 pointers = 24 B $\rightarrow$ **32 B chunk $\rightarrow$ 32 MB**. Ratios **8 $\times$** and **8 $\times$** — identical, because both land in the same chunk size (Topic 34).
3. `~List() { while (head) head = std::move(head->next); }` — the default is unsafe because destruction chains recursively through `next`, one live stack frame per node.
4. A $\rightarrow$ B $\rightarrow$ C with shared both ways. After the external head pointer dies: A's count is 1 (held by B's `prev`), B's is 2 (A's `next`, C's `prev`), C's is 1 (B's `next`). Nothing reaches zero — **all three leak**.
5. Take `Node*&` or return the head · attach before detaching · save `next` before deleting · guard the empty list first · test `fast` before `fast->next` · do not advance after unlinking · terminate the chain before joining · swap `head` and `tail` after reversing.
6. The copy shares `head`. The parameter is destroyed at function exit, freeing every node; the caller's object is now a chain of dangling pointers, and its own destructor at scope exit frees them **a second time** — a double free, plus use-after-free for anything in between.
7. "Every second node removed" $\rightarrow$ odd-even position split, keep one chain (64). "Two lists share a suffix" $\rightarrow$ intersection by pointer switch, comparing addresses (55). "Rotate a playlist" $\rightarrow$ close the circle and cut (63) — and if the playlist is already circular, just move the `last` handle.
8. Self-check against Topic 88. The most commonly mis-remembered detail is the guard order in template 4 and the `SAVE` line in template 3.

C. Code tasks — what to verify

- **C1.** Every assertion passes. That is the point: no `std::list` operation in that sequence invalidates the iterator, which is exactly what a `vector` cannot promise.
- **C2.** Expect similar times; `forward_list` uses the same 32 MB as `list` on glibc but has a smaller container object and no `size()`. The erase is where `forward_list` is awkward — you need `before_begin()` and `erase_after`.
- **C3.** Segfault from stack exhaustion at destruction, typically 100 000–250 000 frames. The iterative destructor handles 10^6 without difficulty.
- **C4.** Valgrind reports the full list as *definitely lost*. With `weak_ptr` for `prev`, zero bytes lost.
- **C5.** ASan reports `attempting double-free` (or `heap-use-after-free`) with two matching free stacks. After adding the deep copy and self-assignment guard, clean.
- **C6.** Any function that fails `hasCycle` or the node count is broken even if its printed output looks right — that is the whole reason to build the harness.
- **C7.** The truncation step is `list.erase(cursor, list.end())` after a new action; the cursor's stability across that operation is precisely why `std::list` is the right container here.

HANDNOTE SUMMARY CARD

- 82 `std::list` = CIRCULAR DLL + SENTINEL (exactly Topic 41)
O(1) push/pop front and back · insert/erase at an iterator · size()
splice of ONE element or a WHOLE list
O(n) splice of a SUB-RANGE (the price of O(1) size, decided in C++11)
members that RELINK instead of moving: sort (merge, stable) · reverse
unique (CONSECUTIVE dupes) · remove · remove_if · merge · splice
`std::sort` DOES NOT COMPILE – list iterators are bidirectional
INVALIDATION: only erase, and only the erased element. everything else
keeps every iterator valid <- the real reason to use it
- 83 `std::forward_list` = SLL, container is ONE POINTER (8 bytes)
no size() · no back() · no push_back · no reverse iteration
everything is _after: insert_after · erase_after · emplace_after
because modifying needs the PREDECESSOR and you cannot go back
before_begin() == the standard library's dummy node
- 84 SMART POINTERS
unique_ptr next single ownership, zero overhead, DEFAULT
TRAP: default destructor RECURSES -> stack overflow at ~100k nodes
FIX: ~List() { while (head) head = std::move(head->next); }
DLL: shared_ptr next + WEAK_PTR prev
both shared => every pair keeps the other alive => WHOLE LIST LEAKS
- 85 THE EIGHT BUGS – ALL OF THEM COMPILE
- | | | |
|----------------|-------------------------|-------------------------------|
| lost head | Node* by value | -> Node*& or return the head |
| self-loop | head=n; n->next=head | -> attach BEFORE detaching |
| use-after-free | delete p; p=p->next | -> save next FIRST |
| null deref | while (c->next) | -> guard the empty list |
| guard order | fast->next && fast | -> test fast FIRST |
| skip a node | advance after unlinking | -> do NOT advance |
| cycle | join before terminate | -> tail->next = nullptr first |
| stale cache | reverse without swap | -> swap(head, tail) |
- ASan catches 4 of 8. IT NEVER CATCHES A CYCLE – a cycle HANGS.
put hasCycle() in the harness.
- 86 LEAKS AND DOUBLE FREE
teardown: save next -> delete -> advance -> head = nullptr
RULE OF THREE: dtor + deep copy ctor + copy assign (self-assign guard)
default copy shares head => BOTH destructors free every node => DOUBLE FREE
cannot write a correct deep copy? = delete the copy operations.
a COMPILE ERROR beats a double free.
- 87 WHERE THEY ACTUALLY LIVE
LRU/LFU (DLL+hash) · undo-redo · browser history · playlists (CDLL)
round-robin (CSLL) · hash chaining (SLL) · allocator free lists
kernel list_head (intrusive CDLL) · FAT cluster chains (indices)
Redis zsets / RocksDB memtable (skip list) · adjacency lists · blockchain
three shapes justify one: held pointers · wrap-around · off-memory links
everything else: use std::vector.

MASTER CHEATSHEET — ALL 88 TOPICS

THE THREE QUESTIONS (they answer nearly every bug in this book)

- 1 Can the FIRST node change? -> Node*& head · return the head · DUMMY
- 2 Do I need the PREDECESSOR? -> stop one early · while (c->next) · DUMMY
- 3 Am I REMOVING a node? -> SAVE next -> UNLINK -> DELETE

THE FOUR TEMPLATES

```
TRAVERSE for (Node* c = head; c; c = c->next)
UNLINK   Node* d = prev->next; prev->next = d->next; delete d;
REVERSE  nxt = c->next; c->next = prev; prev = c; c = nxt; return prev;
SLOW/FAST while (fast && fast->next) { slow++; fast += 2; }
          fast = head          -> SECOND middle
          fast = head->next -> FIRST middle (merge sort, palindrome)
```

MEMORY (64-bit, glibc)

SLL node 16 B -> 32 chunk
DLL node 24 B -> 32 chunk
SAME COST. prev is free.
1e6 ints: vector 4 MB
list 32 MB
cache: 1 miss per NODE vs
1 per 16 array ints
dependent loads => no prefetch

COMPLEXITY — SINGLY LINKED

index / search 0(n)
insert head 0(1)
insert tail no tail* 0(n) · tail* 0(1)
insert/delete at a KNOWN ptr 0(1)
DELETE TAIL 0(n) even with tail*
binary search IMPOSSIBLE
length (cached) 0(1)
DLL: delete known node 0(1)

SECTION MAP

- 1-6 foundation · node = data + link · the list IS the head pointer
- 7-11 heap per node · non-contiguous · 32 B real · cache · pool alloc
- 12-28 create/build/traverse/insert/delete/search/length/update/nth
- 29-33 reverse: whole · recursive · k-groups · sublist(pull-to-front)
· alternate-k (skip k-1, NOT k)
- 34-39 DLL: 4 writes to insert · 2 to unlink · 0(1) from the node alone
- 40-45 circular: hold `last` not `head` · DO-WHILE · Josephus · split
- 46-55 two-pointer: middle · gap n / n+1 · Floyd L = nC - k · palindrome
· intersection (compare ADDRESSES, switch lists)
- 56-62 merge two (dummy + one-write tail) · MERGE SORT is the right sort
· quicksort 3-way but 0(n^2) on sorted · merge k = 0(N log k)
- 63-73 rotate (close the circle) · two-chain pattern (terminate, THEN
join) · reorder = middle+reverse+merge · dupes · add numbers
(while a||b||carry) · flatten
- 74-81 dummy · tail cache (audit EVERY mutator) · clone by interleaving
· LRU = DLL + hash · XOR (know, don't ship) · unrolled · skip
list · static list (indices, serialisable)
- 82-88 std::list (circular DLL+sentinel) · forward_list (_after)
· unique_ptr (iterative dtor!) · weak_ptr for prev · 8 bugs
· Rule of Three · real-world · this page

PATTERN RECOGNITION

"middle/half"	slow-fast	"head might change"	dummy
"nth from end"	gap n	"in pairs/groups"	bounded rev
"cycle/loop"	Floyd	"partition/group by"	two chains
"palindrome"	mid+rev+RESTORE	"sort a list"	merge sort
"intersection"	pointer switch	"k sorted lists"	heap / D&C
"0(1) get and put"	DLL + hash map	"deep copy random"	interleave
"no extra space"	two pointers	"search faster"	skip list

TEST MATRIX — RUN ALL OF IT, EVERY TIME

[] · [x] · [x,y] · head · tail · not found · out of range
[3,3,3] · k=0,1,n,>n · odd n · even n · single-node circle
THEN ASSERT: no cycle · node count unchanged · caches still correct

AFTERWORD — WHERE TO GO NEXT

Eighty-eight topics, twelve sections, and it reduces to three questions and four templates. That compression is the real result of working through the book — not the individual functions, which you can look up, but knowing which of four shapes a new problem is.

What to do in the next two weeks

1. **Implement the LRU cache from memory.** It uses the dummy node, the sentinel, the DLL unlink primitive and a hash map at once. If you can write it cold, Sections 5, 9 and 10 are secure.
2. **Build the test harness** from Practice Set 9 — `hasCycle`, `count`, `isSorted`, `checkInvariants` — and run every function you have written against it. Expect failures; that is the point.
3. **Solve 40-50 problems**, weighted toward Sections 7 and 8, which carry the most interview weight. Reversal, Floyd, merge sort and the two-chain pattern between them cover most of what gets asked.
4. **Write the four templates by hand once a day for a week.** The guard order in slow/fast and the `SAVE` line in reverse are the two details that decay first.

What this unlocks

Trees are linked lists with two `next` pointers, and every traversal you learned here reappears there. Graphs are adjacency lists — linked lists of linked lists. Hash tables use chaining. Heaps, tries and B-trees are all pointer structures with the same ownership and null-boundary questions. The next data structure in Phase 1 will feel substantially easier than this one did, because the hard part was never the linked list — it was learning to reason about pointers, ownership and edge cases at the same time.

THE LAST THING WORTH SAYING

The most valuable sentence in this entire book is the one about when *not* to use a linked list. Being able to explain that `std::vector` usually wins on modern hardware — and exactly why, in terms of cache lines and dependent loads — demonstrates something that reciting complexity tables never will.

End of the book — Topics 1-88 complete. Mark your tracker, then move on to Data Structure 3 of 12.

